Crosslink Guide

Trailing finality for Zcash: the construction, its formal models, and the wallet-visible contract

m@rek.onl

Abstract

Assuming the best-chain consensus model of the *Consensus Guide*, the header-chain commitment interface of the *FlyClient Guide*, the Orchard protocol of the *Ironwood Guide*, the signature foundations of the *Crypto Guide*, and the wallet layer of the *Wallet Guide*, this volume of *The Zcash Arboretum* documents Crosslink, the trailing-finality construction targeted at Zcash: the ebb-and-flow model and what its theorems actually prove, the construction as the Trailing Finality Layer book specifies it, stake and delegation as far as any source pins them down, and the wallet-visible contract two ledgers create. The design is in motion and its sources disagree — the implementation drops mechanisms the book calls central, and the machine-checked model formalises a neighbouring rule set — so every claim carries its classification, every divergence is stated rather than resolved, and the questions a future ZIP must answer are listed as such.

Contents

1 Scope, status, and sources

Crosslink is a hybrid proof-of-work/BFT (Byzantine-fault-tolerant) consensus construction for Zcash: it retains the existing best-chain (PoW) protocol and adds a trailing finality layer (TFL), a BFT protocol (one that reaches agreement by explicit votes and remains correct while at most a bounded fraction of its participants misbehave arbitrarily) whose blocks and the best-chain blocks commit to one another — the eponymous cross-links. The current version is Crosslink 2 (tfl-book @ `fe6e1d6`, `src/design/crosslink.md` lines 1–3; `the-arguments-for-bounded-availability-and-finality-overrides.md` line 9). The construction descends from the ebb-and-flow security model and the Snap-and-Chat construction of Neu, Taş, and Tse ("Ebb-and-Flow Protocols: A Resolution of the Availability–Finality Dilemma", eprint 2020/1091; arXiv 2009.04987, v1 2020-09-10, v3 2021-02-04; published IEEE S&P 2021). Snap-and-Chat's two-ledger architecture Crosslink keeps; its show-stopping defect for Zcash — users cannot in general spend outputs of finalized transactions taken from a rolled-back fork's snapshot, because chain validity in Π_{lc}, Snap-and-Chat's longest-chain sub-protocol, cannot depend on a node's local finalized ledger — is what the TFL book records as forcing the redesign: "We are forced to conclude that the chain ch_i^t must include the information needed to calculate $\mathsf{LOG}_{\mathrm{fin}}$" (tfl-book @ `fe6e1d6`, `notes-on-snap-and-chat.md` lines 73–107). No successor paper covering Crosslink itself has appeared (arXiv and IACR ePrint searches checked 2026-09-05).

No Crosslink consensus change is activated on Zcash Mainnet or public Testnet, although the separate feature net runs prototype code. Nothing is consensus-specified: the official ZIP repository at `e753a6a` (2026-09-03) contains no Crosslink or TFL ZIP and no ZIP specifying NU8. Two public pre-editor drafts did appear in the Shielded Labs ZIP fork at commit `7cdceca` (2026-05-27), but are absent from that fork's current `main` and from the official repository; they are not normative. The earlier implementation book says its own draft was a Google document that would enter the formal process "as the prototype approaches maturity" (zebra-crosslink @ `6d02a1b`, `book/src/design.md` line 7). This is therefore a design-stage volume: it fixes a moving design against commit-pinned sources, classifies every claim by rigor, and asserts nothing normatively. The design corpus splits three ways — a construction book that has been static since 2024-12-13, an implementation whose design has since moved past the book in documented ways, and a machine-checked model of one narrow slice — and the first duty of this section is to say which document is authoritative for what.

1.1 The best-chain protocol as a black box

Legacy consensus internals are the *Consensus Guide*'s subject, and this volume does not re-derive them for proof of work. Equihash, difficulty adjustment, and block relay never appear here; the best-chain protocol enters only through the interface Crosslink's model assumes of any Π_{bc} (tfl-book @ `fe6e1d6`, `construction.md` lines 245–281): a set of chains rooted at a genesis block; a bc-block-validity rule depending only on the block and its ancestors; block production hard unless selected in proportion to resources; a score function strictly increasing along a chain (accumulated work, for PoW); honest nodes adopting a highest-scoring valid chain; and headers that commit to blocks, with header validity (PoW and difficulty adjustment) checkable before block download. The book's own assessment is that "Typically, Bitcoin-derived best chain protocols do not need much adaptation to fit into this model" (`construction.md` line 281).

The safety assumptions Crosslink places on this black box are Prefix Consistency at confirmation depth σ (PSS2016's "consistency") and Prefix Agreement at depth σ, both stated as unconditional properties of executions; the probabilistic reasoning that justifies them is deliberately separated out, with NTT2020 Theorem 2 supplying the corresponding bound $e^{-\Omega(\sqrt{\sigma})}$ on rollback failure (tfl-book @ `fe6e1d6`, `construction.md` lines 285–374, 351–357). We state these assumptions formally in the construction section and use them as the interface; we never derive them from PoW internals. Where the interface touches deployed reality we cite the deployed code: Zebra bounds its best non-finalized chain at `MAX_BLOCK_REORG_HEIGHT` = 1000 blocks, and its own comment concedes that "deeper

reorganisations are outside Zebra's rollback window. This is a local-only node policy; it is not part of consensus" (zebra @ `d1fd7ad`, `zebra-chain/src/parameters/constants.rs` lines 20–30) — the standing admission that today's Zcash has no consensus-level finality at any depth.

1.2 The three bins, and a proved tag

Design-stage prose invites the failure mode the deployed-protocol volumes never face: text that reads as specification but rests on a draft. Following the series convention, every claim in this volume is exactly one of *specified* (a precise, pinned artifact fixes it; we state whether that artifact is a specification, model, or prototype code), *designed but unspecified* (an informal source describes a concrete mechanism no specification fixes; we cite source and date), or an *open problem* (no source resolves it; where the designers acknowledge the gap, we quote them). Some parts of Crosslink's corpus carry actual proofs, so we additionally tag a claim *proved* when a peer-reviewed theorem or a machine-checked artifact establishes it, always together with the scope at which it was established.

The inventory, developed across the rest of the volume, maps as follows. Proved: the NTT2020 theorems for ebb-and-flow and Snap-and-Chat (peer-reviewed, IEEE S&P 2021), and the Quint invariant `prefix_inv` at its small checked scope (§1.3). Specified (pinned, verifiable artifacts only): the Quint `isValid` model itself and the quoted prototype and Zebra constants. Designed but unspecified: the Crosslink 2 construction rules and the two Assured Finality safety theorems. One published proof text is defective, but the construction chapter contains enough premises to complete the intended argument (§6.1). This bin also contains the staking and tokenomics design, the BFT subprotocol instantiation (Tenderlink), active-set selection, user-activated slashing, and the implementation's no-Stalled-Mode variant. Open: precise liveness bounds and the dependence on L, completion of the $\mathsf{LOG}_{\mathrm{ba}}$ safety development, production values of σ and L, and NU8 inclusion and dates (classification synthesized from the sources cited throughout this section).

1.3 The source landscape

Table 1 lists the sources, pinned. All repository facts were verified firsthand at the stated commits; web sources were last checked 2026-09-05.

Artifact	Pin / date	Role and rigor
tfl-book (ECC)	`fe6e1d6`, 2024-12-13	construction; static since pin
Neu–Taş–Tse	eprint 2020/1091; S&P 2021	peer-reviewed lineage
crosslink-spec (Informal Systems)	`8edc3e9`, 2025-07-23	Quint model; proof of concept
quint-crosslink (zmanian)	empty upstream (zero refs)	nothing to examine
zebra-crosslink (Shielded Labs/Eiger)	`6d02a1b`, 2026-04-21	historical prototype + design book
crosslink_monolith (Shielded Labs)	`807b995`, 2026-08-13	active feature-net prototype
zcash/zips	`e753a6a`, 2026-09-03	no Crosslink or NU8-scoping ZIP
zcash/zebra	`d1fd7ad`, 2026-09-04	deployed Π_{bc} facts
crosslink-protocol-guide	`b77d845`, 2026-03-26	secondary; visual tour
simtfl (zcash)	`a1641c8`, 2023-12-07	simulator; no protocol results
Shielded Labs web/forum/Arborist	checked 2026-09-05	status only, dated

Table 1: Crosslink source inventory, pinned. Verification date 2026-09-05.

Current-head boundary (2026-09-05). The four earlier design repositories were rechecked: tfl-book remains at `fe6e1d6`, crosslink-spec at `8edc3e9`, zebra-crosslink at `6d02a1b`, and crosslink-protocol-guide at `b77d845`. Active prototype development has moved to crosslink_monolith, whose current tag `v13` is commit `807b995` (2026-08-13). Official zips `e753a6a` still contains no Crosslink ZIP or ZIP specifying NU8. The source distinctions below are therefore load-bearing: the 2026-04-21

fork describes a historical snapshot, while `v13` records current prototype behaviour and is explicitly neither production-ready nor the code proposed for upstream integration.

The TFL book. The construction source of record is the ECC Trailing Finality Layer book, an mdBook formerly deployed at `https://electric-coin-company.github.io/tfl-book/` (404 since at least 2026-07-15 and rechecked 2026-09-05, following the repository's transfer to `github.com/daira/tfl-book`) and now rendered at `https://daira.github.io/tfl-book/`; the local clone HEAD is `fe6e1d6`, dated 2024-12-13, the merge of PR #162 "crosslink2" by Daira-Emma Hopwood, with no later commits (`git log`). Prehistory: a design sketch was presented at Zcon4 by Nate Wilcox ("Interactive Design of a Zcash Trailing Finality Layer"); book version 0.1.0 introduced Crosslink, and 0.2.0 updated the construction and security analysis to Crosslink 2 (`src/version-history.md`). Three rigor caveats govern every citation of it. First, the book self-describes as "an early and incomplete protocol design proposal", listing as missing the PoS subprotocol selection, issuance and supply mechanics, transaction semantics integration, the transition plan, ZIPs, and the security and economic analyses (`src/introduction/status-and-next-steps.md` lines 1–30). Second, it is internally inconsistent about its own version: `book.toml` still titles the deployed book v0.1.0 while `src/version-history.md` records 0.2.0 ("Crosslink 2") as current (`book.toml` line 9 vs `src/version-history.md` lines 3–5). Third, and most consequentially, its rigor is uneven across chapters: the construction chapter is specification-grade prose, but the liveness argument carries explicit TODOs ("make that more precise"; "more detailed argument needed, especially for the dependence on L"), and the safety section is marked "$TODO$ Not updated for Crosslink 2 below this point" immediately after it, with the $\mathsf{LOG}_{\mathrm{fin}}/\mathsf{LOG}_{\mathrm{ba}}$ proofs below that marker still written in Snap-and-Chat/Crosslink-1 notation — including the sanitization machinery Crosslink 2 exists to remove (`security-analysis.md` lines 5–50, 54, 139–150). Two further defects are findings of this volume, developed in the security-analysis section. The proof text under "Safety Theorem: Final Agreement of Π_{bft} implies Assured Finality" is a verbatim duplicate of the Prefix Agreement proof and never uses Final Agreement (`security-analysis.md` lines 87–91 vs 73–77). This is a source erratum, not an unresolved theorem: the construction chapter's own sketch at line 592 supplies the intended route, and Theorem 6.2 below completes it. The book also concedes that "some rules, e.g. the Linearity rule, only contribute heuristically to security in the analysis so far", with the $\mathsf{LOG}_{\mathrm{ba}}$ development trailing off mid-sentence (`security-analysis.md` lines 275–281, 257).

The book-referenced simtfl repository remains at `a1641c8` (2023-12-07). Its README describes an experimental simulator whose demo is only an example of message passing; it contains no published Crosslink experiment or security result (checked 2026-09-05).

The machine-checked model. The repository `crosslink-spec` (Informal Systems) is pinned at `8edc3e9` (2025-07-23, the merge of PR #4; shallow local clone, depth 1). It formalizes exactly one slice of Crosslink: the `isValid` check of a BFT proposal against local PoW and BFT block stores, written in Quint as a literate source (`validity.md`, tangled to `src/validity.qnt` via `lmt`; a transpiled `validity.tla` is committed) (README lines 25–32; `validity.md` lines 1–7). Its own README fixes its standing: "a proof of concept", whose logic was "inferred ... from an insightful Youtube talk" and "might not be up-to-date with the current protocol design ideas" (README line 27). What was actually checked is premium material at a stated scope: the invariant `prefix_inv` — the PoW history defined by BFT finalization is a path in the PoW block store — survived random simulation of 10,000 traces of 20 steps and TLC model checking in about three minutes at scope `confirmation_depth` = 1 with stores of at most 8 blocks, and a three-block finalization witness trace was produced by both simulation and TLC (README lines 61–93; `validity.md` lines 376–421, 464–481). The model diverges from the book in ways we record as model-versus-specification discrepancies, not protocol facts: its `linear()` requires a strict-ancestor snapshot where the book's Linearity rule permits repeating the parent snapshot (and the book needs that permission for BFT liveness; the Quint source itself com-

ments "this actually seems allowed. TODO: perhaps remove"); it models the BFT store as a single linear list with no forks and omits signatures, notarization proofs, epochs, voting, bft-last-final, and every Π_{bc}-side rule; and its `add_pow` assigns block hashes colliding with genesis, an artifact PR #4 fixed only on the BFT side (`validity.md` lines 173–176, 132–137, 244–247, 270–278, 433–440 vs tfl-book `construction.md` lines 573, 611–618, 113–193). The second Quint repository, quint-crosslink (remote zmanian/quint-crosslink), is empty upstream as well as locally — `git ls-remote` returns zero refs (checked 2026-09-05); there is nothing to examine.

The historical implementation fork. The April 2026 prototype snapshot is zebra-crosslink, Shielded Labs' fork of Zebra built with Eiger; the local clone HEAD is `6d02a1b`, dated 2026-04-21, the merge of PR #282. It ships its own mdBook whose `cl2-construction.md` and `cl2-security-analysis.md` are declared "almost direct paste" copies of the TFL book chapters, committed verbatim "for the purposes of precisely committing to a ... precise set of book contents for a security design review", with the explicit note that Daira-Emma Hopwood and Jack Grigg "have not reviewed and do not necessarily endorse" the changes (`book/src/design/cl2-construction.md` lines 3–7). The fork then diverges from the book in two documented, load-bearing ways. It drops Stalled Mode entirely — "The zebra-crosslink design in this book diverges from this text by *not* including 'Stalled Mode' rules" — relying instead on stakers redelegating to prevent hazards such as BFT stalls (`cl2-construction.md` lines 10–11); and it removes the proposer's outer signature on bft-blocks, conceding that "a direct hash or copy of the most recent BFT finality certificate is insufficient evidence to ensure that specific bitwise copy is canonical" (`cl2-construction.md` lines 14–15). The fork's own warnings bound what this volume may claim for it: "These extensions and changes from the Crosslink 2 construction may violate some of the assumptions in the security proofs" and "We are using a custom-fit in-house BFT protocol for prototyping. Its own security properties need to be more thoroughly verified" (`book/src/design.md` lines 11–13). That in-house engine is tenderlink (`git.sr.ht/~azmr/stuff`, rev `0e88509`), Shielded Labs' lightweight Tendermint implementation that replaced the earlier Malachite (Informal Systems) integration — at the pinned commit `Cargo.lock` contains no malachitebft crates and `src/malctx.rs` survives only as dead code — with ed25519-zebra finalizer keys and BLAKE3 block hashes (`zebra-crosslink/Cargo.toml` lines 74–79; `src/lib.rs`; `src/chain.rs` lines 224–226); the prototype parameters are $\sigma = 3$ and finalization gap bound 7, with the source stating "No verification has been done on the security or performance of these parameters" (`src/chain.rs` lines 219–221). The staking design (the "nutshell" chapter) self-describes as a "provisional sketch" and matches Shielded Labs' published staking design post of 2025-10-27 on Orchard-only staking with revealed amounts, one-epoch unbonding, and power-of-ten stake quantization; on epoch length the sources diverge — the post proposes five-day epochs, the chapter a one-day staking day plus an approximately six-day locked phase (`book/src/design/nutshell.md` lines 13–133; `https://shieldedlabs.net/crosslink-updated-staking-design/`, accessed 2026-09-05).

The active feature-net prototype. Development subsequently moved into Shielded Labs' crosslink_monolith; tag `v13` is commit `807b995` (2026-08-13). Its README says the repository exists to prove ideas, is not production-ready, and is not the code that would be proposed upstream. It nevertheless changes the observable prototype substantially: staking bonds now have create, begin-unbonding, withdraw, and retarget state transitions; the transaction builder accounts for bond creation and withdrawal in value balance; Tenderlink limits the active roster to its first 100 finalizers; and a staking period is 150 blocks with a 70-block action window and a two-period slash-analysis window (`librustzcash/zcash_primitives/src/transaction/mod.rs`; `tenderlink/src/lib.rs`; `zebra-crosslink/zebra-state/src/service/check/delegation.rs`). Five feature-net heights are temporary exceptions to the action window (`zebra-crosslink/zebra-consensus/src/transaction.rs`:936–978). Version 13 also contains user-configured and executable-shipped hard-fork schedules that terminate selected finalizer keys and burn bonds delegated to them

during the preceding 300 blocks. This is user-activated, socially selected slashing, not automatic evidence-based slashing (`zebra-crosslink/zebra-chain/src/parameters/hardfork.rs`; `zebra-crosslink/zebra-state/src/service/finalized_state/zebra_db/slashing.rs`). These are *specified* prototype facts, not proposed consensus rules.

Secondary sources. The community "A Visual Tour of Zcash Crosslink" (crosslink-protocol-guide @ `b77d845`, 2026-03-26, MIT) is a short mermaid-diagram tour with a TFL lexicon; we use it for orientation only, never as a claim source. Incidental ZIP references exist — draft ZIP 218 ("25-second Block Target Spacing", created 2026-03-13) cites Crosslink as complementary, and draft-ecc-onchain-accountable-voting cites the TFL book's Assured Finality definition — but neither specifies any part of Crosslink (zips @ `e753a6a`, `zip-0218.md` lines 77, 742).

1.4 Status: milestones, testnets, and the absence of dates

Shielded Labs' public roadmap records five completed milestones: M1 "Placeholders and Processes" (2025-03-21), M2 "Disconnected BFT" (2025-05-16), M3 "PoW Consensus" (2025-08-28), M4 "Staking Transaction Semantics" (2026-01-28), and M5 "Pre-Productionization, Tech Debt, Final Refactoring" (2026-04-15). The Productionization Phase is stated as Q2 2026 – Q2 2027, in progress, described as deploying Crosslink on a testnet, iterating, "and, if there is consensus, integrating it into the Zcash protocol"; no NU8 or mainnet activation date is given anywhere on it (`https://shieldedlabs.net/roadmap/`, checked 2026-09-05).

Deployment practice runs ahead of specification. FeatureNet, a series of incentivized testnets announced 2026-04-02, launched Season 1 on 2026-04-15: miners, finalizers, and stakers earn real ZEC, with 500 ZEC allocated across seasons (Season 1: 25 ZEC, split 40% miners / 40% stakers / 20% Dev Fund), and the announcement states that "any decision to activate Crosslink on mainnet will follow Zcash's existing governance process" (`https://forum.zcashcommunity.com/t/crosslink-incentivized-feature-net/55210`, checked 2026-09-05). Season 1 experienced stalled BFT finality and divergent chains; organizers selected a canonical block for rewards on 2026-07-03. By the 2026-09-03 workshop notes, the feature net had successfully tested the user-activated slashing mechanism intended to resolve BFT stalls, and regular resets plus a separate nightly network were planned. These are test observations, not mainnet deployment evidence (same forum thread, posts 72, 83, and 86).

On dates this volume is deliberately silent, because the primary sources are. The community "NU8 Decision Framework" thread (opened 2026-02-11) records that no governance framework or dates exist for consensus-change inclusion, while crediting execution: "Shielded Labs has executed well. Milestones 1 through 4 landed on or ahead of schedule" (`https://forum.zcashcommunity.com/t/nu8-decision-framework/54670`, checked 2026-09-05).

1.5 Which source governs each claim

Throughout this volume, "Crosslink 2" means the construction as published in the TFL book at `fe6e1d6`: its model, rules, and safety-theorem statements are the referent of every unqualified claim; the book is the construction source of record, not a consensus specification. "The zebra-crosslink design" means the historical design at `6d02a1b`, including where it contradicts the TFL book (Stalled Mode and the outer signature). "The current prototype" means `crosslink_monolith v13`; it supersedes implementation facts, but its own README disclaims production and upstream status. Where the sources conflict we present each with its date and bin. The Quint model is normative for nothing; it is evidence, at its stated scope, that one slice of the design is coherent, and a source of recorded discrepancies. Design goals frame both documents but bind neither: the TFL book targets expected time-to-finality below 30 minutes, cost-of-attack for a 1-hour rollback not reduced versus today, halting cost above 24 hours of PoW mining revenue, and unchanged wallet (lightwalletd, full-node API)

and GetBlockTemplate-miner interfaces (tfl-book @ `fe6e1d6`, `src/design/goals.md` lines 22, 28–36, 73). The 30-minute goal is uncheckable today because no production σ or L exists in any source.

1.6 Relation to the sibling volumes

The interface-stability goal above motivates citing the deployed-stack volumes; it does not guarantee that an eventual implementation preserves every interface. Anchor semantics and reorg behaviour of the commitment tree are those of the *Ironwood Guide* (§"Resting: the note in the commitment tree", in particular "Anchor choice, reorgs, and the anonymity set"); what a wallet counts as confirmed is the *Wallet Guide*'s ZIP-315 `ConfirmationsPolicy` and expiry treatment (§"Transaction construction" and §"Broadcast, expiry, and the transaction lifecycle"), whose trusted/untrusted depths are exactly the policy layer a finality layer would let wallets strengthen. The header-chain commitment a finality certificate would sit beside, and its unbuilt light-client bridge, are the *FlyClient Guide*'s (§"The deployment gap"); this volume owns their one-way composition.

2 The ebb-and-flow model

Crosslink attaches a BFT finality layer to Zcash's proof-of-work chain, and every security claim it makes is phrased against a model published before any Zcash design work began: the *ebb-and-flow* formulation of Neu, Nusret Tas, and Tse, "Ebb-and-Flow Protocols: A Resolution of the Availability-Finality Dilemma" (arXiv:2009.04987; v1 2020-09-10, v2 2020-12-28, v3 2021-02-04; the abstract page notes "Forthcoming in IEEE Symposium on Security and Privacy 2021"; hereafter [NTT2020]). This section documents that model: the impossibility it responds to, its execution environments, its two-ledger security definition, the snap-and-chat construction and what the paper actually proves about it, and — because the Crosslink books adopt the model only after amending it — the specific points where the Trailing Finality Layer book departs. Throughout, the best-chain protocol Π_{bc} remains a black box, per this volume's scope rule: we record what the model assumes of it (chain structure, confirmation by depth, header validity), never its internals.

Sources. The primary text is arXiv:2009.04987v3 (checked 2026-09-05). The Trailing Finality Layer book (`tfl-book` @ `fe6e1d6f`) cites the IACR ePrint mirror 2020/1091 by page number; every passage the book quotes was verified identical in arXiv v3, and we cite section, definition, and theorem numbers of v3 rather than pages. The Shielded Labs/Eiger book (`zebra-crosslink` @ `6d02a1b8`, 2026-04-21) carries the Crosslink 2 construction and security analysis as `cl2-construction.md` and `analysis/cl2-security-analysis.md` under `book/src/design/`; a diff against `tfl-book` @ `fe6e1d6f` shows only 20 added admonition lines (14 in the construction copy, 6 in the security-analysis copy) and no other change, none touching the [NTT2020] references, the ebb-and-flow discussion, Prefix Agreement, or Assured Finality — the model presentation is textually unchanged between the two books (diff rechecked 2026-09-05), so we cite the `tfl-book` chapter files throughout. The community `crosslink-protocol-guide` (@ `b77d8456`) contains no occurrence of "ebb" at all and is not a source for this section (grep, 2026-09-05). The official ZIP repository (@ `e753a6a3`) contains no Crosslink or trailing-finality draft; its only Crosslink mentions are ZIP 218 citing "mechanisms such as Crosslink" in the context of faster block times, and `draft-ecc-onchain-accountable-voting` citing the book's Assured Finality definition (§2.8).

Remark 2.1 (Binning)**.** Claims in this section about what [NTT2020] states and proves are verifiable against the published text and carry no bin tag; within its model the paper's theorems are settled scholarship, and we cite them by number. Claims about Crosslink itself remain design-stage and carry the series' three bins (*specified*, *designed but unspecified*, *open problem*); no normative Crosslink artifact exists (no draft ZIP in `zips` @ `e753a6a3`), so nothing in this section reaches the specified bin except pinned source code. Where the book disputes the adequacy of the paper's model, the sources

disagree about the model, not about what the paper proves within it; we record such disputes as findings with both citations.

Remark 2.2 (Terminology)**.** The name *ebb-and-flow* denotes the security model and goal (Definition 2.8); *snap-and-chat* denotes the construction proposed in the same paper to achieve that goal (§2.4). The distinction is the book's own admonition (`tfl-book` @ `fe6e1d6f`, `notes-on-snap-and-chat.md`, "Terminology"), and it matters because Crosslink adopts the model while replacing the construction. The book further replaces the paper's "longest chain protocol" by *best-chain protocol* (Π_{bc}): Bitcoin and Zcash follow the consensus-valid chain with most accumulated work, not the longest, and [NTT2020]'s own footnote 2 does not require a longest-chain protocol anyway (§2.4). Finally, [NTT2020] uses "depth" for what Bitcoin and Zcash call "height"; and `zcashd` counts the tip at depth 1 where the book counts it at depth 0 (`tfl-book`, `construction.md` line 66). We write Π_{lc} when reporting the paper and Π_{bc} when reporting the books. Notation for chains: for a chain C and $\sigma \in \mathbb{N}$, the truncation $C^{\lceil\sigma}$ removes the last σ blocks of C; the relation $x \preceq y$ says x is a prefix of y; and $x \sim y$ ("x and y agree") says $x \preceq y$ or $y \preceq x$. Subscripts ($\preceq_{\mathrm{bc}}$, $\sim_{\mathrm{bft}}$) name the chain kind.

2.1 The availability–finality dilemma

The paper opens from an impossibility. Its abstract states the position in three sentences (arXiv:2009.04987v3, Abstract): "The CAP theorem says that no blockchain can be live under dynamic participation and safe under temporary network partitions. To resolve this availability-finality dilemma, we formulate a new class of flexible consensus protocols, ebb-and-flow protocols, which support a full dynamically available ledger in conjunction with a finalized prefix ledger. The finalized ledger falls behind the full ledger when the network partitions but catches up when the network heals."

The impossibility itself is attributed, not proved, in [NTT2020] (§ I-A): "it is impossible for any protocol to be both safe under network partition and live under dynamic participation: individual nodes in the network cannot distinguish between the two scenarios to act differently. This intuition is formalized in [3] and its connection to the CAP theorem [17] was made precise recently in [18]." Reference [3] is Pass and Shi, "The sleepy model of consensus" (ASIACRYPT 2017); reference [17] is Gilbert and Lynch's CAP theorem exposition (SIGACT News 33(2), 2002); reference [18] is Lewis-Pye and Roughgarden, "Resource Pools and the CAP Theorem" (arXiv:2006.10698; v1 2020-06-18; no journal note on the abstract page, checked 2026-09-05), which the book cites as [LR2020] (`tfl-book` @ `fe6e1d6f`, `the-arguments-for-bounded-availability-and-finality-overrides.md`).

The resolution is a two-ledger abstraction. A *conservative* client trusts only the finalized ledger, which is a prefix of the longer, dynamically available ledger believed by an *aggressive* client; "This ebb-and-flow property avoids a system-wide determination of availability versus finality and instead leaves this decision to the clients" ([NTT2020] § I-B). Zcash wallets already practice a client-side version of this choice at the confirmation-depth layer: the ZIP-315 `ConfirmationsPolicy` parameterizes, per wallet, how deep an output must be before it is treated as spendable (*Wallet Guide*, §"Confirmations, trust, and spendability"). The ebb-and-flow model lifts the same freedom to the level of which ledger a client reads.

2.2 Execution model and environments

The model of [NTT2020] § III-A has five cornerstones: n total nodes; time proceeding in slots with synchronized clocks (footnote 4: bounded clock offsets can be captured as part of the network delay); a public-key infrastructure with unique cryptographic identities; a random oracle as the source of randomness; and a probabilistic polynomial-time adversary. Corruption is static and Byzantine: "Before the protocol execution starts, the adversary gets to corrupt (up to) f nodes, then called adversarial. Adversarial nodes surrender their internal state to the adversary and can deviate from the protocol arbitrarily (Byzantine faults) under the adversary's control. The remaining $(n - f)$ nodes are honest and follow the protocol as specified."

Dynamic participation is modelled by sleeping: "The adversary chooses, for every time slot and honest node, whether the node is awake or asleep in that slot ... An honest node that is asleep in a slot does not execute the protocol in that slot, and messages that would have arrived in that slot are queued and delivered in the first slot in which the node is awake again. Adversarial nodes are always awake." The paper is explicit that the permissioned setting and dynamic participation are "two orthogonal aspects of the environment" ([NTT2020] § III-A).

Partial synchrony is modelled with a single asynchrony-to-synchrony transition: "Although in reality, multiple such periods of (a-)synchrony could alternate, we follow the long-standing practice in the BFT literature and study only a single such transition" ([NTT2020] § III-A). Two adversary-environment pairs then formalize the two regimes.

Definition 2.3 (P1 environment $(\mathcal{A}_1(\beta), \mathcal{Z}_1)$; [NTT2020] § III-A)**.** The pair $(\mathcal{A}_1(\beta), \mathcal{Z}_1)$ formalizes a partially synchronous network under dynamic participation, with respect to the fraction β of adversarial nodes: the adversary $\mathcal{A}_1$ corrupts $f = \beta n$ nodes. Before a global stabilization time GST, the adversary can delay network messages arbitrarily; after GST, it must deliver all messages sent between honest nodes within Δ slots. Before a global awake time GAT, the adversary determines which honest nodes are awake or asleep and when; after GAT, all honest nodes are awake. Both GST and GAT are chosen by the adversary, are unknown to the honest nodes, and can be causal functions of the randomness in the protocol.

Definition 2.4 (P2 environment $(\mathcal{A}_2(\beta), \mathcal{Z}_2)$; [NTT2020] § III-A)**.** The pair $(\mathcal{A}_2(\beta), \mathcal{Z}_2)$ formalizes a synchronous network under dynamic participation, with respect to a bound β on the fraction of awake nodes that are adversarial: at all times the adversary must deliver all messages sent between honest nodes within Δ slots; at all times it determines which honest nodes are awake or asleep and when, subject to the constraint that at most a fraction β of awake nodes are adversarial and at least one honest node is awake.

The global awake time exists because without it the model would collapse into the impossibility (footnote 5): "Without slightly restricting dynamic participation via a GAT after which all nodes are awake, this adversary would fall under the CAP theorem so that no secure protocol against it can exist. In many applications it is realistic that every now and then there is a period in which all nodes are awake." The two classical regimes are recovered at the extremes ([NTT2020] § I-D): "If $\mathsf{GAT} = 0$, then the environment is the classical partially synchronous network, and the ledger $\mathsf{LOG}_{\mathrm{fin}}$ has the optimal resilience achievable in that environment. On the other hand, if $\mathsf{GST} = 0$ and $\mathsf{GAT} = \infty$, then the environment is a synchronous network with dynamic participation, and the ledger $\mathsf{LOG}_{\mathrm{da}}$ has the optimal resilience achievable in that environment."

Remark 2.5 (Finding: the single-transition idealization)**.** The book argues at length that the DLS1988 GST formalization was intended for one-shot consensus and "cannot correctly be applied to liveness properties of non-terminating protocols", and that [NTT2020]'s acknowledgement quoted above ("long-standing practice") "is not adequate: ... it is not valid in general to infer that properties holding for the first transition to synchrony also apply to subsequent transitions" (`tfl-book` @ `fe6e1d6f`, `construction.md` lines 13–41; textually unchanged in `zebra-crosslink` @ `6d02a1b8`, `cl2-construction.md`). This is the stated reason Crosslink 2 does not take synchrony or partial synchrony as an implicit assumption of its communication model. The dispute concerns the model's adequacy for non-terminating liveness, not the correctness of the paper's proofs within the model. Classification: the critique and the resulting communication-model choice are *designed but unspecified* (book prose, no normative artifact).

2.3 Security definitions

Definition 2.6 (Ledger security; [NTT2020] Definition 1). Let T_{confirm} be a polynomial function of the security parameter σ. A state machine replication protocol Π outputting a ledger LOG is *secure after time* T, with transaction confirmation time T_{confirm}, if LOG satisfies:

- *Safety:* for any two times $t \geq t' \geq T$ and any two honest nodes i and j awake at times t and t' respectively, either $\mathsf{LOG}_i^t \preceq \mathsf{LOG}_j^{t'}$ or $\mathsf{LOG}_j^{t'} \preceq \mathsf{LOG}_i^t$.
- *Liveness:* if a transaction is received by an awake honest node at some time $t \geq T$, then for any time $t' \geq t + T_{\text{confirm}}$ and honest node j awake at time t', the transaction is included in $\mathsf{LOG}_j^{t'}$.

A protocol safe (live) after $T = 0$ is called simply safe (live). Here LOG_i^t denotes the ledger in the view of node i at time t; for longest-chain protocols, σ is the confirmation delay for transactions.

Definition 2.7 (Flexible protocol; [NTT2020] Definition 2). A *flexible protocol* is a pair of state machine replication protocols (Π_1, Π_2), where Π_1 and Π_2 have the same input transactions txs and output ledgers LOG_1 and LOG_2, respectively.

Definition 2.8 ((β_1, β_2)-secure ebb-and-flow protocol; [NTT2020] Definition 3). A (β_1, β_2)-secure ebb-and-flow protocol is a flexible protocol $(\Pi_{\text{da}}, \Pi_{\text{fin}})$ outputting an available ledger LOG_{da} and a finalized ledger $\mathsf{LOG}_{\text{fin}}$, such that, for security parameter σ and confirmation-time function T_{confirm}:

1. *P1 (Finality):* under $(\mathcal{A}_1(\beta_1), \mathcal{Z}_1)$, the ledger $\mathsf{LOG}_{\text{fin}}$ is safe at all times, and there exists a constant C such that $\mathsf{LOG}_{\text{fin}}$ is live after time $C(\max\{\mathsf{GST}, \mathsf{GAT}\} + \sigma)$, except with probability $\mathsf{negl}(\sigma)$.
2. *P2 (Dynamic Availability):* under $(\mathcal{A}_2(\beta_2), \mathcal{Z}_2)$, the ledger LOG_{da} is secure, except with probability $\mathsf{negl}(\sigma)$.
3. *Prefix:* for any honest node i and time t, $\mathsf{LOG}_{\text{fin},i}^t$ is a prefix of $\mathsf{LOG}_{\text{da},i}^t$.

Here $\mathsf{negl}(\cdot)$ decays faster than every inverse polynomial: for all $c > 0$ there exists σ_0 such that $\mathsf{negl}(\sigma) < \sigma^{-c}$ for all $\sigma > \sigma_0$.

Optimality. The resilience constants are sharp in the paper's model (§ III-C): "Observe that no ebb-and-flow protocol can tolerate a Byzantine adversary $\mathcal{A}_1(\beta_1)$ with $\beta_1 \geq 1/3$ in a partially synchronous network. Similarly, no ebb-and-flow protocol can tolerate a Byzantine adversary $\mathcal{A}_2(\beta_2)$ with $\beta_2 \geq 1/2$ in a synchronous network. Hence the security of Π_{sac} implies that it is optimally resilient. We denote the worst-case adversary-environments as $(\mathcal{A}_1^*, \mathcal{Z}_1) := (\mathcal{A}_1(1/3), \mathcal{Z}_1)$ and $(\mathcal{A}_2^*, \mathcal{Z}_2) := (\mathcal{A}_2(1/2), \mathcal{Z}_2)$."

The paper uses $(1/3, 1/2)$ as threshold labels. They must not be read as security at equality: the tolerable Byzantine fractions are strictly below $1/3$ and $1/2$, respectively. Its shorthand adversary notation does not override the impossibility statement it gives immediately before it; concrete finite rosters also require the appropriate integer quorum bounds.

Relation to flexible BFT. Ebb-and-flow goes beyond the flexible BFT of Malkhi, Nayak, and Ren ([NTT2020] reference [20]) in two ways ([NTT2020] § I-E): dynamic participation is brought in as a client belief, and prefix consistency between the two ledgers is required "in all circumstances" rather than only when both clients' assumptions hold. Where flexible BFT supports a tradeoff for f between $n/3$ and $n/2$, "The snap-and-chat protocol achieves $(n/2, n/3)$, simultaneously optimal. No tradeoff is

necessary" (Fig. 3 caption). The remaining gap to Blum, Katz, and Loss ([NTT2020] reference [27]), a non-flexible protocol with an optimal synchrony/asynchrony tradeoff, "can be interpreted as the value of flexibility".

2.4 The snap-and-chat construction

Construction 2.9 (Snap-and-chat; [NTT2020] §§ I-D, III-B)**.** Nodes run two off-the-shelf protocols in parallel: a dynamically available protocol Π_{lc} (footnote 2: "Longest chain protocols are representative members of this class of protocols, hence the notation Π_{lc}, but this class includes many other protocols as well") and a partially synchronous BFT protocol Π_{bft}. Each node i takes snapshots of its confirmed Π_{lc} ledger ch_i^t and inputs them into Π_{bft}; the output Ch_i^t of Π_{bft} is an ordered list — a chain of chains — of snapshots, where a BFT block carries as payload only a reference $B.\mathsf{ch}$ to an LC block identifying the snapshot. The ledgers are extracted as

$$\mathsf{LOG}_{\text{fin},i}^t := \mathsf{sanitize}(\mathsf{flatten}(\mathsf{Ch}_i^t)), \qquad \mathsf{LOG}_{\text{da},i}^t := \mathsf{sanitize}(\mathsf{flatten}(\mathsf{Ch}_i^t) \,\|\, \mathsf{ch}_i^t),$$

where flatten concatenates the referenced chains of LC blocks as ordered, $\|$ is concatenation, and sanitize removes all but the first occurrence of each block ([NTT2020] § III-B3, Fig. 5 caption). At transaction level (footnote 6): "Formally, LOG_{da} and $\mathsf{LOG}_{\text{fin}}$ are now represented as sequences of LC blocks. Proper transactions ledgers are readily obtained by concatenating the transactions contained in the blocks and removing duplicate and invalid transactions (sanitization)."

The coupling from Π_{lc} into Π_{bft} is a boycott: "in the Π_{bft} sub-protocol, each honest node boycotts the finalization of snapshots that are not confirmed in Π_{lc} in its view" (§ I-D). Concretely, Streamlet's voting rule is extended — "An honest node only votes for a proposed BFT block B if it views $B.\mathsf{ch}$ as confirmed" — and this is the only change (line 19 of Algorithm 1; "Sleepy is applied unaltered and the modification required for Streamlet is minor"). "In addition, ch_i^t is used as side information in Π_{bft} to boycott the finalization of invalid snapshots proposed by the adversary" (§ III-B).

Instantiation. The analyzed instance takes Π_{lc} to be Sleepy consensus (Pass–Shi; permissioned longest chain, where "blocks of a certain depth on the longest chain are confirmed") and Π_{bft} to be Streamlet (epochs with pseudo-random leaders; a block is notarized when at least two-thirds of nodes vote for it; "Out of three adjacent notarized blocks from consecutive epochs, the middle one gets finalized along with its prefix") ([NTT2020] § III-B). Section III-D extends the analysis to HotStuff and PBFT as Π_{bft} with minor modifications, and Theorem 2.10 carries over.

Theorem 2.10 (Snap-and-chat security; [NTT2020] Theorem 1)**.** *The protocol Π_{sac} is a $(1/3, 1/2)$-secure ebb-and-flow protocol. (Proved under the static-corruption, synchronized-slot, PKI plus random-oracle model of §2.2: P1 under $(\mathcal{A}_1^*, \mathcal{Z}_1)$, P2 under $(\mathcal{A}_2^*, \mathcal{Z}_2)$, Prefix by construction; Appendix D proves the same with HotStuff in place of Streamlet.)*

Proof structure. We record what is proved and how ([NTT2020] § III-C, Appendix B, dependency diagrams Figures 7 and 8, boxes 1–8), because the book's departures target specific joints of this argument. P1 safety is Lemma 1 — safety of Π_{bft} by quorum intersection, unaffected by the voting-rule change — plus the observation that ledger extraction preserves safety. P1 liveness is Lemma 4, which needs Theorem 2.11 and Lemmas 2–3 (Streamlet liveness, each carrying the highlighted extra condition that "the leaders' proposals reference LC blocks that are viewed as confirmed by all honest nodes"). P2 is the security of Π_{lc} under $(\mathcal{A}_2^*, \mathcal{Z}_2)$, taken from Pass–Shi (their Theorem 3 and Lemma 1), plus Lemma 2.12. Prefix holds by construction.

Theorem 2.11 (Longest-chain security after max{GST, GAT}; [NTT2020] Theorem 2)**.** *For all $p < (n-2f)/(2\Delta n(n-f))$, there exists a constant $C > 0$ such that for any* GST *and* GAT *specified by $(\mathcal{A}_1^*, \mathcal{Z}_1)$, the protocol $\Pi_{\text{lc}}(p)$ is secure after $C(\max\{\mathsf{GST}, \mathsf{GAT}\} + \sigma)$, with transaction confirmation time $T_{\text{confirm}} = \sigma$, except with probability $e^{-\Omega(\sqrt{\sigma})}$. (Here p is the probability that a given node produces a block in a given slot; the constant C depends on p, n, f, and Δ (footnote 9); footnote 10 improves the error to $A_\varepsilon e^{-a_\varepsilon \sigma^{1-\varepsilon}}$ for any $\varepsilon > 0$ via the DKT+2020 recursive bootstrapping argument.)*

The proof of Theorem 2.11 extends Pass–Shi pivots to *GST-strong pivots* ([NTT2020] Definition 4 and eq. (8)): a slot $t \geq \max\{\mathsf{GST}, \mathsf{GAT}\}$ such that for any $t_a \leq t \leq t_b$, the convergence opportunities (slots in which exactly one honest node produces a block and no other honest block appears within Δ on either side, forcing honest views to converge unless the adversary counters with a block of its own) counted only after $\max\{t_a, \mathsf{GST}, \mathsf{GAT}\}$ outnumber the adversarial slots in $[t_a, t_b]$ — only post-transition convergence opportunities count, because their useful properties fail under asynchrony or while honest nodes may sleep.

Lemma 2.12 (Consistency of $\mathsf{LOG}_{\text{fin}}$ with Π_{lc} under P2; [NTT2020] Lemma 5)**.** *The ledger $\mathsf{LOG}_{\text{fin}}$ is a safe prefix of the output of Π_{lc} in the view of every honest node at all times under $(\mathcal{A}_2^*, \mathcal{Z}_2)$, except with probability $e^{-\Omega(\sqrt{\sigma})}$. The key proof step: for a BFT block to become final in the view of an honest node under $(\mathcal{A}_2^*, \mathcal{Z}_2)$, at least one vote from an honest node is required, and honest nodes only vote for a BFT block if they view the referenced LC block as confirmed; this holds even if the final BFT blocks conflict in the output of Π_{bft}.*

Why the composition is safe. Figure 9 of [NTT2020] presents the trichotomy. (a) When both subprotocols are safe, ch and Ch do not fork. (b) When Π_{lc} is unsafe, "snapshots taken by different nodes or at different times can conflict. However, Π_{bft} is still safe and thus orders these snapshots linearly. Any transactions invalidated by conflicts are sanitized during ledger extraction. As a result, $\mathsf{LOG}_{\text{fin}}$ remains safe." (c) When Π_{bft} is unsafe, in a synchronous network with $n/3 < f < n/2$, "finalization of a snapshot requires at least one honest vote, and thus only valid snapshots become finalized. Since finalized snapshots are consistent, $\mathsf{LOG}_{\text{fin}}$ is consistent with LOG_{lc}."

Simulation evidence. The paper simulated Sleepy-plus-Streamlet with $n = 100$ nodes, $f = 25$ adversarial, $\Delta = 1\,\text{s}$, Sleepy rate $\lambda = 0.1\,\text{s}^{-1}$ (per-node $\lambda_0 = 10^{-3}\,\text{s}^{-1}$), confirmation depth $k = 20$, and Streamlet $\Delta_{\text{bft}} = 5\,\text{s}$ ([NTT2020] § IV). Under dynamic participation, LOG_{da} grows steadily while $\mathsf{LOG}_{\text{fin}}$ stalls whenever fewer than a two-thirds quorum (67 nodes) is awake, catching up afterwards; under intermittent partitions (honest nodes split $2(n-f)/3$ and $(n-f)/3$), BFT finalization stalls, the parts' LOG_{da} drift apart, and on reunion nodes converge on the longer LOG_{da} while $\mathsf{LOG}_{\text{fin}}$ catches up. Simulation code: `https://github.com/tse-group/ebb-and-flow` (footnote 7). The paper offers no implementation against a deployed chain; this matters in §2.6.

2.5 The proof-of-work setting

Section V-B of [NTT2020] adapts the model to the shape Crosslink actually instantiates: nodes come in two flavors, *miners* (quantified by hash rate, powering the PoW longest chain) and *validators* (unique cryptographic identities, providing finality). The informal theorem for this setting reads: in a network environment where (1) communication is asynchronous until a global stabilization time GST after which it becomes synchronous, (2) validators are always awake, and (3) miners can sleep and wake at any time — (P1, Finality) the ledger $\mathsf{LOG}_{\text{fin}}$ is safe at all times and live after GST, if fewer than 33% of validators are adversarial, fewer than 50% of awake hash rate is adversarial, and awake hash rate is bounded away from zero; (P2, Dynamic Availability) if $\mathsf{GST} = 0$, the ledger LOG_{da} is safe and

live at all times, if fewer than 66% of validators are adversarial, fewer than 50% of awake hash rate is adversarial, and awake hash rate is bounded away from zero ([NTT2020] § V-B).

The paper explains why the requirements exceed the permissioned case: under P1, "fewer than 50% of awake hash rate have to be adversarial, and awake hash rate has to be bounded away from zero, since otherwise Π_{lc} might not be live and there would be no input for Π_{bft} to finalize"; under P2, "fewer than 66% of validators have to be adversarial, since otherwise adversarial validators could finalize Π_{lc} blocks that are not on the longest chain, which would later enter the prefix of LOG_{da}" ([NTT2020] § V-B). The paper closes the section with an open question, which we record verbatim: "It remains an open question whether these additional requirements are fundamental or a limitation of the snap-and-chat construction." Classification: *open problem*, the paper's own.

The deployed interface today. On the black-box side of the boundary, the only datum this volume records about the deployed Π_{bc} is its rollback bound: Zebra bounds its best non-finalized chain by `MAX_BLOCK_REORG_HEIGHT` (1000 blocks), whose documentation states "This is a local-only node policy; it is not part of consensus. The window is sized as a defence-in-depth measure against sustained consensus splits"; Zebra finalizes its oldest blocks once the chain grows past this height (`zebra @ d1fd7adf`, `zebra-chain/src/parameters/constants.rs:20--30`, `zebra-state/src/constants.rs:16--19`). Nothing in today's Zcash consensus provides finality; the constant is a node policy, which is precisely the gap Crosslink addresses.

2.6 Findings: where the book amends the paper

The book accepts the ebb-and-flow model as the starting point and then records defects and gaps that shape Crosslink. We reproduce each finding with both citations; all book material below is *designed but unspecified* (prose in `tfl-book` @ `fe6e1d6f`, textually unchanged in `zebra-crosslink` @ `6d02a1b8`; no normative artifact).

Scope of the one-honest-vote argument. The trichotomy case (c) above "is technically correct but has to be interpreted with care. It only applies when the number of malicious nodes f is such that $n/3 < f < n/2$. What we are trying to do with Crosslink is to ensure that a similar conclusion holds even if Π_{bft} is completely subverted, i.e. the adversary has 100% of validators (but only $< 50\%$ of Π_{lc} hash rate)" (`notes-on-snap-and-chat.md`, "Comment on security assumptions"). Crosslink's safety target is therefore strictly stronger than what Theorem 2.10 case (c) delivers.

A hidden safety assumption in Lemma 5. The proof of Lemma 2.12 assumes that "since Π_{lc} is safe ..., the fact that one honest node sees that snapshot as confirmed implies that every honest node sees the same snapshot as confirmed" ([NTT2020] Lemma 5 proof). The book replies that "while this may be a reasonable assumption to make for parts of the security analysis, in practice we should always require any adversary to do the relevant amount of Proof-of-Work to construct block headers that are plausibly confirmed" (`notes-on-snap-and-chat.md`, admonition quoting ePrint p. 9).

A subtlety in sanitization. Two readings of ledger extraction — concatenate transactions then sanitize, versus concatenate blocks, deduplicate blocks, then sanitize — "are equivalent in the setting considered by [NTT2020], but the argument for their equivalence is not obvious"; the equivalence requires that the *only* reasons for contextual invalidity in Π_{lc} are double-spends and missing inputs, and "strictly speaking Snap-and-Chat should require of Π_{lc} that there is no such other reason. This does not seem to be explicitly stated anywhere in [NTT2020]" (`notes-on-snap-and-chat.md`, "Subtlety in the definition of sanitization"). Zcash violates the premise: transaction expiry can be handled (the height-bounded validity of ZIP-203; *Wallet Guide*, §"Expiry (ZIP-203)"), but missing anchors break the argument, because a transaction invalid for a missing anchor can become valid later (anchors are the tree roots developed in the *Ironwood Guide*, §"Resting: the note in the commitment tree").

Spending finalized outputs. In snap-and-chat, transactions in ch_i^t must be able to spend outputs of finalized transactions that are not in ch_i^t's own history — they may come from another fork's snapshot. Since consensus validity of the tip must be a deterministic function of the chain itself, Π_{lc} "must include the information needed to calculate $\mathsf{LOG}^s_{\mathrm{fin},i}$". The book's assessment: "The authors of [NTT2020] probably just missed this: the paper only has evidence that they simulated their construction, rather than implementing it for Bitcoin or any other concrete block chain as Π_{lc}" (`notes-on-snap-and-chat.md`, "Spending finalized outputs" and "Finalization Availability"; compare the simulation-only evidence in §2.4). The repair adds a property the paper does not have — *finalization availability* — via a commitment $\mathsf{LF}(H)$ in each block header to the last final BFT block known to the block producer, governed by the following rule.

Definition 2.13 (Extension rule; `tfl-book` @ `fe6e1d6f`, `notes-on-snap-and-chat.md`)**.** If H is not the genesis block header, then the final BFT block committed to by H either descends from or equals the final BFT block committed to by H's parent:

$$\mathsf{LF}(H^{\lceil 1}) \preceq_{\mathrm{bft}} \mathsf{LF}(H).$$

The book states that the Extension rule "will be preserved into Crosslink 2", and that it yields, by construction and regardless of Π_{bft}: if node i observes no rollback deeper than σ in Π_{lc}, then i observes no instability in $\mathsf{LOG}_{\mathrm{fin},i}$, even if Π_{bft}'s assumptions are violated (`notes-on-snap-and-chat.md`, "Finalization Availability").

2.7 From dynamic availability to bounded availability

The book's one model-level departure from ebb-and-flow concerns what happens during a long partition. Since partition cannot be prevented, the CAP theorem — "that loosely speaking is made precise by [LR2020]" — implies that any finality-providing protocol may stall for as long as the partition lasts; strict dynamic availability then makes the finalization gap grow without bound (`tfl-book` @ `fe6e1d6f`, `the-arguments-for-bounded-availability-and-finality-overrides.md`). The book argues that spending under an unbounded gap is unacceptable, and replaces strict dynamic availability with *bounded availability* — a weakening of dynamic availability in the sense of [DKT2020, arXiv:2010.08154] — plus a *Stalled Mode* (coinbase-only blocks) and explicitly governed *finality overrides*. Block production keeps running during a stall rather than halting, because of tail-thrashing attacks: during a stall, an adversary with 10% of hash power expects to build a 10-block fork in 100 original-network block times at unchanged difficulty, and the k-block tail can "thrash" between chains (same chapter).

Definition 2.14 (Finality depth and the Stalled Mode rule; `tfl-book` @ `fe6e1d6f`, `notes-on-snap-and-chat.md`)**.** For a block header H, let

$$\begin{aligned} \mathsf{tailhead}(H) &:= \mathsf{lastcommonancestor}(\mathsf{snapshot}(\mathsf{LF}(H)), H), \\ \mathsf{finalitydepth}(H) &:= \mathsf{height}(H) - \mathsf{height}(\mathsf{tailhead}(H)). \end{aligned}$$

Consensus rule (Stalled Mode variant): for every Π_{lc} block H, either $\mathsf{finalitydepth}(H) \leq L$, or H follows the Stalled Mode restrictions (for Zcash: coinbase-only blocks), where L is the finality gap bound.

Definition 2.15 (Bounded hazard-freeness; `tfl-book` @ `fe6e1d6f`)**.** For a finality gap bound of L blocks: there is never observed to be, for any node i at time t, a more-available ledger $\mathsf{LOG}^t_{\mathrm{da},i}$

containing a hazardous transaction that originates in a block H of ch_i^t with $\mathsf{finalitydepth}(H) > L$ (`notes-on-snap-and-chat.md`, Definition "Bounded hazard-freeness").

Classification: the bounded-availability argument, Stalled Mode, and finality overrides are *designed but unspecified* — book prose with no draft ZIP (`zips` @ `e753a6a3`). The governance shape of finality overrides in particular has no normative text.

2.8 Crosslink 2's counterpart properties

The book restates the model's goals in a different proof style before replacing the construction: security arguments take the form "in an execution where safety properties are not violated, undesirable thing cannot happen", with probabilistic reasoning separated out (`tfl-book` @ `fe6e1d6f`, `construction.md` lines 342–359). On the relation to the paper: [NTT2020] Theorem 2 "essentially says that under certain conditions Prefix Agreement holds except with probability $e^{-\Omega(\sqrt{\sigma})}$"; the bound is not tight (footnote 10's bootstrapping via DKT+2020 § 4.2 brings it to $A_\varepsilon e^{-a_\varepsilon \sigma^{1-\varepsilon}}$); and in the proofs of Lemma 5 and Theorem 1 this probability "just passes through the proof from the premisses to the conclusions, because the argument is not probabilistic". The counterpart properties, in the book's execution-based style (the star is the book's protocol-name wildcard: $\Pi_{\star\mathrm{bc}}$ ranges over $\{\Pi_{\mathrm{origbc}}, \Pi_{\mathrm{bc}}\}$, likewise for bft, so each property is stated once for the original and the adapted subprotocol alike):

Definition 2.16 (Prefix Consistency; `tfl-book` @ `fe6e1d6f`, `construction.md`)**.** An execution of $\Pi_{\star\mathrm{bc}}$ has *Prefix Consistency* at confirmation depth σ iff for all times $t \leq u$ and all nodes i, j (potentially the same) such that i is honest at time t and j is honest at time u,

$$(\mathsf{ch}_i^t)^{\lceil\sigma} \preceq_{\mathrm{bc}} \mathsf{ch}_j^u.$$

(Taken from PSS2016's "consistency".)

Definition 2.17 (Prefix Agreement; `tfl-book` @ `fe6e1d6f`, `security-analysis.md`)**.** An execution of $\Pi_{\star\mathrm{bc}}$ has *Prefix Agreement* at confirmation depth σ iff for all times t, u and all nodes i, j (potentially the same) such that i is honest at time t and j is honest at time u,

$$(\mathsf{ch}_i^t)^{\lceil\sigma} \sim_{\mathrm{bc}} (\mathsf{ch}_j^u)^{\lceil\sigma}.$$

Definition 2.18 (Final Agreement; `tfl-book` @ `fe6e1d6f`, `security-analysis.md`)**.** An execution of $\Pi_{\star\mathrm{bft}}$ has *Final Agreement* iff for all $\star$bft-valid blocks C in honest view at time t and C' in honest view at time t', $\mathsf{bftlastfinal}(C) \sim_{\star\mathrm{bft}} \mathsf{bftlastfinal}(C')$.

Definition 2.19 (Assured Finality; `tfl-book` @ `fe6e1d6f`, `construction.md` line 505)**.** An execution of Crosslink 2 has *Assured Finality* iff for all times t, u and all nodes i, j (potentially the same) such that i is honest at time t and j is honest at time u, $\mathsf{fin}_i^t \sim_{\mathrm{bc}} \mathsf{fin}_j^u$.

Theorem 2.20 (Safety from either subprotocol; `tfl-book` @ `fe6e1d6f`, `security-analysis.md`)**.** *Prefix Agreement of Π_{bc} at confirmation depth σ implies Assured Finality; and, independently, Final Agreement of Π_{bft} implies Assured Finality. Safety of Crosslink 2's main goal therefore holds under*

> *reasonable assumptions about* either *subprotocol.*

The book notes that its $\mathsf{LOG}_{\mathrm{fin}}$ safety proof from Prefix Agreement "does not depend on any safety property of Π_{bft}, and is an elementary proof. ([NTT2020, Theorem 2] is a much more complicated proof that takes nearly 3 pages, not counting the reliance on results from [PS2017].)" This is the book's answer to the one-honest-vote scope finding of §2.6: the target regime includes a completely subverted Π_{bft}.

> **Theorem 2.21** (Ledger prefix property; `tfl-book` @ `fe6e1d6f`, `construction.md` lines 522–536)**.** *For any node i honest at time t, and any confirmation depth μ, $\mathsf{fin}_i^t \preceq (\mathsf{ba}_\mu)_i^t$, where $(\mathsf{ba}_\mu)_i^t = (\mathsf{ch}_i^t)^{\lceil\mu}$ if $\mathsf{fin}_i^t \preceq (\mathsf{ch}_i^t)^{\lceil\mu}$, and fin_i^t otherwise. This is the analogue of the Prefix property of Definition 2.8, and it holds by construction of $(\mathsf{ba}_\mu)_i^t$.*

The definitions above are shapes for comparison with [NTT2020]; the constructions behind them are canonical in this volume's construction section — fin and ba_μ in §3.6, the block structure in §3.3, and bft-last-final with the validity rules in §3.4 and §3.5.

Prefix Consistency and partitions. When Prefix Consistency is assumed of PoW protocols, it implicitly rules out partitions of the honest network longer than about σ block times; GKL2020 "proves" it only because the required no-partition assumption is implicit in the (partially) synchronous communication model. BFT protocols, by contrast, can be safe under full asynchrony — which is why Crosslink 2's communication model makes no implicit synchrony assumption, "or else we could end up with a protocol with weaker safety properties than Π_{origbft} alone" (`tfl-book` @ `fe6e1d6f`, `construction.md` lines 317–333; compare Remark 2.5).

What Crosslink 2 does not inherit. The model section must not imply that Crosslink 2 inherits [NTT2020]'s P2 as proved. The book itself notes that its "$\mathsf{LOG}_{\mathrm{ba}}$ does not roll back past the agreed $\mathsf{LOG}_{\mathrm{fin}}$" theorem "is not as strong as we would like", and of the bound L on non-Stalled-Mode blocks after the finalization point, "we have also not proven that so far" (`security-analysis.md`). Classification: Crosslink 2's dynamic availability analogue is an *open problem* in the book's own accounting; its safety counterparts (Theorem 2.20) are *designed but unspecified*. The published Prefix Agreement proof is complete; the published Final Agreement proof text is defective, but §6.1 supplies the missing argument. Neither has peer review, machine checking, or normative force. The Assured Finality definition already circulates beyond the book: the ZIPs repository's `draft-ecc-onchain-accountable-voting` cites it (`zips` @ `e753a6a`, line 169).

2.9 Context: Gasper, Casper FFG, and the successor line

The paper's motivating negative result is a balancing attack on Gasper (Ethereum's then-candidate consensus) in the standard synchronous model with adversarial — not stochastic — network delay, showing Gasper is not live and that its block proposal mechanism is rendered unsafe; for large n, any positive adversarial fraction f/n suffices, with the first opportune epoch after about n/f epochs and the per-epoch launch probability approximately β ([NTT2020] §§ II, V-A, Appendix A). Appendix E recapitulates the bouncing attack on Casper FFG and concludes that the "bidirectional interaction" between a finality gadget and its blockchain "is intricate to reason about and a gateway for liveness attacks", in contrast to the "completely decoupled" confirmation and finalization of snap-and-chat.

A finding follows for this volume: the book acknowledges Appendix E's warning while Crosslink 2 itself "involves a bidirectional dependence between Π_{bft} and Π_{bc}" — the book takes the bouncing attack as a lesson to be addressed, not a prohibition (`tfl-book` @ `fe6e1d6f`, `security-analysis.md` lines 15–20). The Extension rule of Definition 2.13 and the finality-depth rule of Definition 2.14 are both instances of that dependence.

The successor paper. The same authors identified a second dilemma: no protocol can provide both accountable safety and liveness under dynamic participation. Their resolution, *accountability gadgets*, checkpoints a longest-chain protocol using any BFT protocol as a black box ("The Availability-Accountability Dilemma and its Resolution via Accountability Gadgets", arXiv:2105.06075; latest v3 2022-05-18; checked 2026-09-05). We record it here for attribution and for the outlook; whether Crosslink's design draws on it is a question for the design-lineage section, not this one.

Deployment status. As verified on 2026-09-05, Crosslink is implementation-stage, not consensus-stage. Shielded Labs completed Milestone 5 on 2026-04-15 and now marks the Q2 2026–Q2 2027 productionization phase "In Progress". Its incentivized feature net has run prototype seasons, encountered stalled BFT finality and divergent chains, and by 2026-09-03 had tested the user-activated slashing recovery. No official ZIP or Mainnet activation is fixed (`https://shieldedlabs.net/roadmap/`; `https://forum.zcashcommunity.com/t/crosslink-incentivized-feature-net/55210`, posts 72 and 86).

2.10 Machine-checked status

One machine-checked artifact exists, and its scope must be stated precisely to avoid overclaiming. The Informal Systems Quint specification (`crosslink-spec` @ `8edc3e94`) is a proof-of-concept model of *only* the isValid check for a Crosslink BFT proposal against a single node's PoW and BFT block stores — the Tail Confirmation rule (σ-block tail) and the Linearity rule — with the PoW and BFT chains modelled abstractly (actions `add_pow` and `try_add_bft`). The README states that the logic was "inferred ... from an insightful Youtube talk" and "might not be up-to-date with the current protocol design ideas" (`README.md`). It does *not* formalize the ebb-and-flow ledgers $\mathsf{LOG}_{\mathrm{fin}}/\mathsf{LOG}_{\mathrm{da}}$ or the P1/P2/Prefix properties of Definition 2.8: the machine-checked coverage of the ebb-and-flow model itself is nil.

What is checked. As formalized (`validity.md`, "Validity Function"), the check is isValid(*bft_store*, *pow_store*, *p*) requiring: the store contains *p.block*; the proposal context equals `bft_store.last()`; sigmaConfirmed — a fork-free tail of at least σ PoW blocks, some element of which has the proposed block as its parent, and whose parents all lie within the tail or are the proposed block itself; and linear. The invariant `prefix_inv`, requiring for all consecutive indices *i*

```
pow_store.isDescendant(bft_store[i].pow_hash, bft_store[i+1].pow_hash)
```

and glossed "The PoW history defined by finalization is a path in the PoW blockstore", passed random simulation (10000 traces of 20 steps, no violation) and TLC model checking after transpiling to TLA+ on the verification scope (stores of up to 8 blocks, `confirmation_depth = 1`, about three minutes). The property `finalization_witness` generates traces with 3 BFT blocks, and `run finalizationTest` is an assertion-checked concrete execution. Files: `src/validity.qnt` (tangled from `validity.md` via `lmt`), `src/validity.tla`, `src/validity.cfg`; instances: verification with `confirmation_depth` = 1 and store limit 8, simulation with `confirmation_depth` = 2 and no limit (`README.md` §§ 2–3; `validity.md`, "Invariants"/"Tests", instances block lines 463–482).

Finding: divergences from the book. Function linear requires

```
work_store.isDescendant(context.pow_hash, b.hash)
      and not (context.pow_hash == b.hash),
```

the second conjunct carrying the specification's own comment "`// this actually seems allowed. TODO: perhaps remove`" — that is, it is *stricter* than the book's Linearity rule $\mathsf{snapshot}(B^{\lceil 1}_{\mathrm{bft}}) \preceq_{\mathrm{bc}} \mathsf{snapshot}(B)$, which permits equality. Additionally, isValid requires *p.bftContext* = `bft_store.last()` (proposals only on the current BFT tip), an abstraction not present as such

in the book (`crosslink-spec` @ `8edc3e94`, `validity.md` lines 131–176; `tfl-book` @ `fe6e1d6f`, `construction.md`, Linearity rule).

The repository `quint-crosslink` (`github.com/zmanian/quint-crosslink`) contains no commits locally, and `git ls-remote` against the upstream returns no refs — the repository is empty upstream as well (checked 2026-09-05). The Quint ground truth is `crosslink-spec` in its entirety.

Classification. The Quint artifact itself is *specified* in the only sense available before a protocol specification: verifiable, machine-checked source at a pinned commit. The claim it checks (`prefix_inv`) is far weaker than the model properties of this section; every ebb-and-flow-level property of Crosslink 2 remains *designed but unspecified* or *open*, per §2.8.

3 The Crosslink construction

Crosslink couples Zcash's proof-of-work best-chain protocol to a Byzantine-fault-tolerant (BFT) protocol so that transactions become final some time after they first appear in PoW blocks — the property the Trailing Finality Layer book names *Trailing Finality*, and whose target is *Assured Finality*: the assurance that transactions cannot be reverted *by that protocol*. The book deliberately eschews "absolute finality", because out-of-band forks can always revert anything (tfl-book @ `fe6e1d6`, `src/terminology.md`:11–13, 45). The two-ledger shape — a conservative finalized ledger beside an available one — descends from the ebb-and-flow paradigm of Neu, Tas, and Tse, "Ebb-and-Flow Protocols: A Resolution of the Availability-Finality Dilemma" (arXiv 2009.04987, v3 of 2021-02-04; IEEE S&P 2021; checked 2026-09-05).

The current version of the construction is *Crosslink 2*: the book's construction chapter defines it as such, and version-history entry 0.2.0 records the update of both the construction and its security analysis from Crosslink 1 (`construction.md`:1–3; `src/design/crosslink.md`:3; `src/version-history.md`). Throughout this section, a citation of the form `construction.md`:*n* means `src/design/crosslink/construction.md` in the ECC Trailing Finality Layer book at commit `fe6e1d6`; other files of that repository are cited by path at the same commit.

Crosslink 2 modifies a BFT protocol Π_{origbft} and a best-chain protocol Π_{origbc} into two coupled protocols Π_{bft} and Π_{bc}, by adding structural elements, changing validity rules, and changing honest-node behaviour. A Crosslink 2 node must participate in both Π_{bft} and Π_{bc}; acting as a bft-proposer, a bft-validator, or a bc-block-producer is optional, but all such actors are assumed to be Crosslink 2 nodes (`construction.md`:103–111).

The stakes for the rest of the Zcash stack are concrete. Today confirmation depth is policy, not property: wallets gate spendability on heuristic depths (*Wallet Guide*, §"Confirmations, trust, and spendability") and recover from rollbacks by truncate-and-rescan (*Wallet Guide*, §"Reorganisations: truncate and rescan"); an Orchard anchor is only as settled as the chain suffix beneath it (*Ironwood Guide*, §"Resting: the note in the commitment tree"); and Zally's coin-weighted vote pins its electorate to off-chain roots for a Zcash snapshot that is not consensus-final today. The vote chain stores the coordinator's roots verbatim, and the circuit explicitly does not authenticate their provenance (`vote-sdk/api/snapshot.go`:44–66; `vote-sdk/x/vote/keeper/msg_server.go`:36–138; `voting-circuits/src/delegation/README.md`, "Public Input Provenance"; `valargroup/vote-sdk` @ `cb915f51` and `valargroup/voting-circuits` @ `4c39abd5`). Crosslink 2 would replace the rollback heuristics with a protocol property; it would not remove Zally's independent provenance assumption.

Classification. Every rule in this section is *designed but unspecified*: the book is the construction source of record, and no Crosslink or trailing-finality ZIP exists. As of zips @ `e753a6a` (2026-09-03), the corpus mentions Crosslink only in a citation footnote in draft ZIP 218 ("25-second Block Target Spacing") and in the accountable-voting draft's citation of Assured Finality. Neither specifies Crosslink. Two artifact classes attach *specified* status, in the pre-spec sense of verifiable pinned

sources, to fragments of the material: a machine-checked Quint model of one validity check (§3.8) and prototype code (§3.9) — and both diverge from the book text in ways we record. Per-subsection Classification paragraphs refine this default.

3.1 Model and notation

The communication model takes neither synchrony nor partial synchrony as an implicit assumption: unless otherwise specified, messages can be arbitrarily delayed or dropped; a message is received at most once, and is nonmalleably authenticated. The book argues at length against applying the Global Stabilization Time formalization of partial synchrony (Dwork, Lynch, and Stockmeyer, 1988) to liveness of non-terminating block-chain protocols (`construction.md`:13–41). Where a subprotocol's own analysis needs timing assumptions — as Streamlet's liveness analysis does below — those assumptions are imported explicitly, not ambiently.

Throughout, subscript $\star$ ranges over the two chain flavours bc and bft; a block determines the chain of its ancestors with itself as tip, and we pass between block and chain freely. In this book's model, "honest at time t" means having followed the protocol at every time up to and including t, not merely behaving honestly at that instant (`construction.md`:53).

Definition 3.1 (Chain notation)**.** Let C be a $\star$-chain and $k \in \mathbb{N}$.

(i) *Truncation.* The chain $C\lceil^k_\star$ is C with the last k blocks pruned, except that if $\mathrm{len}(C) \leq k$ the result is the genesis chain consisting only of $\mathsf{Origin}_\star$. The block at depth k in C is the tip of $C\lceil^k_\star$. For a bft-proposal P with parent block B and $k \geq 1$, define $P\lceil^k_{\mathrm{bft}} := B\lceil^{k-1}_{\mathrm{bft}}$ (`construction.md`:61–63, 121).

(ii) *Prefix and agreement.* We write $B \preceq_\star C$ iff the chain with tip B is a prefix of the chain with tip C (including $B = C$). Blocks B and C *agree*, written $B \sim_\star C$, iff $B \preceq_\star C$ or $C \preceq_\star B$; they *conflict* otherwise (`construction.md`:69–75).

(iii) *Linearity.* A time series $S : I \to \star$-block is $\star$-linear iff $t \leq u$ implies $S(t) \preceq_\star S(u)$ (`construction.md`:69–75).

Lemma 3.2 (Linear prefix)**.** *If $A \preceq_\star C$ and $B \preceq_\star C$, then $A \sim_\star B$.*

Proof. The chain of ancestors of C is $\star$-linear, and blocks A and B both lie on that chain (`construction.md`:77–82). □

Definition 3.3 (Agreement on a view)**.** An execution of a protocol Π has *Agreement on the view* $V : \mathrm{Node} \times \mathrm{Time} \to \star$-chain iff for all times t, u and all Π nodes i, j (potentially the same) such that i is honest at time t and j is honest at time u, we have $V_i^t \sim_\star V_j^u$ (`construction.md`:96–99).

Remark 3.4 (Depth conventions)**.** The book's "depth" differs from both neighbouring conventions: NTT2020 uses *depth* for what the book calls height, and the book's count differs by one from the zcashd confirmation-depth convention — the tip sits at depth 0, not 1 (`construction.md`:65–67; arXiv 2009.04987). The confirmation-depth defaults quoted from the *Wallet Guide* in §3.7 use the wallet convention.

3.2 The two black boxes

Consistent with the scope of this series, both subprotocols enter the construction only through their interfaces. We record what Crosslink 2 assumes of each; we do not derive the internals of either — in

particular, PoW block production remains a black box.

Requirements on Π_{origbc}. Block producers are selected by a resource-proportional random process. A function score : bc-valid-chain → Score is fixed, with < a strict total order on Score and, for any non-genesis bc-valid-chain H, $\text{score}(H\lceil^{1}_{\text{bc}}) < \text{score}(H)$; for PoW instantiations the score is accumulated work. Honest nodes choose a highest-scoring valid chain as their best chain, with any tie-breaking rule. Headers commit to blocks, and header validity is necessary but not sufficient for block validity (`construction.md`:245–290, 255–257). The model also fixes an abstract predicate iscoinbaseonlyblock : bc-block → boolean, true iff the block has exactly one transaction and that transaction is a coinbase transaction — one that only distributes newly issued funds and has no inputs (`construction.md`:269–271). The last gloss is the book's abstraction, not the deployed Zcash definition: a Zcash coinbase transaction has one null-prevout transparent input and collects miner subsidy plus transaction fees (Zcash Protocol Specification, §"Coinbase Transactions"). Later uses of the predicate refer only to the book's abstract one-transaction block class.

The two chain-quality properties the analysis consumes are stated as unconditional properties of executions; the probabilistic reasoning that makes them plausible for a PoW protocol is deliberately separated from the construction (`construction.md`:245–290, 337–366).

Definition 3.5 (Prefix Consistency)**.** An execution of Π_{bc} has *Prefix Consistency* at confirmation depth σ iff for all times $t \le u$ and all nodes i, j (potentially the same) such that i is honest at time t and j is honest at time u, we have $\text{ch}_i^t\lceil^{\sigma}_{\text{bc}} \preceq_{\text{bc}} \text{ch}_j^u$ (`construction.md`:287–290; the property is PSS2016 §3.3 "consistency").

Definition 3.6 (Prefix Agreement)**.** An execution of Π_{bc} has *Prefix Agreement* at confirmation depth σ iff it has Agreement on the view $(i, t) \mapsto \text{ch}_i^t\lceil^{\sigma}_{\text{bc}}$ (`construction.md`:337–340).

Requirements on Π_{origbft}. The BFT protocol proceeds in epochs with proposals, ballots, and a *notarization rule*. The notarization rule must require at least a two-thirds absolute supermajority of the epoch's voting units to have been cast for a proposal P, and may require other conditions; a notarization proof can be obtained only with knowledge of ballots collectively satisfying the rule (`construction.md`:129).

A ⋆*bft-block* consists of (P, proof_P) re-signed by the same proposer using a strongly unforgeable signature scheme (*Crypto Guide*, §"Digital signatures"). It is bft-block-valid iff P is bft-proposal-valid, proof_P validly proves that the notarization rule was satisfied by ballots cast for P, and the proposer's outer signature on (P, proof_P) is valid. A bft-proposal's parent reference hashes the *entire* parent bft-block: proposal, proof, and outer signature (`construction.md`:156–161).

A voting unit is cast *non-honestly* iff it is cast other than by its holder (for example after key compromise), it is double-cast for distinct proposals, or its holder should not have cast that vote according to the honest-voting conditions in its view (`construction.md`:131–154).

Definition 3.7 (One-third bound on non-honest voting)**.** An execution of Π_{bft} has the *one-third bound on non-honest voting* property iff for every epoch, strictly fewer than one third of the total voting units for that epoch are *ever* cast non-honestly (`construction.md`:138–141).

The bound quantifies over ballots cast at *any* time in the entire execution. Consequently the validator keys of honest nodes must remain secret indefinitely, and rotated-out keys must be securely deleted (`construction.md`:131–154; tfl-book issue #140).

Finality inside the BFT protocol is a function of context, never an objective predicate: it is correct to talk about the last final block *of a chain*, but not to call any bft-block objectively bft-final (`construction.md`:171–187).

Definition 3.8 (Required properties of bft-last-final)**.** Function bft-last-final : bft-block → bft-block-or-⊥ outputs, for a bft-block-valid context C, the last ancestor of C that is final in the context of C. Required: bft-last-final(C) = ⊥ iff C is not bft-block-valid; if C is bft-block-valid, then bft-last-final(C) $\preceq_{\text{bft}}$ C (hence is itself bft-block-valid), and for all bft-valid-blocks D with $C \preceq_{\text{bft}} D$, bft-last-final(C) $\preceq_{\text{bft}}$ bft-last-final(D). Also bft-last-final($\mathsf{Origin}_{\text{bft}}$) = $\mathsf{Origin}_{\text{bft}}$ (`construction.md`:177–183).

Definition 3.9 (Final Agreement)**.** An execution of Π_{bft} has *Final Agreement* iff for all bft-valid blocks C in honest view at time t and C' in honest view at time t', we have bft-last-final(C) $\sim_{\text{bft}}$ bft-last-final(C'). A block is *in honest view* if a party observes it at some time at which that party is honest (`construction.md`:200–206).

Worked instance: adapting Streamlet. The book's running Π_{origbft} is an adapted Streamlet. Its notion of "notarized" is identified with bft-block-validity by having the proposer aggregate a two-thirds-supermajority proof into a submitted block, making notarization objectively and feasibly verifiable, so that valid chains consist of notarized blocks; the liveness timing analysis then assumes a notarized block is received within two epochs, and progress needs five consecutive epochs with honest leaders. Streamlet's finality rule — three adjacent blocks with consecutive epoch numbers finalize the chain up to the second — becomes origbft-last-final: for an origbft-valid-block C, origbft-last-final(C) is the last origbft-valid-block $B \preceq_{\text{origbft}} C$ such that either $B = \mathsf{Origin}_{\text{origbft}}$ or B is the second block of a group of three adjacent blocks with consecutive epoch numbers (`construction.md`:210–243, 224, 234–241, 618).

Classification. *Designed but unspecified* throughout; the requirements are the book's model choices, argued but not proven against any deployed BFT protocol. A specification would need to fix the concrete Π_{origbft}, its voting-unit assignment, and the notarization proof format.

3.3 Structural additions

Crosslink 2 adds three structural elements (`construction.md`:417–421):

(1) each bc-header has, in addition to the origbc-header fields, a context_bft field that commits to a bft-block;

(2) each bft-proposal has a headers_bc field containing a sequence of exactly σ bc-headers, zero-indexed and deepest first;

(3) each non-genesis bft-block likewise has a headers_bc field with exactly σ bc-headers; the genesis bft-block has headers_bc = null.

Definition 3.10 (Snapshot)**.** For a bft-block or bft-proposal B:

$$\mathsf{snapshot}(B) := \begin{cases} \mathsf{Origin}_{\text{bc}} & \text{if } B.\mathsf{headers\_bc} = \mathsf{null}, \\ B.\mathsf{headers\_bc}[0]\lceil_{\text{bc}}^{1} & \text{otherwise} \end{cases}$$

(`construction.md`:423–430). The snapshot is the parent of the deepest supplied header, so the σ headers sit directly on top of it.

Definition 3.11 (LF and the finalization candidate)**.** For a bc-block H:

$$\mathsf{LF}(H) \coloneqq \text{bft-last-final}(H.\mathsf{context\_bft}),$$
$$\mathsf{candidate}(H) \coloneqq \text{last-common-ancestor}\big(\mathsf{snapshot}(\mathsf{LF}(H)),\ H\lceil^{\sigma}_{\mathrm{bc}}\big).$$

When H is the tip of a node's bc-best-chain, $\mathsf{candidate}(H)$ gives the candidate finalization point, subject to the local no-rollback condition of updatefin (Construction 3.17; `construction.md`:431–438).

Why full headers. The headers_bc field demonstrates to any party seeing a proposal or block that the snapshot had been confirmed when the proposal was made. Full headers, rather than hashes, let a validator check Proofs-of-Work and difficulty adjustment locally *before* downloading blocks, limiting denial-of-service; nodes may trim headers overlapping with ancestor bft-blocks. The genesis special case — $\mathsf{headers\_bc} = \mathsf{null}$ with the snapshot patched to $\mathsf{Origin}_{\mathrm{bc}}$ — avoids a hash cycle between the two genesis blocks (`construction.md`:440–454).

Why only context_bft**.** No separate final_bft field is needed in bc-headers: the context block alone determines its last final ancestor, and bft-last-final is efficiently computable for typical BFT protocols (`construction.md`:456–460).

Classification. *Designed but unspecified*. A specification would need to pin the commitment encoding of context_bft, the header wire format, and the trimming rule.

3.4 The BFT side: Π_{bft}

Definition 3.12 (Π_{bft} validity)**.** The genesis rule is that $\mathsf{Origin}_{\mathrm{bft}}$ is bft-block-valid (`construction.md`:568). A bft-proposal (respectively, non-genesis bft-block) B is bft-proposal-valid (respectively, bft-block-valid) iff *all* of:

Inherited origbft rules. The corresponding origbft-proposal-validity (respectively, origbft-block-validity) rules hold for B; these are assumed to include the *Parent rule* that B's parent is bft-valid.

Linearity rule. $\mathsf{snapshot}(B\lceil^{1}_{\mathrm{bft}}) \preceq_{\mathrm{bc}} \mathsf{snapshot}(B)$.

Tail Confirmation rule. The headers B.headers_bc form the σ-block tail of a bc-valid-chain.

(`construction.md`:570–577.)

Finality-in-context is unchanged from Π_{origbft}: bft-last-final is defined the same way as origbft-last-final (`construction.md`:621–623).

Objective validity versus honest voting. The validity rules are separated from the honest voting condition on purpose: even a proposal voted for by 100% of validators is not bft-proposal-valid unless it satisfies the rules, and a purportedly valid bft-block carrying a valid notarization proof is not recognized by *any* honest node if it fails the other bft-block-validity rules. This separation is essential to keeping the finalized chain safe against a complete break of Π_{bft} — even of the validator signature algorithm — as long as Π_{bc} remains safe (`construction.md`:581–585).

The Linearity rule. Within a bft-valid-chain, the referenced bc-snapshots cannot roll back. The rule replaces and implies Crosslink 1's Increasing Score rule; it prevents attacks that propose bc-chains forked from much earlier (lower-difficulty) blocks; it bounds the disruption to the bounded-available ledger even if Π_{bft} is subverted, because the finalized chain *is* a Π_{bc} chain; and it eliminates the Snap-and-Chat/Crosslink 1 "sanitization" step entirely. The rule is relative to the parent bft-block's snapshot even when that parent is not yet final in the current context (`construction.md`:587–609).

The rule deliberately permits keeping the *same* snapshot as the parent. The allowance is required to preserve liveness of Π_{bft} relative to Π_{origbft}: honest proposers may need to propose at a minimum rate — Streamlet, for example, needs three notarized blocks in consecutive epochs and assumes proposals from honest leaders each epoch. The alternative of null proposals was rejected because it would require specifying null-proposal rules in Π_{origbft} (`construction.md`:611–619). The machine-checked model of §3.8 diverges from the book exactly here.

> **Construction 3.13** (Honest proposal)**.** An honest proposer chooses P.headers_bc as the σ-block tail of its bc-best-chain if that is consistent with the Linearity rule, and otherwise sets P.headers_bc to the same headers_bc as P's parent bft-block. It makes no proposals until its bc-best-chain is at least $\sigma + 1$ blocks long, since otherwise headers_bc cannot be constructed; at exactly $\sigma + 1$ blocks the snapshot is $\text{Origin}_{\text{bc}}$, which satisfies Linearity because $\text{Origin}_{\text{bft}}$'s snapshot is $\text{Origin}_{\text{bc}}$ (`construction.md`:625–637).

> **Construction 3.14** (Honest voting)**.** A validator first updates its view of both subprotocols with the bc-headers in P.headers_bc, downloading and validating the referenced bc-blocks, and recursively any unseen bft-chains referenced by their context_bft fields. The validation order — supermajority checks first, then PoW header checks, before block download — bounds denial-of-service, and the Linearity rule ensures only one bc-chain need be validated per bft-chain. The validator votes for P *only if*:
>
> **Valid proposal criterion.** Proposal P is bft-proposal-valid.
>
> **Confirmed best-chain criterion.** The chain snapshot(P) is part of the validator's bc-best-chain at a bc-confirmation-depth of at least σ.
>
> (`construction.md`:639–657.)

The Confirmed best-chain criterion is subtle. It ensures that snapshot(P) is bc-block-valid with σ bc-block-valid blocks after it in the validator's best chain, but P.headers_bc[$\sigma - 1$] need *not* be in the validator's best chain — the chain may fork after snapshot(P). Proposal validity must remain objective. Versus Snap-and-Chat: that construction had the confirmed-snapshot voting condition, but neither shipped headers with proposals nor made objective confirmation a validity matter; shipping headers improves the finalization-latency/security trade-off (`construction.md`:659–678).

Classification. *Designed but unspecified*. The prototype does not yet enforce this subsection's rules on its voting path (§3.9).

3.5 The best-chain side: Π_{bc}

All Crosslink 2 consensus-rule changes to Π_{bc} are non-contextual; modulo them, contextual validity is the same as in Π_{origbc}. The bc-transaction consensus rules are entirely unchanged: no reordering, no sanitization; parent bc-chains and heights are preserved (`construction.md`:267, 701–703).

Definition 3.15 (Π_{bc} validity)**.** For the genesis bc-block we must have $\mathsf{Origin}_{\mathrm{bc}}.\mathsf{context\_bft} = \mathsf{Origin}_{\mathrm{bft}}$, and therefore $\mathsf{LF}(\mathsf{Origin}_{\mathrm{bc}}) = \mathsf{Origin}_{\mathrm{bft}}$ (`construction.md`:684). A bc-block H is bc-block-valid iff *all* of:

Inherited origbc rules. Block H satisfies the corresponding origbc-block-validity rules.

Valid context rule. H.context_bft is bft-block-valid.

Extension rule. $\mathsf{LF}(H\lceil^{1}_{\mathrm{bc}}) \preceq_{\mathrm{bft}} \mathsf{LF}(H)$.

Last Final Snapshot rule. $\mathsf{snapshot}(\mathsf{LF}(H)) \preceq_{\mathrm{bc}} H$.

Finality depth rule. Define $\text{finality-depth}(H) \coloneqq \text{height}(H) - \text{height}(\mathsf{snapshot}(\mathsf{LF}(H)))$; then either $\text{finality-depth}(H) \leq L$ or is-stalled-block(H).

(`construction.md`:686–693; the community protocol guide restates the Last Final Snapshot rule verbatim, crosslink-protocol-guide @ `b77d845`, `src/tfl-lexicon.md`:3–5.)

Stalled Mode. The finality gap at time t is, roughly, the number of bc-blocks not yet finalized; the Finality depth rule makes this precise as the objective finality-depth of a block. If the gap exceeds the threshold L, that likely signals an exceptional or emergency condition in which accepting spends into new bc-blocks is undesirable. The Stalled Mode policy is modelled by the predicate is-stalled-block : bc-block → boolean; a block satisfying it is a *stalled block* (`construction.md`:401–413). The node-local prose notion ("blocks not yet finalized at time t") and the objective per-block notion differ in corner cases just after a reorg (`construction.md`:695–699).

Two cautions. First, a bc-block producer is only constrained to produce stalled blocks while its view of the finalization point is not advancing; an adversary that subverts the BFT protocol *without* stalling finalization is never constrained by Stalled Mode (`construction.md`:401–413). Second, the book warns that an honest bc-block-producer must *not* use information from the BFT protocol beyond the specified consensus rules to decide which bc-valid-chain to follow; additional constraints could let an adversary that subverts Π_{bft} influence the bc-best-chain in ways outside the safety argument (`construction.md`:715–717).

Construction 3.16 (Honest block production)**.** The producer must follow the Π_{bc} validity rules, including producing a stalled block when the Finality depth rule requires it. To choose H.context_bft it considers the set

$$\{\, T \,:\, T \text{ is bft-block-valid and } \mathsf{LF}(H\lceil^{1}_{\mathrm{bc}}) \preceq_{\mathrm{bft}} \text{bft-last-final}(T) \,\},$$

chooses one of the longest such chains C — breaking ties first by maximizing the score of the snapshot of bft-last-final(C), then by smallest hash — and sets H.context_bft $= C$. Rationale: choosing a longest chain follows Streamlet's build-on-longest-notarized; the score tie-break inhibits an adversary from finalizing a lesser-scoring chain (`construction.md`:705–731).

Classification. *Designed but unspecified*; the deployment track has, at the pinned commit, dropped the Stalled Mode rules and does not enforce the Extension, Last Final Snapshot, or Finality depth rules in bc-block validity (§3.9).

3.6 The two ledgers

Each node i derives two chains from its views: a locally finalized chain fin^{t}_{i} and a locally bounded-available chain $(\mathsf{ba}_{\mu})^{t}_{i}$, for a per-node confirmation depth μ. The book's macros render the task

names localfin and $\mathsf{localba}_\mu$ as fin and ba, and $\mathsf{snapshotlf}(H)$ as $\mathsf{snapshot}(\mathsf{LF}(H))$ (tfl-book @ `fe6e1d6`, `macros.txt`:29, 63–64); we use the rendered forms.

The finalized chain. The locally finalized chain starts at $\mathsf{fin}_i^0 = \mathsf{Origin}_{\mathrm{bc}}$ and must not be exposed to clients until the node has synced (`construction.md`:462–488).

Construction 3.17 (Finalization update (updatefin))**.** On a best-chain update at time t, with previous state fin_i^s, node i computes $N \coloneqq \mathsf{candidate}(\mathsf{ch}_i^t)$ and:

(i) if $N \succeq_{\mathrm{bc}} \mathsf{fin}_i^s$, sets $\mathsf{fin}_i^t \leftarrow N$;

(ii) otherwise sets $\mathsf{fin}_i^t \leftarrow \mathsf{fin}_i^s$, and if additionally $N \not\preceq_{\mathrm{bc}} \mathsf{fin}_i^s$ — that is, N conflicts with fin_i^s — records a *finalization safety hazard*, possible only if the global safety assumptions are violated. The hazard record includes ch_i^t and the fin update history since the last update that was an ancestor of N.

(`construction.md`:475–486.)

Definition 3.18 (Local finalization linearity)**.** Node i has *Local finalization linearity* up to time t iff the time series of bc-blocks $\mathsf{fin}_i^{r \le t}$ is bc-linear (`construction.md`:466–469).

The update rule guarantees Local finalization linearity *by construction* (`construction.md`:462–488). Local state is needed because on a reorg — even one shorter than σ — the value $\mathsf{candidate}(\mathsf{ch}_i^t)$ may temporarily roll back; if Final Agreement holds, the new snapshot is an ancestor of the old and will roll forward along the same path, but applications must not see the temporary rollback (`construction.md`:497–499). This is precisely the service today's applications lack: wallets repair rollbacks after the fact by truncate-and-rescan (*Wallet Guide*, §"Reorganisations: truncate and rescan"), and anchor validity tracks the unfinalized chain (*Ironwood Guide*, §"Resting: the note in the commitment tree").

Lemma 3.19 (Local fin-depth)**.** *In any execution of Crosslink 2, for any node i honest at time t, there exists a time $r \le t$ such that* $\mathsf{fin}_i^t \preceq_{\mathrm{bc}} \mathsf{ch}_i^r\lceil_{\mathrm{bc}}^{\sigma}$.

Proof. By $\mathsf{candidate}(\mathsf{ch}_i^u) \preceq_{\mathrm{bc}} \mathsf{ch}_i^u\lceil_{\mathrm{bc}}^{\sigma}$ applied at the last time fin changed, or at genesis (`construction.md`:490–495). □

Definition 3.20 (Assured Finality)**.** An execution of Crosslink 2 has *Assured Finality* iff for all times t, u and all nodes i, j (potentially the same) such that i is honest at time t and j is honest at time u, we have $\mathsf{fin}_i^t \sim_{\mathrm{bc}} \mathsf{fin}_j^u$ (`construction.md`:504–506).

Assured Finality is the main safety goal of Crosslink 2, intended to hold under reasonable assumptions about *either* Π_{origbft} *or* Π_{origbc}. Its restriction to $i = j$ says that one node's observations are pairwise compatible, not that they advance monotonically: a rollback along the same branch preserves compatibility. The source's claimed equivalence to Local finalization linearity (`construction.md`:508) therefore needs the separate no-rollback update rule. That rule guarantees local monotonicity independently; agreement between different nodes is the additional safety property.

Why $\mathsf{candidate}(H)$**, not** $\mathsf{snapshot}(\mathsf{LF}(H))$**.** The Last Final Snapshot rule guarantees $\mathsf{snapshot}(\mathsf{LF}(H)) \preceq_{\mathrm{bc}} H$, but *not* that its depth relative to H is at least σ. Using $\mathsf{candidate}(H)$ ensures the finalization point is at least σ-confirmed, which the proof of Assured Finality needs: $\mathsf{candidate}(H) \preceq_{\mathrm{bc}} H\lceil^{\sigma}_{\mathrm{bc}}$ combines with the Local fin-depth lemma and Prefix Agreement. An open book TODO offers the alternative of strengthening the Last Final Snapshot rule to $\mathsf{snapshot}(\mathsf{LF}(H)) \preceq_{\mathrm{bc}} H\lceil^{\sigma}_{\mathrm{bc}}$ (`construction.md`:510–518).

Definition 3.21 (Locally bounded-available chain)**.** For a per-node confirmation depth μ:

$$(\mathsf{ba}_\mu)^t_i = \begin{cases} \mathsf{ch}^t_i\lceil^{\mu}_{\mathrm{bc}} & \text{if } \mathsf{fin}^t_i \preceq_{\mathrm{bc}} \mathsf{ch}^t_i\lceil^{\mu}_{\mathrm{bc}}, \\ \mathsf{fin}^t_i & \text{otherwise} \end{cases}$$

(`construction.md`:522–527).

The "otherwise" arm is needed because after the best chain switches forks under Zcash's damped difficulty adjustment (`PoWAveragingWindow` = 17 blocks, `PoWMedianBlockSpan` = 11 blocks, quoted by the book from the Zcash protocol specification § diffadjustment), the truncated best chain $\mathsf{ch}^t_i\lceil^{\mu}_{\mathrm{bc}}$ might not descend from fin^t_i even with all safety assumptions satisfied. The source says that fin switches forks, but the no-rollback update rule prohibits that; the mutable best-chain view is the relevant object. The definition also makes the Ledger prefix property trivial to prove. The security goal for ba_μ is Agreement on the view ba_μ (Definition 3.3; `construction.md`:545–555, 552).

Theorem 3.22 (Ledger prefix property)**.** *For any node i that is honest at time t, and any confirmation depth μ,* $\mathsf{fin}^t_i \preceq_{\mathrm{bc}} (\mathsf{ba}_\mu)^t_i$.

Proof. By construction of $(\mathsf{ba}_\mu)^t_i$ (`construction.md`:531–536). □

Lemma 3.23 (Local ba-depth)**.** *In any execution of Crosslink 2, for any confirmation depth $\mu \le \sigma$ and any node i honest at time t, there exists a time $r \le t$ such that* $(\mathsf{ba}_\mu)^t_i \preceq_{\mathrm{bc}} \mathsf{ch}^r_i\lceil^{\mu}_{\mathrm{bc}}$.

Proof. Either $(\mathsf{ba}_\mu)^t_i = \mathsf{fin}^t_i$, and the Local fin-depth lemma applies with $\mu \le \sigma$; or $(\mathsf{ba}_\mu)^t_i = \mathsf{ch}^t_i\lceil^{\mu}_{\mathrm{bc}}$, trivially with $r = t$ (`construction.md`:538–543). □

Stall, do not finalize unsafely. In normal operation fin^t_i lags only a few blocks behind $(\mathsf{ba}_\sigma)^t_i$, unless the chain enters Stalled Mode. With $\mu = \sigma$, the choice between following fin and following ba is a choice of stall behaviour: stall immediately (fin), or keep processing user transactions until L blocks have passed (ba) (`construction.md`:415).

Syncing and checkpoints. Implementations should bake a checkpointed bft-block $B_{\mathrm{checkpoint}}$ into each release; node i exposes fin^t_i and $(\mathsf{ba}_\mu)^t_i$ only once synced, that is, once $B_{\mathrm{checkpoint}} \preceq_{\mathrm{bft}} \mathsf{LF}(\mathsf{ch}^t_i)$, $\mathsf{snapshot}(B_{\mathrm{checkpoint}}) \preceq_{\mathrm{bc}} \mathsf{fin}^t_i$, and the timestamp of fin^t_i is within a threshold of the current time (`construction.md`:557–562).

The chapter closes: "At this point we have completed the definition of Crosslink 2. In Security Analysis of Crosslink 2, we will prove it secure." The structural rules above are the complete definition; all proofs beyond the in-chapter lemmas live in `security-analysis.md` (`construction.md`:735); this volume's account of that analysis is §6.

Classification. *Designed but unspecified*, with the candidate-versus-strengthened-rule fork explicitly open in the book.

3.7 Parameters

The construction has three parameters and fixes production values for none of them.

- The bc-confirmation-depth $\sigma \in \mathbb{N}^+$ (`construction.md`:391). No production value is fixed anywhere in the sources.
- The finalization gap bound $L \in \mathbb{N}^+$, significantly greater than σ; "In practice, L should be at least 2σ" (`construction.md`:391, 405).
- The per-node depth μ with $0 < \mu \leq \sigma$; the default and recommendation is $\mu = \sigma$. A smaller μ is at the node's own risk and does not affect fin (`construction.md`:395–399).

Both prototype snapshots ship `PROTOTYPE_PARAMETERS` with $\sigma = 3$ and $L = 7$ — satisfying $L \geq 2\sigma$. The April source adds the explicit caveat "No verification has been done on the security or performance of these parameters" (zebra-crosslink @ `6d02a1b`, `zebra-crosslink/src/chain.rs`:216–222; crosslink_monolith @ `807b995`, `librustzcash/zcash_primitives/src/bft.rs`:388–404). For calibration: today's wallet policy defaults are 3 confirmations for trusted and 10 for untrusted outputs under the ZIP-315 `ConfirmationsPolicy` (*Wallet Guide*, §"Confirmations, trust, and spendability") — a per-wallet risk heuristic, where σ is a consensus-visible input to finalization. On the node side, the k-deep interface constant of the deployed best-chain protocol is `MAX_BLOCK_REORG_HEIGHT` = 1000 in mainline Zebra, documented as "a local-only node policy; it is not part of consensus" and sized as defence-in-depth against sustained consensus splits (zebra @ `d1fd7ad`, `zebra-chain/src/parameters/constants.rs`:30); the zebra-crosslink fork still uses the older value 99 = `MIN_TRANSPARENT_COINBASE_MATURITY` − 1 (zebra-crosslink @ `6d02a1b`, `zebra-state/src/constants.rs`:31, `zebra-chain/src/transparent.rs`:52). How that constant will relate to σ is undetermined.

Classification. The parameter constraints are *designed but unspecified*; the concrete values are an *open problem*. The quoted constants are *specified* in code at the pinned commits.

3.8 The machine-checked fragment

The Informal Systems Quint specification (crosslink-spec @ `8edc3e9`) is an explicit proof of concept covering *only* the `isValid` check for BFT proposals — Tail Confirmation plus Linearity — over abstract single-node PoW and BFT block stores. Its README states the logic was inferred mostly from a YouTube talk and "might not be up-to-date with the current protocol design ideas". The Quint file `src/validity.qnt` is tangled from `validity.md` via the `lmt` tool (README.md:25–32; validity.md:1–7).

The checked predicate is:

```
pure def isValid(bft_store, pow_store, p) = all {
   pow_store.contains(p.block),
   p.bftContext == bft_store.last(),
   sigmaConfirmed(p.block, p.confirmation_tail, confirmation_depth),
   linear(p.block, p.bftContext, pow_store)
}
```

where `sigmaConfirmed(b, t, sigma)` requires a fork-free confirmation tail of size *at least* σ whose parents chain back to `b`, and `linear(b, context, work_store)` requires `work_store.isDescendant(context.pow_hash, b.hash)` *and* `not(context.pow_hash == b.hash)` (crosslink-spec @ `8edc3e9`, `validity.md`:131–138, 150–158, 171–176).

Machine-checked results. The invariant `prefix_inv` — "the PoW history defined by BFT finalization is a path in the PoW block store", formalized for all adjacent indices i as

```
pow_store.isDescendant(bft_store[i].pow_hash, bft_store[i+1].pow_hash)
```

— found no violation over 10000 random traces of 20 steps, and was model checked by TLC (via `quint compile --target tlaplus`) in about three minutes on the verification instance (`confirmation_depth = 1`, at most 8 blocks per store; the simulation instance uses `confirmation_depth = 2`, unbounded). The reachability witness `finalization_witness` (3 BFT blocks stored) was found by both the simulator and TLC, and the run `finalizationTest` passes under `quint test` (README.md:39–89; validity.md:376–461, 405–410, 463–482).

Divergences from the book. The model checks a rule set that neighbours, but does not match, the book's:

- Its `linear` predicate *forbids* keeping the same snapshot as the parent, via the conjunct `not(context.pow_hash == b.hash)`, whereas the book's Linearity rule allows equality and argues the allowance is necessary for Π_{bft} liveness (§3.4). The spec itself carries the comment "this actually seems allowed. TODO: perhaps remove" (`validity.md`:171–176 versus `construction.md`:573, 611–619).
- Its `sigmaConfirmed` accepts a tail of size $\geq \sigma$ where the book requires exactly σ headers; its proposal carries the snapshot block itself plus an *unordered* set of tail blocks, against the book's ordered deepest-first header sequence with $\mathsf{snapshot}(B) = B.\mathsf{headers\_bc}[0]\lceil_{\text{bc}}^{1}$; and its `isValid` requires `p.bftContext == bft_store.last()`, modelling the BFT side as a single forkless linear chain, unlike the book's set of bft-chains (`validity.md`:65–96, 131–158 versus `construction.md`:417–436).
- Not modelled at all: context_bft in bc-blocks, the Extension, Last Final Snapshot, and Finality depth rules, Stalled Mode, bft-last-final, voting and notarization proofs, and signatures (crosslink-spec @ `8edc3e9` README.md, validity.md).

Classification. The results above are *specified* in the strongest pre-spec sense available — machine-checked at a pinned commit — but of a *neighbouring* rule set: stricter than the book on snapshot equality, looser on tail size, excluding BFT forks, and silent on the whole Π_{bc} side. We therefore cite the Quint model as a proof-of-concept formalization, never as verification of the book's construction.

3.9 Divergences in the implementation track

The Shielded Labs/Eiger implementation (zebra-crosslink @ `6d02a1b`) carries a byte-for-byte copy of the book's construction chapter, prepended with three warning admonitions: the copy is "an almost direct paste" committing to a precise text for a security design review, not reviewed or endorsed by Daira-Emma Hopwood or Jack Grigg; the zebra-crosslink design diverges by *not* including the Stalled Mode rules, on the rationale that stakers redelegating prevents hazards such as BFT stalls; and it does not use the outer signature, trading a hazard many participants could leverage for one only the current proposer could, judged not worth it — with the recorded consequence that a direct hash or copy of the most recent BFT finality certificate is insufficient evidence that a specific bitwise copy is canonical (`book/src/design/cl2-construction.md`:3–17, diffed against `construction.md` at `fe6e1d6`).

At the April pinned commit, the further construction-level divergences are:

- *Fat pointer for* context_bft. The fork adds `fat_pointer_to_bft_block` to the PoW block header: a bundle of Ed25519 signed precommit votes — a 44-byte vote template whose first 32 bytes are the BLAKE3 hash of the target BFT block, plus 96-byte entries of public key and vote signature — constructed from the deciding round's signed precommit votes (tenderlink's

FatPointerToBftBlock3, converted at zebra-crosslink/src/lib.rs:1846–1860); the earlier Malachite commit-certificate conversion survives only in the undeclared dead-code module src/malctx.rs. It replaces both the book's plain context_bft commitment and the notarization-proof/outer-signature structure with a self-certifying pointer (zebra-chain/src/block/header.rs:109–133, 195–201; zebra-crosslink/src/lib.rs:1811–2052).

- *Immediate finality instead of* bft-last-final. The BFT engine is Tendermint-style: the in-house tenderlink package (git.sr.ht/~azmr/stuff rev 0e88509), driven directly from zebra-crosslink/src/lib.rs. Each tenderlink-decided BFT block is treated as immediately final: the handler (installed as tenderlink::ClosureToPushDecidedBlock, lib.rs:1234) sets the latest final block from the decided block's headers and issues zebra_state::Request::CrosslinkFinalizeBlock so the PoW state finalizes that ancestor — the implementation's fin extraction. With no BFT-final block known, the node falls back to the PoW block at MAX_BLOCK_REORG_HEIGHT depth (zebra-crosslink/Cargo.toml:74–79; zebra-crosslink/src/lib.rs:235–330, 499–601).

- *Validity rules mostly unimplemented.* The Rust BftBlock carries the fields version, height, previous_block_fat_ptr, finalization_candidate_height, and headers; its sole constructor try_from validates only headers.len() = σ and logs "not yet implemented: all the documented validations"; deserialization rejects more than 2048 headers. The vote-path validation checks only that the new block's previous fat pointer hashes to the current tip's, and that the node knows the PoW block referenced by headers.first(). The Linearity rule, Tail Confirmation validation, and the Confirmed best-chain criterion are *not* enforced on this path; the decided-block handler additionally verifies fat-pointer signatures, with a TODO to check the public keys against the roster (zebra-crosslink/src/chain.rs:61–151, 99–104; zebra-crosslink/src/lib.rs:522–541, 790–822).

- *Header-end inconsistency.* A source comment claims that in-memory order is "reversed from the specification", but the actual FindBlockHeaders path returns headers after the intersection in increasing height order, deepest first. The accessors disagree: finalization_candidate() returns headers.last() while the decided-block handler computes the newly final PoW hash from headers.first(); the consistency assertion against finalization_candidate_height is commented out, and header fetching carries the comment "TODO: do we want these in chain order or walk-back order". For a stable tip at T, the supplied headers span $T-\sigma+1$ through T: the accessor selects T, the handler selects $T-\sigma+1$, and the book's snapshot is their predecessor $T-\sigma$ (zebra-state/src/service/read/find.rs, collect_chain_headers; chain.rs:52–54, 124–126; lib.rs:543–546, 809, near :432).

Current delta. In crosslink_monolith v13, the main finalization path consistently takes the first header as the candidate; the April header-end inconsistency above is historical. The constructor still validates only the number of headers and still logs that the other documented validations are unimplemented, so this correction does not establish the Linearity, Tail Confirmation, or best-chain checks. The BFT block format now supports version 2 fields for hard-fork data and an earliest PoW inclusion height. These are *specified* prototype facts at 807b995, not a validation of the TFL-book construction (librustzcash/zcash_primitives/src/bft.rs; zebra-crosslink/zebra-crosslink/src/lib.rs).

Beyond the book's construction, the deployment track adds a mechanism-design layer — finalization rewards paid only in blocks that advance finality, a staking transaction format with the Crosslink Staking Orchard Restriction, staking epochs, a roster in ledger state, and the General Ledger State Design Invariant that all ledger state, including the PoS/BFT roster and finality status, is computable entirely from PoW blocks and transactions, with finality status the sole "height-eventual" state (book/src/design/nutshell.md:23–128).

Classification. The code facts above are *specified* in the pinned-commit sense. Milestone 5 ("Pre-Productionization") completed 2026-04-15, productionization is ongoing with no confirmed mainnet activation date, and the first independent security and mechanism-design review is complete (`https://forum.zcashcommunity.com/t/shielded-labs-crosslink-deployment-updates/49706`; `https://shieldedlabs.net/roadmap/`; checked 2026-09-05). The four-item list is pinned to historical commit `6d02a1b`; the current-delta paragraph is pinned to `807b995`. Neither is deployed construction.

4 Stake, delegation, and slashing

Crosslink's finality vote is stake-weighted, so the construction owes answers to the questions every proof-of-stake system faces: how ZEC becomes voting weight, how a wallet delegates without surrendering custody, and what punishes a finalizer that misbehaves. This section collects those answers — and their absences. The division of labour among the sources is unusual and governs every citation below: the ECC design book abstains from the whole topic, listing "PoS subprotocol selection" and "Issuance and supply mechanics, such as how much ZEC stakers may earn" among its "Major Missing Components" (tfl-book, `src/introduction/status-and-next-steps.md` at `fe6e1d6f`), so every concrete stake, delegation, and slashing decision lives with Shielded Labs: the zebra-crosslink design book (below, "the implementation book") and two dated posts[1].

No Crosslink or staking ZIP exists in the official repository: at `zcash/zips @ e753a6a3` (2026-09-03) the only Crosslink references are a citation footnote in ZIP 218 and a tfl-book reference in `draft-ecc-onchain-accountable-voting`; the "IO Finalizer" and "Spend Finalizer" of ZIP 374 are unrelated PCZT roles. The implementation book states "We will be refining a ZIP Draft [Google Doc link] which will enter the formal ZIP process as the prototype approaches maturity" (`book/src/design.md` at `6d02a1b8`). The machine-checked corpus is equally silent: the Informal Systems Quint model covers only the proposal-validity function (Tail Confirmation and Linearity rules) and models no stake, roster, voting power, delegation, bonding, or slashing (crosslink-spec @ `8edc3e94`, `validity.md`, `src/validity.qnt`). Everything below is therefore *designed but unspecified* at best, or implementation-defined prototype behaviour, or an *open problem*; we keep the bin visible throughout. The implementation book itself warns that its extensions "may violate some of the assumptions in the security proofs" of the Crosslink 2 construction and calls its staking chapter "a provisional sketch" that "will eventually be entirely redundant with... a canonical ZIP" (`book/src/design.md`, `book/src/design/nutshell.md` at `6d02a1b8`).

4.1 Stake in the abstract construction

The Crosslink 2 construction treats stake as an abstraction: "For each epoch, there is a fixed number of voting units distributed between the star-bft-nodes, which they use to vote for a star-bft-proposal" (tfl-book, `src/design/crosslink/construction.md` at `fe6e1d6f`; the implementation book mirrors the passage verbatim in `book/src/design/cl2-construction.md`). The notarization rule for a proposal P "must require at least a two-thirds absolute supermajority of voting units in P's epoch to have been cast for P", and may require more (ibid.). How ZEC-denominated stake maps onto voting units is delegated to the unselected PoS subprotocol — precisely the "Major Missing Component" above.

The adversary bound is stated over the same abstraction. Notation follows the book: Π_{bft} is the BFT subprotocol as adapted by Crosslink, Π_{origbft} its underlying engine; the book's $\star$bft is a wildcard ranging over both.

[1] Shielded Labs, "Crosslink: Updated Staking Design", `https://shieldedlabs.net/crosslink-updated-staking-design/` (forum post 2025-10-27, blog dated 2025-11-05); "Crosslink Early Tokenomics Design", `https://forum.zcashcommunity.com/t/crosslink-early-tokenomics-design/52154` (2025-09-12); deployment updates, `https://forum.zcashcommunity.com/t/shielded-labs-crosslink-deployment-updates/49706`. Last checked 2026-09-05.

Definition 4.1 (One-third bound on non-honest voting)**.** An execution of Π_{bft} has the *one-third bound on non-honest voting* property iff for every epoch, *strictly* fewer than one third of the total voting units for that epoch are ever cast non-honestly. A voting unit is cast non-honestly iff: it is cast other than by the holder of the unit (due to key compromise or any flaw in the voting protocol, for example); or it is double-cast; or the holder, following the conditions for honest voting in $\Pi_{\star\mathsf{bft}}$ according to its view, should not have cast that vote (tfl-book, `src/design/crosslink/construction.md` at `fe6e1d6f`, near-verbatim).

The quantifier "ever" is load-bearing. The book's own security caveat spells out the consequence: the bound counts ballots cast at *any* time in the execution, including votes cast with a compromised key for a long-finished epoch; "Therefore, validator keys of honest nodes must remain secret indefinitely. Whenever a key is rotated, the old key must be securely deleted", with further discussion deferred to tfl-book issue #140 (ibid.). We return to what this demands of finalizer key management in §4.7.

The one commitment the ECC book does make about delegation is a goal, not a mechanism: "The protocol should enable users of shielded mobile wallets to delegate ZEC to PoS consensus providers and earn a return on that ZEC coming via ZEC issuance or fees. Doing this may expose users to a risk of loss of delegated ZEC (such as through 'slashing fees'). The protocol must guarantee that PoS consensus providers have no discretionary control over such delegated funds (including that they cannot steal those funds)" (tfl-book, `src/design/goals.md` at `fe6e1d6f`). Note the tension recorded for §4.6: the goal contemplates "slashing fees" as a user risk; the 2025 implementation design forswears automatic slashing, while the current prototype adds socially selected bond burns.

Classification. *Designed but unspecified*: the voting-unit model and Definition 4.1 come from a design book with security arguments, not a specification. The mapping from stake to voting units — proportional weighting as in the prototype (§4.8) versus discretized units, and how the per-epoch roster snapshot is taken — is an *open problem*: no design document fixes it.

4.2 Positions, the roster, and the active set

The canonical staking statement is one sentence: "Staking in Crosslink takes place exclusively within the Orchard pool" (Shielded Labs, Updated Staking Design, 2025-10-27/11-05). Participants lock ZEC to delegate stake to a chosen finalizer and earn rewards; Penumbra-style fully shielded delegations were considered and deferred to future versions, prioritizing time-to-market (ibid.). The 2025-10/11 design predates the proposed NU6.3 pool split. Its words therefore mean the then-single Orchard-protocol shielded pool; they do not decide whether a future Crosslink design would bond value in the legacy Orchard pool or the proposed Ironwood pool. That migration question is open; neither pool's note machinery is altered here.

Definition 4.2 (Staking position, roster, stake weight, active set)**.** The *Roster* is a Ledger State table tracking staking positions created by stakers and attached to specific finalizers. Each *staking position* comprises at least: a target finalizer verification key, a staked ZEC amount, and an unstaking/redelegation verification key. The finalizer verification key "serves as the sole on-chain reference to a specific finalizer"; an entity may produce any number of finalizer keys. Unstaking/redelegation keys "are unique to the position itself (and not linked on-chain to a user's other potential staking positions or other activity)". The sum of outstanding staking positions designating a specific finalizer verification key at a given block height is the *stake weight* of that finalizer, and equals its BFT voting weight. At a given height the top K ("currently" $K = 100$) finalizers by stake weight are the *active set* (implementation book, `book/src/design/nutshell.md` at `6d02a1b8`).

The roster obeys the implementation book's General Ledger State Design Invariant: "All changes to Ledger State can be computed entirely from (PoW) blocks, transactions, and data transitively referred to by such. This includes all existing Ledger State in mainnet Zcash today, the PoS and BFT roster state (see below), and the finality status of blocks and transactions" (ibid.). Roster and active set are thus height-immediate — a function of the chain up to the height in question. Finality status is the sole height-eventual Ledger State: a property of height H computable only from a later attesting block; all nodes seeing the attesting block agree, and if the attesting block rolls back, the protocol guarantees an alternative block will eventually attest the same candidate (ibid.). Wallets today approximate the missing finality signal with ZIP-315 confirmation depths (*Wallet Guide*, §"Transaction construction"); Crosslink turns it into consensus state.

Classification. *Designed but unspecified* throughout. The active-set bound is explicitly provisional ("currently" 100), and the book's own TODO list flags "active set selection" as unresolved (`book/src/design/nutshell.md` at `6d02a1b8`). The April prototype applied no top-K limit; current `v13` limits the active Tenderlink roster to its first 100 members (`tenderlink/src/lib.rs` at `807b995`).

4.3 Staking actions and draft consensus rules

The implementation book extends the transaction format with an optional staking-action field carrying four abstract actions (`book/src/design/nutshell.md` at `6d02a1b8`):

- *stake* — creates a staking position assigning a transparent ZEC amount, which must be a power of 10, to a finalizer, and includes a signature verification key authorizing later redelegate/unstake actions;
- *redelegate* — identifies a position, a new finalizer, and a signature valid under the position's published verification key;
- *unbond* — moves a position into the *unbonding* state: redelegations become invalid and the amount stops contributing to staking weight for finalizers or the staker;
- *claim* — removes an unbonding-state position and contributes the ZEC to the chain value pool balance, which — by the Orchard restriction below — may only go to an Orchard destination and/or fees.

Value flows must balance per the Zcash protocol specification's chain value pool accounting (§4.17); the Orchard side of that balance is the value-commitment machinery of the *Ironwood Guide* (§"Conservation without disclosure: value commitments and the binding signature"). The draft consensus rules, verbatim from the implementation book (`book/src/design/nutshell.md` at `6d02a1b8`):

1. *Orchard restriction* (context-free validity check): "A transaction which contains any *staking actions* must not contain any other fields contributing to or withdrawing from the Chain Value Pool Balances *except* Orchard actions, explicit transaction fees, and/or explicit ZEC burning fields."
2. *Staking epochs*: "Once Crosslink activates at block height `H_crosslink_activation`, *staking epochs* begin, which are continguous [sic] spans of block heights with two phases: *staking day* and the *locked phase*." The staking-day length is a protocol constant of roughly 24 hours of blocks; the locked-phase length is a constant of roughly 6×24 hours, "so that *staking day* is approximately one day per week."

3. *Locked staking actions restriction*: "If a transaction includes any staking actions *except for* redelegate, and that transaction is in a block height in a *locked phase* of the *staking epoch*, then that block is invalid."

4. *Unbonding delay*: "If a transaction is [sic] block height H includes any *claim* staking actions which refer to *unbonding state* positions which have not existed in the unbonding state for at least one full staking epoch, that block is invalid." The source notes the implication that the same staking day cannot include both an *unbond* and a *claim* for one position.

5. *Amount quantization*: "If a transaction includes any staking actions with an amount which is not a power of 10 ZEC (or ZAT), that transaction is invalid." The book's TODO list clarifies the intent that the restriction apply only to the *stake* (create-position) action.

The quantization is a privacy device. The blog states it plainly: "Staking transactions must be made in exponential increments of 10 (e.g., 1, 10, 100, 1,000, 10,000, etc.)"; "This approach prevents observers from easily inferring individual staking patterns, preserving privacy while maintaining accountability"; and, on the other side of the trade, "For security and transparency, staked amounts are revealed so the community can verify the total ZEC staked and its distribution across finalizers", with "Transactions... processed in fixed 5-day epochs, grouping all commitments into a single batch" (Updated Staking Design, 2025-10-27/11-05). Another Zcash design also quantises shielded weight: Zally uses one ballot per 0.125 ZEC (`valargroup/voting-circuits` @ `4c39abd5`, `src/params.rs`:14).

Two gaps are visible already at this level. First, the *stake* action assigns "a transparent ZEC amount" while staking is exclusively Orchard-pool: how the revealed, quantized amount balances against Orchard value fields under §4.17 is unspecified. The April prototype sidestepped it entirely — a transaction carrying a staking action bypassed the `has_inputs_and_outputs` consensus check with a "TODO: real staking transactions with inputs/outputs" (`zebra-consensus/src/transaction/check.rs` at `6d02a1b8`). Current `v13` instead includes bond creation as a negative staking-pool balance and withdrawal as a positive one; the transaction has one optional staking-action field (`librustzcash/zcash_primitives/src/transaction/builder.rs` and `mod.rs` at `807b995`). This demonstrates a prototype accounting route but does not specify the eventual format. Second, the phrase "a power of 10 ZEC (or ZAT)" leaves the bucket range ambiguous: whether sub-1-ZEC buckets (10^k zatoshi) are permitted, and hence the minimum stake, is unspecified; the blog's examples start at 1 ZEC. How the eventual format pays fees and whether it permits several actions remain open in the design; `v13`'s prototype format permits one optional action (`book/src/design/nutshell.md` at `6d02a1b8`; `librustzcash/zcash_primitives/src/transaction/mod.rs` at `807b995`).

The delegation-safety goals for the first deployment are recorded as issues: "Delegating to a finalizer does not enable the finalizer to steal your funds" and delegation "does not leak information that links the user's action to other information about them, such as their IP address, their other ZEC holdings that they are choosing not to stake, or their previous or future transactions" (GH #124, implementation book, `book/src/design/scoping.md` at `6d02a1b8`). The explicit first-deployment trade-offs cut the other way: *not* "supporting a large number of finalizers so that any user who wants to run their own finalizer can do so", no support for casual users running finalizers, and no tokens representing staked ZEC that can be traded while the stake stays locked — the last excluded in both the tfl-book goals and the blog's V1 (ibid.; tfl-book, `src/design/goals.md` at `fe6e1d6f`).

Classification. *Designed but unspecified*: the actions and rules above are draft consensus text in a self-described provisional sketch. The transaction format (value balancing), fee payment, bucket range, and action multiplicity are *open problems* acknowledged in the sources.

4.4 Epochs and parameter drift

The staking-time parameters have changed at every publication and the sources now disagree. Table 2 pins each statement to its date.

Source, date	Epoch	Unbonding	Also fixed
Early Tokenomics Design (forum t/52154, 2025-09-12)	10-day staking / batching windows	30 days	top-100 roster; 40/40 issuance split; no protocol slashing; finalizer commissions expected
Updated Staking Design (forum 2025-10-27; blog 2025-11-05)	fixed 5-day epochs, one batch per epoch	1 epoch; funds available in 5–10 days	Orchard-pool exclusivity; power-of-10 amounts
Mechanism-design audit proposal (2025-12-11)	5-day epochs	5–10 days	40/40 split; "no protocol-level slashing in V1"
Implementation book (`6d02a1b8`, snapshot 2026-04-21)	staking day (∼1 day) + locked phase (∼6 days)	≥ 1 full staking epoch	redelegate exempt from the locked phase
Current prototype (`807b995`, `v13`, 2026-08-13)	150-block period; actions in first 70 blocks, with five temporary exception heights	≥ 75 blocks before begin-unbonding and again before withdrawal	retarget exempt; slash look-back 300 blocks

Table 2: Staking-parameter statements by source and date. Sources: forum t/52154 (2025-09-12); Shielded Labs blog (2025-10-27/11-05); `book/src/design/analysis/history/2025-12-11-mech-design/proposal.md` and `book/src/design/nutshell.md`, both at `6d02a1b8`; `librustzcash/zcash_primitives/src/transaction/mod.rs` and `zebra-crosslink/zebra-consensus/src/transaction.rs` at `807b995`. Web sources checked 2026-09-05.

The blog shortened the September windows "citing liquidity and accessibility"; the implementation book — which post-dates the blog — lengthens the effective cycle again to approximately one staking day per week. The current prototype fixes block-count values for its feature net, but no design source adopts them for production. The divergence is not cosmetic on one point: the book's locked-phase rule exempts *redelegate*, making redelegation valid at any height, while the blog has participants "reallocate their stake to another finalizer during these windows", implying batched redelegation. In the baseline design, redelegation is the staker's only protocol-mediated censure mechanism (§4.6), so its latency is security-relevant. The current prototype's socially selected burn is a separate hard-fork mechanism.

Classification. The first four rows are *designed but unspecified*; the `v13` row is a *specified* prototype fact. Production values remain an *open problem*. A ZIP must pin epoch length, staking-day length, and unbonding duration in blocks, and must fix redelegation latency explicitly.

4.5 Rewards and issuance

The implementation book routes staking rewards through the coinbase, gated on finality: consensus rules require the coinbase transaction to transfer the Crosslink-reward portion of new issuance *only* in blocks that advance finality. Per-block Crosslink rewards otherwise accumulate as *pending finalization rewards*, and a finality-updating block must distribute the entire pending amount to stakers and finalizers, computed against the active set as of the most recent final block prior to the update (`book/src/design/nutshell.md` at `6d02a1b8`). With $c := \texttt{COMMISSION\_FEE\_RATE} = 0.1$ a fixed protocol parameter, the slices of the pending amount S_{total} are

$$S_{\mathsf{finalizer}} = c \cdot \frac{W_{\mathsf{finalizer}}}{W_{\mathsf{total}}} \cdot S_{\mathsf{total}} \qquad \text{(active-set finalizers only)}, \qquad S_{\mathsf{staker}} = (1-c) \cdot \frac{W_{\mathsf{staker}}}{W_{\mathsf{total}}} \cdot S_{\mathsf{total}}, \quad (1)$$

where $W_{\mathsf{participant}}$ is that participant's total stake weight and $1-c = 0.9$ is called the staker reward rate. The staker weight W_{staker} counts all of a staker's positions regardless of whether they are delegated

to an active-set finalizer; the sum of all slices is S_{total} or less, "with the difference coming from stake assigned to finalizers outside the active set" (ibid.).

The issuance side is thinner. The Early Tokenomics post fixes "the block reward should be split equally (40/40) between miners and finalizers" (forum t/52154, 2025-09-12), and the mechanism-design audit proposal confirms the "40/40 issuance split" as an audit assumption (`book/src/design/analysis/history/2025-12-11-mech-design/proposal.md` at `6d02a1b8`). The destination of the remaining 20% is not stated in any source we located. How Crosslink rewards interact with the issuance schedule, halvings, and the 21M cap is exactly the "Issuance and supply mechanics" component the tfl-book lists as missing (tfl-book, `src/introduction/status-and-next-steps.md` at `fe6e1d6f`). The commission model is also unsettled: the book fixes $c = 0.1$ for all active-set finalizers, but the September post said "We also expect finalizers to charge commissions", suggesting per-finalizer market rates (forum t/52154, 2025-09-12).

One UX goal collides with the quantization rule. Goal GH #158 asks for compounding returns without user or wallet action (implementation book, `book/src/design/scoping.md` at `6d02a1b8`), but a reward paid to an Orchard destination cannot silently increase a power-of-10 position. No source resolves the collision; the book's own TODO list flags "rewards distribution via coinbase" and "quantization of amounts / time" as unresolved (`book/src/design/nutshell.md` at `6d02a1b8`).

Classification. *Designed but unspecified*: Equation (1) and the pending-rewards rule are draft consensus text. Issuance interaction, the 20% remainder, the commission model, reward eligibility across the unbond–claim lifecycle, and the compounding mechanism are *open problems*. The April prototype implemented none of this; current `v13` implements a different fixed, directly compounding prototype reward (§4.8).

4.6 Automatic and user-activated slashing

The design commitment is verbatim and unambiguous: "There will be no automatic, protocol-level slashing in this design, so the power lies fully in the stakers' hands to reward and censure node operators" (forum t/52154, 2025-09-12); the audit proposal restates the constraint as "no protocol-level slashing in V1" (`book/src/design/analysis/history/2025-12-11-mech-design/proposal.md` at `6d02a1b8`).

This prohibition describes automatic, evidence-triggered slashing in the baseline design. Current prototype `v13` implements a different mechanism: users configure a hard-fork rule with a PoW activation height, a BFT certificate height, and a list of terminated finalizer keys. At activation, affected finalizers leave the active roster and tracked bonds that were actively delegated to them during the preceding 300-block analysis window are marked burned. A delegation interval opened and closed within one block is dropped from this index; marking a bond burned does not claw back an already completed withdrawal. One such rule ships in the feature-net executable. Selection of the keys is social and out of band; the code does not infer guilt from an on-chain equivocation proof. This is a *specified* prototype fact, not a normative or automatic slashing rule (crosslink_monolith @ `807b995`, `librustzcash/zcash_primitives/src/bft.rs`, `zebra-crosslink/zebra-chain/src/parameters/hardfork.rs`, and `zebra-crosslink/zebra-state/src/service/finalized_state/zebra_db/slashing.rs`; the burn operation itself is in `zebra-crosslink/zebra-state/src/service.rs`).

The ECC book never specified slashing either; it appears only as discussion. A "Potential changes" passage observes that "In most PoS protocols, the requirement to have a minimum amount of stake in order to make a proposal acts as a gatekeeping filter on proposals, and potentially allows parties that make invalid proposals to be slashed", that stake-holding validators "can be slashed", and that "there is no technical reason why the ability to make a bft-proposal has to be gatekept by a stake requirement — given the situation of Zcash in which we already have a mining infrastructure" (tfl-book, `src/design/crosslink/potential-changes.md` at `fe6e1d6f`). The closest the book comes

to accountability machinery is a sketch in its questions file: a *bft-final-vee* — two final bft-blocks with the same parent — as objective rollback evidence; the observation that more than 1/3 of stake voting for conflicting blocks in one epoch proves the adversary-stake assumption was violated; and the note that such evidence "can be posted in a transaction to the bc-chain", optionally latching Stalled Mode until manual intervention (tfl-book, `src/design/crosslink/questions.md` at `fe6e1d6f`). None of this is adopted as a rule.

What replaces automatic slashing in the baseline design is a chain of accountability between roles, stated in the implementation book's Five Component Model (`book/src/design/five-component-model.md` at `6d02a1b8`). Stakers police finalizers: it is "the job of the stakers to hold the finalisers accountable by detecting and punishing misbehavior... by redelegating stake away from ill-behaved finalizers to well-behaved finalizers". Users police stakers, ultimately via "an emergency user-led hard fork (which is what happened in the STEEM/HIVE hard fork)". The same file bounds finalizer power: finalizers "cannot unilaterally compromise liveness or censorship-resistance (even if an attacker controls $> 2/3$ of the stake-weighted votes)"; "A plurality of the finalisers (controlling $> 1/3$ of the stake-weighted votes) could execute stall attacks"; and finalizers "cannot unilaterally execute rollback attacks even though they can prevent rollback attacks".

The chain has a load-bearing dependency on censorship-resistance of block production. A collusion controlling both $> 1/2$ hashpower and $> 2/3$ stake could violate finality and execute rollback and unavailability attacks; and "Censorship-resistance is necessary for stakers to stake, unstake, and redelegate, which means censorship-resistance is necessary for stakers to be able to perform their duty of holding the finalizers accountable. Therefore, if an attacker controls $> 1/2$ of the hashpower (and is thereby able to censor), then the attacker can prevent the stakers from holding the finalizers accountable" (ibid.; the file ends mid-list on the "$> 2/3$ stake and $< 1/2$ hashpower" scenario — the document is incomplete). The reliance on stakers is so central that the April zebra-crosslink design drops the tfl-book's Stalled Mode entirely: "This simplification to the protocol rules follows the rationale that we are already relying heavily on the stakers to guard safe operation of the protocol by redelegating appropriately to prevent hazards like BFT stalls" (`book/src/design/cl2-construction.md` at `6d02a1b8`).

Shielded Labs frames the substitution quantitatively as Penalty-for-Failure-to-Defend (PFD): penalties are assessed per property ($\text{PFD}_{\text{liveness}}$ versus $\text{PFD}_{\text{finality}}$); Tendermint-style slashing yields a "financial value gauge" PFD, while PoW — and the baseline design's automatic-slashing-free PoS — yields a *rate* PFD, the loss of future revenue rate; the two types "cannot be directly compared" (`book/src/design/analysis/pffd.md` at `6d02a1b8`, a paste of GH issue #264). Redelegation-based punishment is thus a rate penalty standing in for the gauge penalty of slashing. For redelegation to work, stakers must be able to observe misbehaviour: Crosslink finality "is accountable because the chain contains explicit voting records which indicate each participating finalizer's on-chain behavior" (`book/src/design/terminology.md` at `6d02a1b8`), and goal GH #214 asks that "Skilled users can easily collect records of finalizer behavior and performance" (`book/src/design/scoping.md` at `6d02a1b8`) — accountability is informational, feeding redelegation, not punitive. The terminology echoes the accountability line of the Ebb-and-Flow literature: the construction cites Neu–Tas–Tse [NTT2020, arXiv 2009.04987] as background (tfl-book, `src/design/crosslink/construction.md` at `fe6e1d6f`), and the same authors' accountability-gadgets paper (arXiv 2105.06075, Financial Cryptography 2022) is the natural successor for this topic; NTT2020 remains at v3 (2021-02-04) as of 2026-09-05.

The backstop of the chain — the user-led emergency hard fork — has a drafted scope. The governance proposal limits legitimate emergency intervention to layer-1 security failures, explicitly including "situations in which would-be miners, finalizers, or stakers are being systematically censored or prevented from participating in block production, finalization, or staking"; it excludes "staking or unbonding parameters" changes from emergency scope; and it states that "Control over a large share of hashpower, finalization power, or stake does not itself constitute a violation" (`book/src/design/userled-emergency-hardforks.md` at `6d02a1b8`).

Classification. The absence of automatic slashing is *designed but unspecified*; the user-activated burn is a *specified* `v13` prototype fact. The accountability chain and PFD framing are design rationale. The tfl-book's evidence sketches (bft-final-vee and conflicting-vote proofs) remain an *open problem*: whether a future specification adds evidence-based slashing, how socially selected burns are authorized, and what accountability records a ZIP requires are unresolved. The tfl-book's own delegation goal budgets for "slashing fees" (§4.1).

4.7 Finalizer keys

Definition 4.2 makes the finalizer verification key the sole on-chain reference to a finalizer, and makes position keys deliberately unlinkable to a user's other activity. Everything else about finalizer key management is prototype fact or gap. The implementation uses single ed25519 keys per finalizer (ed25519-zebra; "ed25519 public key of the finalizer"), and votes are plain ed25519 signatures: the vote wire format is 76 bytes — a 32-byte ed25519 finalizer public key, a 32-byte BLAKE3 value hash (all-zero encoding Nil), an 8-byte height, and a 4-byte round whose most significant bit flags a commit — with a signed vote appending a 64-byte ed25519 signature over those 76 bytes (`zebra-crosslink/src/lib.rs`, `MalVote`, lines 1811–2052, and `zebra-chain/src/block/header.rs`, lines 109–133, at `6d02a1b8`). No FROST or threshold-signing design for finalizer keys is sourced anywhere in the Crosslink corpus: a search for "frost" over tfl-book, crosslink-protocol-guide, crosslink-spec, zebra-crosslink, and the current monolith returns only upstream Zebra RedPallas/Orchard files unrelated to finalizers (negative grep, 2026-09-05). Threshold custody is established practice elsewhere in the Zcash stack — FROST per RFC 9591 in the PCZT signing flow (*Wallet Guide*, §"Partially created transactions (PCZT)") — which makes its absence here a visible gap rather than an exotic ask.

The gap matters because of Definition 4.1's caveat: honest validator keys must remain secret *indefinitely*, with secure deletion on rotation (tfl-book, `src/design/crosslink/construction.md` at `fe6e1d6f`, issue #140) — yet no key-rotation or revocation mechanism for finalizer verification keys in the roster is specified in any source. Current `v13` serializes an `UpdateFinalizerKey` action kind, but its state code explicitly says that action is not implemented; an enum tag is not a rotation mechanism (`librustzcash/zcash_primitives/src/transaction/mod.rs`; `zebra-crosslink/zebra-state/src/service/finalized_state/zebra_db/slashing.rs` at `807b995`). Bootstrap is equally unspecified: no source examined (tfl-book, implementation book, either blog post) defines eligibility or formation of the initial finalizer set. The April prototype bootstraps ad hoc: the roster starts empty; keys are deterministically derived from the config field `insecure_user_name` ("temp seed for private/public key pair"); test instrumentation can force-include roster members (`TFInstr::ROSTER_FORCE_INCLUDE`); and a node adds itself with `voting_power` 0 when absent from the roster it hands to the BFT engine (`zebra-crosslink/src/lib.rs`, `zebra-crosslink/src/test_format.rs` at `6d02a1b8`).

Classification. The ed25519 key and vote formats are implementation-defined prototype facts. The unimplemented `v13` tag changes no bin: key rotation, revocation, threshold custody, and initial-roster formation (including any minimum self-stake, and eligibility beyond top-K by stake weight) are *open problems* in every design source.

4.8 Prototype snapshots

April snapshot. The zebra-crosslink prototype (@ `6d02a1b8`, 2026-04-21) implements a reduced, implementation-defined variant of the above; we record it as code fact about a prototype, not as design. The placeholder activation height is `TFL_ACTIVATION_HEIGHT = BlockHeight(2000)` (`zebra-crosslink/src/lib.rs`).

Roster mechanics. Staking state changes take effect only when their PoW block is finalized: function `update_roster_for_block()` runs while walking each newly finalized block and ap-

plies StakingAction commands from Transaction::VCrosslink transactions. The variants of StakingActionKind are Add, Sub, Clear, Move, and MoveClear — names diverging from the book's stake/redelegate/unbond/claim — with fields including insecure_target_name and insecure_source_name, defined in the Shielded Labs librustzcash fork (git rev 33dc74bf; not present upstream) (zebra-crosslink/src/lib.rs, zebra-chain/src/transaction.rs, Cargo.toml at 6d02a1b8). The BFT engine is Tenderlink — Shielded Labs "moved away from Malachite and now maintain our own lightweight version of Tendermint, called Tenderlink" (Updated Staking Design, 2025-10-27/11-05; dependency tenderlink 0.1.0, rev 0e885095) — and the roster handed to it is sorted by descending (stake, public key) — "Needs to uniquely & stably tie-break members with the same stake" — with cumulative stake tracked "to make weighted round-robin easy" for proposer selection (zebra-crosslink/src/lib.rs, Cargo.toml at 6d02a1b8).

Rewards. The prototype ignores the coinbase design entirely: a constant R = pos_total_reward = 6000 zats ("arbitrary scale") is apportioned per newly finalized PoW block as $\text{reward}_i = \lfloor v_i \cdot R / \sum_j v_j \rfloor$ over finalizer voting powers v_i, with the integer-division dust going to the largest-stake finalizer, and the result added directly to voting_power — auto-compounding into stake weight rather than paid via coinbase. The code acknowledges its artifacts: "the compounding is per PoW block" and "finalizers added in the same PoS will get rewards for PoW blocks they couldn't have actually voted on" (zebra-crosslink/src/lib.rs at 6d02a1b8).

Gaps against the draft rules. (1) No power-of-10 quantization check, no staking-epoch/staking-day/locked-phase gating, and no unbonding-delay enforcement exist anywhere in the staking path. (2) No top-K = 100 active-set truncation: every roster member with non-zero power participates. (3) Staking transactions are stubs: a VCrosslink transaction carrying a staking action bypasses the has_inputs_and_outputs check, so no value balancing against the chain value pools occurs. (4) The bonding lifecycle (unbond/claim) is not implemented; Sub/Clear remove weight immediately (zebra-consensus/src/transaction/check.rs, zebra-crosslink/src/lib.rs at 6d02a1b8; negative grep for quantiz/unbond/epoch, rechecked 2026-09-05).

Current v13 delta. The active prototype at 807b995 has activation height 0 and replaces the stub actions with an optional transaction field containing create-bond, begin-unbonding, withdraw, retarget, register-finalizer, convert-finalizer-reward, or update-finalizer-key. Only the first four have a bond state machine; the last three are serialized placeholders. Create and withdrawal participate in transaction and chain value balance. Begin-unbonding requires 75 blocks since creation, and withdrawal requires 75 blocks since begin-unbonding; retargeting does not reset that clock. Except at five hard-coded feature-net heights, every action other than retarget must fall in the first 70 blocks of a 150-block period; retarget is exempt from both that window and the delay. Tenderlink limits the active roster to its first 100 members.

The reward mechanism remains deliberately prototypical but is no longer the April mechanism: each accepted PoW block for which at least one bond is active adds a fixed 500,000,000-zatoshi amount proportionally to all active bonds, gives rounding remainder to the largest bond, and increases the staking-bonded value pool. Rewards therefore compound directly; no finalizer commission or finality-triggered coinbase payout is implemented. Reorg and finalized state paths record and reverse or commit these bond increments. The user-activated burn mechanism is described in §4.6 (crosslink_monolith @ 807b995; librustzcash/zcash_primitives/src/transaction/mod.rs; zebra-crosslink/zebra-consensus/src/transaction.rs; zebra-crosslink/zebra-state/src/service.rs; zebra-crosslink/zebra-state/src/service/non_finalized_state/chain.rs; tenderlink/src/lib.rs).

Status. Crosslink remains in feature-net validation: the Incentivized Crosslink Feature Net launched with Milestone 5 (completed 2026-04-15; Season 1 from 2026-04-15; 500 ZEC allocated overall, 25 ZEC for Season 1; "Only cTAZ earned via block rewards counts"), following Milestone 4's independent security review (concluded 2026-02-10). By 2026-09-03 the feature net had tested a user-activated recovery from BFT stalls; no Mainnet activation date is stated (feature-net thread, posts 72 and 86, checked 2026-09-05).

Classification. Implementation-defined throughout, by the projects' own descriptions. The April book warns that the in-house BFT protocol's "own security properties need to be more thoroughly verified" (`book/src/design.md` at `6d02a1b8`); the current monolith README says it is not production-ready and not the eventual upstream proposal. Neither snapshot constrains a future ZIP.

4.9 What a specification must pin down

We close with the open problems, each traceable to a gap documented above.

1. Epoch parameterization: epoch length, staking-day length, and unbonding duration, in blocks (Table 2).
2. Redelegation latency: locked-phase exemption versus batched windows — the timing of the baseline design's staker-censure action (§4.4).
3. The staking transaction format: how a revealed, quantized, "transparent" amount balances against Orchard value fields under protocol specification §4.17 (§4.3).
4. Quantization bucket range and minimum stake: "power of 10 ZEC (or ZAT)" (§4.3).
5. Commission: protocol-fixed $c = 0.1$ versus per-finalizer market rates (§4.5).
6. Issuance: the 40/40 split's remaining 20%, pending-reward interaction with halvings and the 21M cap (§4.5).
7. Reward mechanics: coinbase distribution versus compounding, and the GH #158 goal against power-of-10 positions (§4.5); reward eligibility across the unbond–claim lifecycle and for stake on non-active finalizers.
8. Fees and multiplicity of staking actions per transaction (§4.3).
9. Voting-unit mapping and the per-epoch roster snapshot (§4.1).
10. Initial-roster formation, minimum self-stake, and finalizer eligibility beyond top-K (§4.7).
11. Finalizer key rotation and revocation under the indefinite-secrecy requirement (§4.7).
12. Accountability records, and whether any future version adopts evidence-based slashing (§4.6).
13. A formal model: nothing in the machine-checked corpus covers the roster, delegation, epochs, unbonding, or reward arithmetic (crosslink-spec @ `8edc3e94` models proposal validity only).

5 The wallet-visible contract

Crosslink changes what a Zcash node presents to everything above it. Today a wallet, an exchange, or the voting stack consumes one object — the node's best chain — and layers its own confirmation policy on top (*Wallet Guide*, §"The proposal"). Under Crosslink 2 the node maintains two chains and a per-block finality status, and every consumer must decide which chain it reads and what it does when the two stop tracking each other. This section fixes that contract as the sources state it: the two ledger views and the theorems relating them (§5.1), finality as a queryable status (§5.2), the interface the prototype implements today (§5.3), and the four wallet-facing decisions no source resolves — anchor policy (§5.4), confirmation policy (§5.5), stall behaviour (§5.6), and the service-facing protocol (§5.7).

Two texts carry the design (pinned, with the rest of the corpus, in Table 1). The ECC Trailing Finality Layer book (`daira/tfl-book` @ `fe6e1d6f`, 2024-12-13; hereafter the *tfl-book*) defines the Crosslink 2 construction and has not changed since; its own status page calls the design "an early and incomplete protocol design proposal" (`src/introduction/status-and-next-steps.md`).

The Shielded Labs design book inside the implementation repository (`zebra-crosslink` @ `6d02a1b8`, 2026-04-21; hereafter the *SL book*) pastes the construction verbatim, declares divergences in admonitions, and self-describes as a provisional precursor to "a canonical Zcash Improvement Proposal submission" (`book/src/design/nutshell.md`). No such ZIP exists: a repository-wide search of `zcash/zips` @ `e753a6a3` (2026-09-03) finds no Crosslink deployment artifact and no ZIP specifying NU8. Shielded Labs' productionization phase runs Q2 2026–Q2 2027 with no announced mainnet activation (`https://shieldedlabs.net/roadmap/` and `https://forum.zcashcommunity.com/t/shielded-labs-crosslink-deployment-updates/49706`, checked 2026-09-05). No design claim in this section is therefore *specified*; pinned prototype behaviour is *specified* only as code, and the open problems are named as such.

5.1 Two ledger views

We write $\preceq_{\mathrm{bc}}$ for the prefix order on bc-valid chains, $B \sim_{\mathrm{bc}} C$ when one of B, C is a prefix of the other, and $C^{\lceil k}$ for chain C with its last k blocks removed. The bc-best-chain of node i at time t is ch_i^t, notation the tfl-book adopts from the ebb-and-flow model of Neu, Tas, and Tse (arXiv 2009.04987v3, 2021-02-04, still the latest revision as of 2026-09-05; tfl-book `construction.md`:5).

Definition 5.1 (Ledger views of Crosslink 2)**.** Fix the bc-confirmation-depth $\sigma \in \mathbb{N}^+$. At every time t, each node i derives two node-local chains from its bc-best-chain and its BFT context:

(i) the *finalized chain* $\mathsf{localfin}_i^t$: a bc-valid chain obtained at the fixed confirmation depth σ via the finalization update rule (Construction 3.17, applied to the candidate of Definition 3.11);

(ii) the *bounded-available chain*, for a per-node depth μ with $0 < \mu \leq \sigma$:

$$(\mathsf{localba}_\mu)_i^t = \begin{cases} \mathsf{ch}_i^{t\lceil\mu} & \text{if } \mathsf{localfin}_i^t \preceq_{\mathrm{bc}} \mathsf{ch}_i^{t\lceil\mu}, \\ \mathsf{localfin}_i^t & \text{otherwise.} \end{cases}$$

(tfl-book `src/design/crosslink/construction.md`:391–399, 520–527.)

Both views are plain block chains. Snap-and-Chat and Crosslink 1 instead exposed *sanitized transaction logs* — flattened sequences with invalid transactions filtered out — and the tfl-book drops that machinery "because we do not need sanitization, there is no need to express it as a log of transactions rather than blocks" (`construction.md`:393); §5.4 returns to why this matters to wallets. The otherwise-case of the definition covers post-reorg difficulty situations in which a new best chain outscores the old one in fewer than σ blocks, and it makes the following theorem trivial (`construction.md`:547–555).

Theorem 5.2 (Ledger prefix property)**.** *For any node i that is honest at time t, and any confirmation depth μ:* $\mathsf{localfin}_i^t \preceq_{\mathrm{bc}} (\mathsf{localba}_\mu)_i^t$.

Proof. By construction of $(\mathsf{localba}_\mu)_i^t$: the otherwise-branch returns $\mathsf{localfin}_i^t$ itself (tfl-book `construction.md`:531–536). □

The depth μ is the node's own choice; the default "should be" $\mu = \sigma$, and choosing $\mu < \sigma$ "is at the node's own risk and may increase the risk of rollback attacks against $(\mathsf{localba}_\mu)_i^t$ (it does not affect $\mathsf{localfin}_i^t$)" (`construction.md`:395–399, Security caveat admonition).

Monotonicity and hazards. The finalized view never moves backward: the update rule advances $\mathsf{localfin}_i$ to the new candidate only when that candidate descends from the current value, which yields

Local finalization linearity — the time series $\mathsf{localfin}_i^{r \le t}$ is bc-linear. If the candidate instead conflicts, the node keeps its old value and "should record a finalization safety hazard", which "can only happen if global safety assumptions are violated"; the record should include ch_i^t and the history of localfin updates (`construction.md`:462–488). Single-node Assured Finality means compatible observations; monotonicity additionally follows from the update rule, not from compatibility alone (the source's equivalence claim at line 508 is too strong). Local monotonicity does not establish agreement across nodes.

The syncing gate. Neither view is exposed to a client before the node has synced: "this chain state should not be exposed to clients of the node until it has synced", where synced means a baked-in checkpoint bft-block $B_{\text{checkpoint}}$ satisfies $B_{\text{checkpoint}} \preceq_{\text{bft}} \mathsf{LF}(\mathsf{ch}_i^t)$, $\mathsf{snapshot}(B_{\text{checkpoint}}) \preceq_{\text{bc}} \mathsf{localfin}_i^t$, and the timestamp of $\mathsf{localfin}_i^t$ is within a threshold of the current time (`construction.md`:464, 529, 557–562).

Latency, and which view an application reads. The old analysis estimates a finalized view about $\mu+1+\sigma$ bc-blocks behind the tip (tfl-book `security-analysis.md`:265–269), but that passage lies below the "Not updated for Crosslink 2" marker and uses the earlier $\mathsf{LOG}_{\text{fin},\mu}$ construction. Crosslink 2's fin does not depend on the per-node μ; the old formula is not a validated latency bound for it. The book states the application-facing choice explicitly: since localfin lags $\mathsf{localba}_\sigma$ by only a few blocks outside Stalled Mode, "the main factor influencing the choice of a given application to use $\mathsf{localfin}_i^t$ or $(\mathsf{localba}_\mu)_i^t$ is not the average latency, but rather the desired behaviour in the case of a finalization stall: i.e. stall immediately, or keep processing user transactions until L blocks have passed" (`construction.md`:415). Stalls are §5.6.

Parameters. Production values of σ and of the finalization gap bound L are unchosen. The tfl-book requires L "significantly greater than σ" and "at least 2σ" in practice (`construction.md`:391, 405); both prototype snapshots ship $\{\sigma = 3,\ L = 7\}$, and the April source warns "No verification has been done on the security or performance of these parameters" (`zebra-crosslink/src/chain.rs`:206–222; crosslink_monolith @ `807b995`, `librustzcash/zcash_primitives/src/bft.rs`:388–404). Wall-clock statements are additionally provisional: block target spacing is 75 s today, and ZIP 218 (Status: Draft) proposes 25 s at NU7, describing itself as "complementary to ... finality mechanisms such as Crosslink" (`zips/zip-0218.md`:3, 76–78).

Classification. *Designed but unspecified* throughout: Definition 5.1 and Theorem 5.2 come from a design book at pre-ZIP rigor, and the constants are prototype code, not consensus.

5.2 Finality as a status

The tfl-book's term of art is *assured finality*: a protocol property that assures transactions cannot be reverted *by that protocol*. It eschews "absolute finality" because "it is not feasible for any protocol to prevent reversing final transactions 'out of band' "; "final" in the book always means assured finality, in contrast to proof-of-work's probabilistic finality (tfl-book `src/terminology.md`:11–13, 19), whose exact arithmetic — and its divergence as the adversary nears half the work — is the *Consensus Guide*'s subject (§"What the numbers say").

> **Definition 5.3** (Assured Finality)**.** An execution of Crosslink 2 has *Assured Finality* iff for all times t, u and all nodes i, j (potentially the same) such that i is honest at time t and j is honest at time u: $\mathsf{localfin}_i^t \sim_{\text{bc}} \mathsf{localfin}_j^u$ (tfl-book `construction.md`:501–506).

Theorem 5.4 (Two-sided safety). *An execution of Crosslink 2 has Assured Finality if* either *of the following holds:*

(i) the Π_{bc} *subprotocol has Prefix Agreement at confirmation depth* σ *— honest best-chains, truncated by* σ*, are pairwise compatible across all nodes and times; or*

(ii) the Π_{bft} *subprotocol has Final Agreement — the last-final blocks of any two bft-valid blocks in honest view are pairwise compatible.*

(tfl-book `security-analysis.md`*:72–77, 86–91; hypothesis definitions at* `construction.md`*:337–340, 203–206.)*

Assured Finality therefore survives the failure of either subprotocol's safety assumption alone. A safety failure therefore requires both premises to fail, although failure of both does not by itself imply an attack succeeds. This is the conditional guarantee a wallet's "final" badge would rest on. We defer both arguments to the security analysis (§6.1); branch (i) runs through the Local fin-depth and Linear prefix lemmas, while branch (ii) runs through Final Agreement and the Linearity rule. Prefix Agreement and Final Agreement are stated in full as Definitions 3.6 and 3.9.

One caveat bounds what is actually proved. Below the marker "$TODO$ Not updated for Crosslink 2" the tfl-book's safety chapter still argues in the Snap-and-Chat sanitized-log formalism (LOG_{fin}, LOG_{ba}, san-ctx), and the LOG_{ba} agreement proof trails off unfinished ("What we want to show is that, under some conditions on executions, ...") (`security-analysis.md`:54, 137–160, 253–257); the SL book copies this state verbatim. The bin split for this section: Theorem 5.4 is completely stated. Branch (i) carries the book's correct proof; branch (ii)'s published proof text is defective, but Remark 6.3 records a complete argument; transaction-log-level safety for the bounded-available view is *designed but unspecified* with an incomplete proof.

The Shielded Labs definition. The SL book defines "crosslink finality" by five properties: *accountable* (on-chain voting records), *irreversible within protocol scope* (undoing requires software modification or human intervention), *objectively verifiable* (determinable from local block history alone), *globally consistent* (all sufficiently-synced nodes agree on finality status), and an *asymmetric cost-of-attack defense* that is a literal TODO in the source (`book/src/design/terminology.md`, "Crosslink Finality"). Its ledger-state framing makes finality the sole "height-eventual Ledger State": a property of height H computable only from a later attesting block, on which all observers of that attesting block agree "even though they may observe the attesting block rollback. In the event of such a rollback, the protocol guarantees that an alternative block will eventually attest to the finality status of the same candidate block"; the General Ledger State Design Invariant requires all ledger state, finality status and the proof-of-stake roster included, to be computable entirely from PoW blocks and transactions (`book/src/design/nutshell.md`:106–128). For light clients the SL book adds a negative constraint: its design drops the tfl-book's "outer signature", so "a direct hash or copy of the most recent BFT finality certificate is insufficient evidence" that a given bitwise copy is or will become canonical (`book/src/design/cl2-construction.md`:3–16) — any wire-format finality proof must be designed around that.

Classification. Definition 5.3 and Theorem 5.4 are *designed but unspecified* (complete statements and proofs, pre-ZIP rigor); the SL five-property definition is *designed but unspecified* with its fifth property an acknowledged gap; ledger-level localba safety is an *open problem* in the sources' own accounting.

5.3 The prototype node interface

Current prototype `v13` (crosslink_monolith @ `807b995`) exposes finality through a typed internal service and a JSON-RPC surface. The internal state service defines

```
enum TFLBlockFinality { NotYetFinalized, Finalized, CantBeFinalized }
```

with request variants including

`IsTFLActivated`, `FinalBlockHeightHash`, `FinalBlockRx`, `SetFinalBlockHash`, `BlockFinalityStatus(height, hash)`, `TxFinalityStatus(txid)`, `Roster`, `FatPointerToBFTChainTip`, `StakingCmd`, `FinalizersRecencyStatus`, `WalletStakingAction`, `WalletStakingPositions`

(`zebra-crosslink/zebra-state/src/crosslink.rs`; variant `FinalBlockRx` is a broadcast receiver). The RPC methods, ten declarations still doc-commented as "Placeholder", are:

- `is_tfl_activated`;
- `get_tfl_roster_zec` and `get_tfl_roster_zats`;
- `get_tfl_fat_pointer_to_bft_chain_tip`;
- `staking_command`;
- `get_tfl_final_block_hash` and `get_tfl_final_block_height_and_hash`;
- `get_tfl_block_finality_from_hash` and `get_tfl_tx_finality_from_hash`;
- `set_tfl_finality_by_hash`;
- subscribe, stream, and blocking-notification methods for final blocks and transactions;
- `get_tfl_recency_status`;
- wallet UFVK, staking-action, and staking-position methods.

The docs concede that the final-block RPCs "for the sake of testing ... currently treat pre-reorg block as final".

Block-level finality is computed as a tri-state: a height above the final height returns `NotYetFinalized` ("N.B. this may be invalidated by the time it is received"). At the final height, the queried hash is compared with the stored final hash; below it, with the best-chain hash at that height. Equality returns `Finalized`, and inequality `CantBeFinalized` (`zebra-crosslink/zebra-crosslink/src/lib.rs`:1611–1655). This implements a local status lookup, not a proof of the SL book's global-consistency claim; best-chain membership and stored finality must themselves remain coherent for that interpretation to hold.

Two limitations prevent this RPC surface from realizing the SL book's full finality contract:

(i) Transaction finality cannot express `CantBeFinalized`. Request `TxFinalityStatus` returns `Finalized` iff a transaction found in the best chain has `tx.height` $\leq$ `final_height`, returns `NotYetFinalized` for a best-chain transaction above that height, and returns `None` when the transaction lookup fails; a literal `// TODO: CantBeFinalized` remains. The underlying transaction query returns `None` off the best chain, so a losing-fork transaction is not falsely finalized, but it is indistinguishable from an unknown transaction (`zebra-crosslink/zebra-crosslink/src/lib.rs`:1806–1828; `zebra-crosslink/zebra-state/src/request.rs`). The separate blocking block-notification method built on `FinalBlockRx` retries `NotYetFinalized`, but returns immediately on `None` or either terminal status (`zebra-crosslink/zebra-rpc/src/methods.rs`:2182–2218).

(ii) Method `set_tfl_finality_by_hash` is intended to be regtest-only but currently ships `regtest_override = true` (`zebra-crosslink/zebra-rpc/src/methods.rs`:2094–2127).

Both are prototype-status limitations rather than design statements.

Fallback depth. Whenever no BFT-final block is known, the implementation falls back to the block-locator's last hash as its final block: function `tfl_reorg_final_block_height_hash` returns that last locator hash, which functions as the pre-reorg final point once the chain is deep enough. Current `v13` pins `MAX_BLOCK_REORG_HEIGHT` at 99; an internal `latest_final_block`, once set by BFT, takes precedence (`zebra-crosslink/zebra-crosslink/src/lib.rs`:458–555; `zebra-crosslink/zebra-state/src/constants.rs`; `librustzcash/components/zcash_protocol/src/consensus.rs` at `807b995`). Upstream Zebra meanwhile moved the same constant to 1000 (commit `02f9648a5`, 2026-06-15; relocated to `zebra-chain` by `8767a91ba`), documenting it as "a local-only node policy; it is not part of consensus" sized "as a defence-in-depth measure against sustained consensus splits" (`zebra` @ `d1fd7adf`, `zebra-chain/src/parameters/constants.rs`:20–30). The lesson for prose about pre-Crosslink "de facto finality": the rollback depth is node policy, not consensus, and it currently differs by a factor of about ten between upstream and the feature-net prototype. The *Ironwood Guide*'s anchor-depth discussion (§"Resting: the note in the commitment tree") states the same policy-not-limit framing, and the *Consensus Guide* owns the full three-way divergence register — specification, retired zcashd, deployed zebra — in §"Reorg rails, maturity, and the finality floor".

Classification. *Specified as code* at prototype rigor only. Several RPC comments still call their methods placeholders, and nothing in this subsection is consensus.

5.4 Anchor semantics: the undecided keystone

Crosslink 2 leaves transaction validity alone: "The consensus rules applying to bc-transactions are entirely unchanged, including any rules that depend on bc-block height or previous bc-blocks", because the construction never reorders or sanitizes transactions; contextual validity in Π_{bc} is that of Π_{origbc} modulo the non-contextual new block rules (tfl-book `construction.md`:267, 701–703). The Orchard anchor rule is therefore inherited as-is: a spend may reference the treestate of *any* main-chain block from Orchard activation onward, with no sliding window (*Ironwood Guide*, §"Resting: the note in the commitment tree", subsection "Anchor choice, reorgs, and the anonymity set"; mainnet floor 1,687,104) — evaluated against the transaction's candidate block ancestry, not against a node's truncated localba display. No source restricts spend anchors to the finalized prefix: not the tfl-book, not the SL book, not either implementation snapshot (no Crosslink-dependent anchor check at `6d02a1b8` or `807b995`), and not the ZIP corpus (@ `e753a6a3`).

The wallet-level anchor policy under two ledgers is thus the single most consequential undecided item in the contract. A future ZIP must decide two things: (a) whether consensus adds any anchor restriction — nothing is proposed anywhere — and (b) what a wallet's default anchor and note-selection policy becomes relative to the finalized height. The branch costs are concrete:

- *Anchor only in* localfin. Newly received notes must wait for their treestate to finalize before they can be spent. No Crosslink 2 bound on that latency is established (§5.1); L bounds a block gap, not how long a stall lasts. In exchange, Assured Finality protects that anchor's block in the agreed finalized history. This safety property alone does not guarantee its presence in every transient best-chain view, timely inclusion, or transaction validity under other rules. The wallet must still check the candidate ancestry; the *Wallet Guide*'s retry caveat remains relevant whenever an anchor is absent from that ancestry (§"The proposal").

- *Keep the tip-relative anchor.* ZIP 315's rule — "if the current block is at height H, the anchor SHOULD reflect the final treestate of the block at height H-3" (`zips/zip-0315.rst`:586–587) — preserves latency and the anonymity-set size and uniformity that motivate near-tip anchoring (*Ironwood Guide*, ibid.), and retains the reorg caveat for the non-final suffix.

The bounded-availability argument already couples anchors to guaranteed balances: a party's guaranteed minimum balance is "the minimum balance taken over all possible transaction histories

that extend the finalized chain", accounting for republication of its transactions without consent; "we know that shielded transactions cannot be reapplied after their anchors have been invalidated. It may be desirable to further constrain re-use in order to make guaranteed minimum balances easier to compute" (tfl-book `the-arguments-for-bounded-availability-and-finality-overrides.md`:50). That sentence is the closest any source comes to proposing an anchor restriction, and it proposes nothing concrete.

Why plain chains matter here: under Snap-and-Chat sanitization, expiry preserved the idempotence argument (an expired transaction stays expired on recheck), but anchors broke it — "it *is* technically possible for a duplicate transaction that was invalid because of a missing anchor to *become* valid in a subsequent block ... the same commitment treestate can be produced by two unrelated transactions" — and some finalized outputs could become unspendable outright, transactions in $\mathrm{LOG}_{\mathrm{fin}}$ stranded on a fork no longer in any best chain (tfl-book `notes-on-snap-and-chat.md`:67–70, 125 and "Spending finalized outputs"). Crosslink 2 fixes both by construction, via the Linearity and Last Final Snapshot rules, and avoiding sanitization "would avoid breaking assumptions that may be made by existing Zcash node implementations and other consumers of the Zcash block chain" (`security-analysis.md`:263; `potential-changes.md`:348–363). The wallet-visible consequence: under Crosslink 2, exactly the anchor and expiry semantics documented in the *Ironwood Guide* and *Wallet Guide* carry over — on each of the two chains separately.

Classification. The inherited anchor rule is *designed but unspecified* (construction text, both books). The anchor policy under two ledgers — both the consensus question and the wallet default — is an *open problem*, and this volume flags the branch-cost analysis above as its own synthesis.

5.5 Confirmation policy under two ledgers

The deployed baseline is ZIP 315 (Status: Draft), which the *Wallet Guide* develops in §"The proposal": a ConfirmationsPolicy with trusted depth 3, untrusted depth 10, zero-conf shielding allowed by default, an advisory floor of 1, and the $H - 3$ anchor rule quoted above. ZIP 315 predates Crosslink and never mentions it.

How that policy maps onto a finality status is unresolved everywhere. The candidate shapes: the trusted/untrusted distinction collapses to final/not-final; it becomes a three-state policy (final / trusted-unfinalized / untrusted-unfinalized); or it stays unchanged with finality surfaced only in UX. Shielded Labs commits to exposing the status — "augmentation of existing RPC APIs for blocks and transactions to include finality status" (`book/src/design/deliverables.md`, GH #105) — but no mapping into any confirmations policy exists in any source. A related open point is whether displayed confirmation counts should saturate at "final": the SL book argues that when users' rollback thresholds diverge during a stall, "those users who require finality and those users who do not ... diverge in their expectations and behavior, which is a growing economic and governance risk" (`book/src/design/security-properties.md`:17–19) — finality removes that fragmentation only if wallets adopt it uniformly.

The ECC design goals pull in two directions at once: wallet developers "should not be required to make any changes through protocol transitions as long as they rely solely on the lightwalletd protocol or a full node API"; yet "the Crosslink construction may require wallets to alter their context of valid transactions differently from the current NU5/NU6 consensus protocol"; and there must be "no security or safety degradation due to wallet user behavior ... assuming users follow their current behaviors unchanged" (tfl-book `src/design/goals.md`:9, 15–16, 28). The first goal is untestable today: no lightwalletd or zaino wire protocol for finality status exists in any source.

Classification. *Open problem* in every direction that matters to a wallet author; the only *specified*-adjacent artifact is ZIP 315 itself, a Draft that predates the construction.

5.6 Stalls: the divergence that decides wallet behaviour

The finality gap at time t is, roughly, the number of bc-blocks not yet finalized; when roughly constant it corresponds to the time a transaction takes to finalize after inclusion. A gap exceeding L "likely signals an exceptional or emergency condition, in which it is undesirable to keep accepting user transactions that spend funds" (tfl-book `construction.md`:403–405). The construction enforces this as a block-validity rule: with $\mathsf{finality\text{-}depth}(H) := \mathsf{height}(H) - \mathsf{height}(\mathsf{snapshot}(\mathsf{LF}(H)))$, every bc-block must satisfy $\mathsf{finality\text{-}depth}(H) \leq L$ or be a stalled block (`construction.md`:680–693).

Stalled Mode, as ECC designed it. The proposed Zcash instantiation of is-stalled-block is coinbase-only blocks — "Since funds cannot be spent in coinbase-only blocks, the vast majority of attacks that we are worried about would not be exploitable" — possibly with shielded coinbase disabled; mining pools cannot distribute rewards during the stall (a rug-pull risk the book notes); and Stalled Mode "can only be entered by consensus of a larger validator set, or if there is an availability failure of the finalization protocol" (tfl-book `the-arguments-for-bounded-availability-and-finality-overrides.md`:24, 103–111, 187).

The divergence. The SL book pastes the construction verbatim but disclaims exactly this rule: "The zebra-crosslink design in this book diverges from this text by *not* including 'Stalled Mode' rules. This simplification to the protocol rules follows the rationale that we are already relying heavily on the stakers to guard safe operation of the protocol by redelegating appropriately to prevent hazards like BFT stalls" (`book/src/design/cl2-construction.md`:9–12). No stalled-mode or coinbase-only enforcement exists anywhere in the implementation (grep over `zebra-crosslink` @ `6d02a1b8` sources). The paste disclaimer adds that Daira-Emma Hopwood and Jack Grigg "have not reviewed and do not necessarily endorse its content or design changes relative to Crosslink 2"; apart from the admonitions the construction text is byte-identical to the tfl-book's.

Remark 5.5 (Expiry as stall cleanup — synthesis)**.** Neither book discusses the interaction with transaction expiry; the following is this volume's synthesis. Under the ECC branch, coinbase-only blocks keep heights advancing. A pending spend using the usual target-plus-40 expiry therefore eventually expires if blocks keep arriving; a transaction already pending at the stall normally has no more than that interval left. This is not universal: custom expiries, transactions with no expiry, and new transactions created during the stall need separate treatment. Wallet note locks release only after observing the applicable expiry and applying the wallet's policy (*Wallet Guide*, §"Expiry (ZIP-203)" and §"Expiry and fund availability"). Under the SL branch finite-expiry pending spends still expire, but new spends can keep confirming into an unbounded finalization gap — the condition the ECC arguments chapter deems unacceptable — and a service's `NotYetFinalized` exposure grows without bound. Whether a Crosslink ZIP should couple the expiry delta to L (for instance $\Delta < L$, or Δ on the order of the finality latency) is unaddressed in any source.

Overrides. Long rollbacks are argued to need a formalized, governance-specified procedure, with the Bitcoin 2010 value-overflow rollback (53 blocks, about 9 hours) and the Ethereum DAO fork as precedents; bounded availability is complementary because it bounds the period of spend transactions an intentional rollback could affect, and the envisioned procedures preserve coinbase-only mining rewards across the rollback (tfl-book `the-arguments…md`:29, 148–198). On the governance side, the SL draft ZIP on user-led emergency hardforks "explicitly rejects the use of an emergency user-led hard fork to mitigate harm arising from application-layer failures" — not wallet bugs, bridge exploits, custodial failures, nor DAO-style reversals (`book/src/design/userled-emergency-hardforks.md`:13) — bounding what the escape hatch may be used for.

Classification. The Finality depth rule is *designed but unspecified* (tfl-book normative text); its omission is likewise *designed but unspecified* (SL book admonition plus absent code). The conflict

itself is a finding: a ZIP must pick one branch, and each branch changes what wallets and exchanges must handle during a stall — a hard availability stop with expiry churn, or indefinitely growing unfinalized exposure. Remark 5.5 is synthesis.

5.7 Exchanges, bridges, and downstream protocols

The stated service-facing goal is uniformity: "CEXes and decentralized services like DEXes/bridges are willing to rely on Zcash transaction finality. E.g. Coinbase reduces its required confirmations to no longer than Crosslink finality. All or most services rely on the same canonical (protocol-provided) finality instead of enforcing their own additional delays or conditions, so users get predictable transaction finality across services" (`book/src/design/scoping.md`:10, 30; GH #131, #128). The committed interface deliverables are finality status on block and transaction RPCs (GH #105), proof-of-stake RPCs for finalizer identities, staking weights, and delegation info (GH #104), and "contingency mode, finality traversal" RPCs (GH #172); external validation partnerships prioritize wallet developers, finalizers, and exchanges (`book/src/design/deliverables.md`).

The risk model a service would price is the SL five-component allocation: miners "cannot unilaterally violate finality, even if an attacker controls $> 1/2$ of the hashpower"; finalizers cannot unilaterally compromise liveness or censorship-resistance even with $> 2/3$ of stake-weighted votes, and cannot execute rollbacks though they can prevent them; more than $1/3$ of stake can stall; and a collusion of $> 1/2$ hashpower with $> 2/3$ stake can violate finality, execute rollbacks, and mount unavailability attacks (`book/src/design/five-component-model.md`:24–46). The SL book separates Finality Progress from Liveness precisely because the anticipated failure is a stall under continued block production, and part of the design intent is to "convert censorship attacks into stall attacks" (`book/src/design/security-properties.md`:17–19, 35).

What remains open on this surface: deposit semantics during a finalization stall; whether `CantBeFinalized` can ever surface for a transaction a service already credited; the contingency-mode and finality-traversal RPC semantics (GH #172 is open); and the light-client wire protocol of §5.5. Each is an *open problem*: the slogan is designed, the contract behind it is not.

Downstream protocols. For Zally coin voting, Crosslink finality bears on the snapshot pin: a round binds its snapshot height and block hash in its round identifier (`valargroup/vote-sdk` @ `cb915f51`, `proto/svote/v1/tx.proto`:36–48 and `x/vote/keeper/msg_server.go`:30–48). Requiring the pinned block hash to identify a block in the finalized prefix would protect the pin under Assured Finality; comparing heights alone does not authenticate a hash on another fork. An orthogonal trust gap is untouched: the ante handler passes the stored coordinator roots to the proof verifier as public inputs, while the circuit documentation explicitly disclaims authenticating their provenance (`valargroup/vote-sdk` @ `cb915f51`, `x/vote/ante/validate.go`:147–177; `valargroup/voting-circuits` @ `4c39abd5`, `src/delegation/README.md`, "Public Input Provenance"). This resolution analysis is this volume's synthesis.

Project Tachyon and Crosslink are asserted to be independent. Bowe's founding post sees "no reason to believe that Tachyon will conflict" with Crosslink, and Shielded Labs calls them "separate efforts that don't necessarily interact" for roadmapping or timelines (Bowe, "Tachyon: Scaling Zcash with Oblivious Synchronization", 2025-04-02; forum t/50789 #11, 2025-10-28). No interaction analysis exists in either direction, and the ZIP repository still contains no ZIP specifying NU8 (`zcash/zips` @ `e753a6a3`, 2026-09-03).

Classification. The service goals and deliverables are *designed but unspecified* (scoping and deliverables documents with open issue numbers); the five-component allocations are *designed but unspecified* security claims assessed in §6.6; every concrete service-facing behaviour during a stall is an *open problem*.

6 Security posture and open problems

The security posture of Crosslink 2 is best stated as an inventory of five artifacts of unequal maturity. The design book states its central safety theorem twice over, from two independent assumptions. The second theorem's published proof text is defective, but its intended argument can be completed (Remark 6.3); the analysis then breaks off: everything below a stated line of its analysis is marked "Not updated for Crosslink 2", and its last proof sketch ends mid-sentence (tfl-book @ `fe6e1d6`, 2024-12-13, `src/design/crosslink/security-analysis.md`, lines 54 and 253–257). A formal model machine-checks exactly one validity predicate of the BFT side and nothing else (crosslink-spec @ `8edc3e9`, 2025-07-23). The implementation departs from the analyzed design in two load-bearing ways that it documents itself (zebra-crosslink @ `6d02a1b`, 2026-04-21); the active feature-net prototype has since moved again (crosslink_monolith @ `807b995`, 2026-08-13). An independent mechanism-design review finds the construction's safety "structurally strong once finality is reached" and its finality progress "economically fragile in the baseline design" (`https://www.nikete.com/crosslink_zebra_audit.pdf`, completion announced 2026-02-05). We take the five in turn, keep the bin of every claim visible — *specified* here can only mean verifiable source at a pinned commit, since no Crosslink ZIP exists (zips @ `e753a6a`, 2026-09-03) — and close with the open-problem ledger and its revisit triggers. The rules and properties named below are stated in §3.4, §3.5, and §3.6; the source pins are those of Table 1.

6.1 The safety theorems and their assumptions

The book's central safety goal is *Assured Finality* (Definition 3.20): in a given execution, for all times t, u and all nodes i, j such that i is honest at time t and j is honest at time u, the finalized views agree, $\mathsf{fin}_i^t \sim_{\mathrm{bc}} \mathsf{fin}_j^u$ — either is a prefix of the other (tfl-book @ `fe6e1d6`, `construction.md`, lines 504–506). The name is chosen against "absolute finality": the property binds *the protocol*, and the book states explicitly that out-of-band reversal by community fork remains possible (`terminology.md`, lines 11–13). The implementation's gloss, "crosslink finality", strengthens the wording: accountable via explicit on-chain voting records, irreversible within protocol scope, objectively verifiable from local block history, globally consistent (zebra-crosslink @ `6d02a1b`, `book/src/design/terminology.md`, lines 7–13). Two theorems each imply Assured Finality from a safety property of one subprotocol alone.

> **Theorem 6.1** (Prefix Agreement implies Assured Finality)**.** *In an execution of Crosslink 2 in which subprotocol Π_{bc} has Prefix Agreement (Definition 3.6) at confirmation depth σ — all honest nodes' bc-best-chains, truncated by σ blocks, pairwise agree across all times — that execution has Assured Finality (tfl-book @ `fe6e1d6`, `security-analysis.md`, lines 72–77; Prefix Agreement at `construction.md`, lines 96–99 and 337–340).*

The written proof is complete and short: the Local fin-depth lemma places each fin_i^t inside some earlier σ-truncated best chain of node i (`construction.md`, lines 491–494), Prefix Agreement relates any two such chains, and the Linear prefix lemma — two prefixes of a common chain agree — closes the argument (`construction.md`, lines 77–82). The book contrasts this with the corresponding result for Snap-and-Chat: "[NTT2020, Theorem 2] is a much more complicated proof that takes nearly 3 pages, not counting the reliance on results from [PS2017]" (`security-analysis.md`, line 162). The comparison is fair in kind but not in coverage: NTT2020's Theorem 2 is a probabilistic security statement about the longest-chain protocol itself (failure probability $e^{-\Omega(\sqrt{\sigma})}$ for block-production rate $p < (n-2f)/(2\Delta n(n-f))$; arXiv:2009.04987v3, Appendix C), whereas Theorem 6.1 *assumes* the corresponding property of Π_{bc} as a black box. That division of labor is this series' scope rule in protocol form: Crosslink consumes k-deep-confirmation guarantees of the PoW layer; it does not re-derive them.

Theorem 6.2 (Final Agreement implies Assured Finality)**.** *In an execution of Crosslink 2 in which subprotocol Π_{bft} has Final Agreement (Definition 3.9) — for all bft-valid blocks C, C' in honest view at any times, bft-last-final$(C) \sim_{\text{bft}}$ bft-last-final(C') — that execution has Assured Finality (tfl-book @ `fe6e1d6`, `security-analysis.md`, lines 86–91; Final Agreement at `construction.md`, lines 203–206).*

Proof. Fix honest observations (i, t) and (j, u). Let $r \leq t$ and $s \leq u$ be the last times their local finalization values changed, taking genesis time if they never changed. Honesty at t or u means protocol compliance at every earlier time. By the update rule and the definition of candidate,

$$\mathsf{fin}_i^t \preceq_{\text{bc}} \mathsf{snapshot}(\mathsf{LF}(\mathsf{ch}_i^r)), \qquad \mathsf{fin}_j^u \preceq_{\text{bc}} \mathsf{snapshot}(\mathsf{LF}(\mathsf{ch}_j^s)).$$

The corresponding $\mathsf{context}_{\text{bft}}$ blocks are bft-valid blocks in honest view, so Final Agreement makes their LF blocks bft-comparable. Along the later LF block's ancestor path, the Parent and Linearity rules make their snapshots bc-comparable. Without loss of generality, let the first snapshot prefix the second. Both local finalization values are then prefixes of the second snapshot, so the Linear prefix lemma makes them bc-comparable. This is Assured Finality. The argument is the construction chapter's sketch at line 592, expanded using its definitions at lines 423–435 and 462–488. □

Remark 6.3 (The written proof of Theorem 6.2 is defective)**.** The proof text pasted under the theorem in the book is a verbatim duplicate of the proof of Theorem 6.1: it invokes Prefix Agreement of Π_{bc}, not Final Agreement of Π_{bft} (`security-analysis.md`, lines 86–91). The source therefore needs an erratum. The proof supplied immediately above uses its intended Linearity route and only premises already stated in the construction. Bin: *designed but unspecified*; the argument is not peer-reviewed or machine-checked, but the source-text defect does not leave the theorem logically unsupported.

The pair of theorems, where both hold, gives the design's headline property: safety of the finalized ledger requires only *one* of {Π_{bc} safety, Π_{bft} safety} — the stronger of the two. The book carries the same two-route structure down to the ledger level: in an execution where Π_{bc} has Prefix Agreement at depth σ *or* Π_{bft} has Final Agreement, any two honest nodes' finalized ledgers $\mathsf{LOG}_{\text{fin}}$ agree at all times (`security-analysis.md`, lines 117–176). That section, however, sits below the "Not updated for Crosslink 2" line (line 54), so its statements are for Crosslink 1 and their transfer is unverified.

Two further results are proved, both assuming the *one-third bound on non-honest voting* (Definition 3.7): strictly fewer than one third of each epoch's voting units are ever cast non-honestly, counted over the entire execution (`construction.md`, lines 131–141). First, uniqueness: any bft-valid block for a given epoch in honest view commits to the same proposal, by quorum intersection of a pair of two-thirds vote sets, adapted from [CS2020, Lemma 1] (`security-analysis.md`, lines 208–213). Second, snapshot validity: with Prefix Consistency of Π_{bc} at depth σ added, every bc-chain snapshot contributing to $\mathsf{LOG}_{\text{fin}}$ eventually sits in every honest node's bc-best-chain (`security-analysis.md`, lines 232–251).

Assumption transfer. The theorems assume properties of the *modified* subprotocols Π_{bc} and Π_{bft}, while the underlying literature supports the *original* protocols Π_{origbc} and Π_{origbft}. In the BFT direction the transfer is argued: Π_{bft} only adds structural components and tightens validity rules, so every Π_{bft} execution maps to a Π_{origbft} execution with the additions erased. In the PoW direction it is acknowledged as unproven: the modifications "seem unlikely to have any significant effect on this property", followed by "TODO: that is a handwave; we should be able to do better, as we do for Π_{bft} below" (`security-analysis.md`, lines 119 and 170). Bin: *open problem*.

The residual attack the design accepts. If the network is partitioned *and* Π_{bft} is subverted by more than one third of validators, the adversary can vote with an additional one third on each side of the fork, so that each side's last-final bft-block is consistent with its own fork. The book calls this

unavoidable: "Π_{bc} doesn't include the mechanisms needed to maintain finality under partition; it takes a different position on the CAP trilemma." Snap-and-Chat, by contrast, loses safety under BFT subversion even *without* a partition; Crosslink's stated aim is to limit safety loss to attacks "like" the unavoidable one (`security-analysis.md`, lines 109–115). The corresponding goal exceeds NTT2020's own argument: the paper's Figure-9c reasoning — one honest vote is needed to finalize a snapshot, hence only valid snapshots finalize — applies only for adversarial fractions $n/3 < f < n/2$, whereas the book wants the conclusion to survive an adversary holding 100% of validators but under 50% of Π_{lc} hash rate (`notes-on-snap-and-chat.md`, lines 205–225).

Classification. Every statement above is *designed but unspecified*: the proofs live in a design book at a pinned commit, not in a ZIP, a peer-reviewed paper, or machine-checked form. Within that bin, Theorem 6.1 is complete on its stated assumptions; Theorem 6.2 has the source-text erratum and the completed argument of Remark 6.3; the ledger-level and snapshot-validity theorems predate Crosslink 2; and the $\Pi_{origbc} \to \Pi_{bc}$ assumption transfer is an acknowledged gap.

6.2 Liveness and the unfinished analysis

The liveness side of the analysis is prose, and the book says so. Two conclusions are argued but not proven: subprotocol Π_{bc} remains live under the same conditions as Π_{origbc}, because the added consensus rules never obstruct producing a coinbase-only stalled block on the current best chain; and Π_{bft} remains live under the same conditions as $\Pi_{origbft}$, because the Linearity and Tail Confirmation rules always permit an honest proposer to re-propose its parent's `headers_bc` (tfl-book @ `fe6e1d6`, `security-analysis.md`, lines 25–42). Two open TODOs sit inside the argument: that a new higher-scoring snapshot eventually becomes final ("make that more precise"), and that Stalled Mode triggers only when the BFT liveness assumptions are violated — "more detailed argument needed, especially for the dependence on L", where the dependence involves σ, network delay, honest hash rate, and honest proposers and voting units (lines 46 and 50). The quantitative relationship among these parameters was never worked out anywhere in the sources we have read; the book's design guidance is only that L be "significantly greater than σ" and "in practice ...at least 2σ" (`construction.md`, lines 391 and 405). Bin: *open problem*.

Three further admissions bound what the analysis currently delivers. First, the rollback exposure of the available ledger: the book proves that LOG_{ba} never rolls back past the agreed LOG_{fin}, but notes the result is weaker than desired, since rollbacks of LOG_{ba} down to the finalization point "may be of unbounded length" — bounding them by L is asserted loosely and "we have also not proven that so far" (`security-analysis.md`, lines 180–190). Second, rule necessity: the claim that every Crosslink 2 rule is individually necessary is qualified — "some rules, e.g. the Linearity rule, only contribute heuristically to security in the analysis so far" (lines 275–281). Third, validation cost: honestly voting on a proposal may recursively require validating the referenced bft-chain, each bft-block's `headers_bc` chain, and so on — the book's own gloss is "Wait what, how much validation is that?" — with denial-of-service bounded only by validation ordering (check the supermajority and PoW first) and by the Linearity rule limiting live bc-chains per bft-chain to one (`construction.md`, lines 641–652).

Remark 6.4 (Fork-choice independence)**.** Crosslink 2 deliberately does not modify the fork-choice rule of Π_{bc}, and the book identifies this as the reason the bidirectional-dependence liveness failure of Casper-FFG-style designs (NTT2020, Appendix E, "Bouncing Attack on Casper FFG") does not arise: an honest bc-block-producer "must not use information from the BFT protocol, other than the specified consensus rules, to decide which bc-valid-chain to follow" (`security-analysis.md`, lines 15–23; `construction.md`, lines 715–717). The same modularity is why requiring a node's bc-best-chain to extend $\mathsf{snapshot}(B)$, for B the last final bft-block in that node's view, was rejected as a fork-choice constraint: under a potentially subverted Π_{bft}, "we cannot say anything useful about $\mathsf{snapshot}(B)$", so the constraint would break the modular safety and liveness arguments. Honest validators may instead

simply refuse to vote for proposals embedding long rollbacks — the voting rule is "only if", not "iff" — at an accepted cost in BFT liveness ("and that's okay"; `questions.md`, lines 44–58).

Remark 6.5 (The model is not NTT2020's model)**.** The book's communication model deliberately avoids global-stabilization-time partial synchrony as a base assumption — messages may be arbitrarily delayed or dropped — with a detailed critique of applying the DLS1988 GST formalization to liveness of non-terminating blockchain protocols, including in NTT2020 and the Streamlet paper; safety properties are stated as unconditional properties of executions, with probabilistic reasoning factored out separately (`construction.md`, lines 13–41 and 342–366). A consequence worth recording: NTT2020 defines a (β_1, β_2)-secure ebb-and-flow protocol by exactly such GST/GAT-parameterized environments (Definition 3) and proves Snap-and-Chat optimally $(1/3, 1/2)$-secure (Theorem 1; arXiv:2009.04987v3), and no theorem of that shape appears for Crosslink 2 in any source we have read. The two analyses are therefore not directly comparable, by the book's own choice of model. Bin: *open problem* — no (β_1, β_2)-style security statement for Crosslink 2 exists in any pinned source.

The book's status chapter has not moved since 2024: "This is an early and incomplete protocol design proposal. It has not been well vetted for feasibility and safety", with "Security and safety analyses" still on the missing-components list beside subprotocol selection, issuance, a transition plan, and ZIPs (`status-and-next-steps.md`, lines 3 and 19–27). The book has been untouched since 2024-12-13 while the implementation advanced through 2026-04; which document is the authoritative security argument for the protocol actually being built is itself an open problem (§6.7).

6.3 Bounded availability, Stalled Mode, and overrides

Crosslink's finality is *conditional* by construction. An honest validator votes for a proposal only if it is bft-proposal-valid and its snapshot is σ-confirmed in the validator's own bc-best-chain (`construction.md`, lines 654–657), so a coalition holding more than one third of an epoch's voting units can stall finality indefinitely: finality arrives only while a two-thirds quorum remains live and honest enough. The design's answer is to make the stall visible and bounded in its consequences rather than impossible: once the gap between the tip and the finalized snapshot exceeds L blocks, the Finality depth rule forces every new block to be a coinbase-only "stalled" block (`the-arguments-for-bounded-availability-and-finality-overrides.md`, lines 180–188), and Stalled Mode "can only be entered by consensus of a larger validator set, or if there is an availability failure of the finalization protocol" (tfl-book @ `fe6e1d6`, `the-arguments-….md`, line 187). The same machinery is a censorship defense: part of the design intent is "to 'convert censorship attacks into stall attacks'" — an attacker cannot censor targeted transactions without stalling the whole service, and a stall is immediately and consensually visible while censorship tends to be visible only to its victims (`book/src/design/security-properties.md`, line 35).

The argument for bounding availability at all is the CAP position: finality plus dynamic availability plus the possibility of partition implies an unbounded finalization gap (made precise by [LR2020]), and allowing spends into an unbounded gap risks a crisis of confidence; the book cites Ethereum's May 2023 double finality stall (≈25 and ≈64 minutes, a Prysm resource-exhaustion bug) as evidence that stalls happen, that short ones self-heal, and that long ones need human intervention (tfl-book @ `fe6e1d6`, `the-arguments-….md`, lines 31–41 and 135–139). The reason to prefer coinbase-only blocks over halting outright is the tail-thrashing attack: during a stall of a halted chain, an adversary with 10% of hash power expects to build a 10-block fork in 100 original-network block times at unchanged difficulty because the honest chain is not advancing, letting the k-block tail thrash between chains — and hash rate is expected to *fall* during a stall as miners anticipate rollback (lines 200–229). Two caveats attach. The constraint binds only while a producer's view of the finalization point is not advancing: "an adversary that has subverted the BFT protocol in a way that does not keep the finalization point from advancing, can always avoid being constrained by Stalled Mode" (`construction.md`, lines 409–411). And finality can be overridden deliberately: the book argues override must remain possible for exploited flaws (Bitcoin's 2010 value-overflow rollback of 53 blocks over roughly nine

hours; Ethereum's DAO fork), should be *formalized* under a well-defined governance process, and composes with bounded availability — validators asked to stop cause a stall, the chain enters Stalled Mode after L blocks, only a bounded window of user spends is exposed to the rollback, and miners' coinbase-only rewards would be preserved across it (`the-arguments-….md`, lines 148–198).

Remark 6.6 (Long-range attacks and key hygiene)**.** The one-third bound counts ballots cast at *any* time in the execution, so a compromised old validator key used to vote for a long-finished epoch violates it retroactively. The book's consequence is operational: "validator keys of honest nodes must remain secret indefinitely. Whenever a key is rotated, the old key must be securely deleted." The recommended mitigation is a checkpoint bft-block baked into each released node version, with local finality exposed to clients only once synced past it; improvements are tracked as tfl-book issue #140 (`construction.md`, lines 149–154 and 557–562). Bin: *designed but unspecified*, with the issue open.

Classification. The Finality depth rule and Stalled Mode are *designed but unspecified* consensus rules of the book's Crosslink 2; the implementation has dropped them (§6.5). The governance process that a formalized finality override presupposes does not exist in any pinned source; the accountability machinery that would let a bc-transaction carry objective evidence of a BFT safety violation (two final bft-blocks with the same parent, a "bft-final-vee") is a change marked [Recommended] but not adopted into Crosslink 2, and remains unimplemented (`potential-changes.md`, lines 12–24; `questions.md`, lines 41–42). Both are *open problems*.

6.4 Machine-checked coverage: the Quint model

The only machine-checked artifact in the corpus is the Informal Systems Quint specification, and its own README bounds its scope: it is a self-described proof of concept that encodes "a tiny part of the system, namely the validity check, of a crosslink proposal within the BFT consensus mechanism", with most of the required information "inferred ...from an insightful Youtube talk", a warning that "the encoded logic might not be up-to-date with the current protocol design ideas", and a closing question to the community "whether we should move forward with applying for a grant" (crosslink-spec @ `8edc3e9`, `README.md`, lines 27 and 93).

What the model checks is nonetheless crisp. The validity predicate is

$$\begin{aligned}\mathsf{isValid}(\mathit{bft}, \mathit{pow}, p) = \mathit{pow}.\mathsf{contains}(p.\mathsf{block}) \;\wedge\; & p.\mathsf{bftContext} = \mathit{bft}.\mathsf{last}() \\ \wedge\; & \mathsf{sigmaConfirmed}(p.\mathsf{block}, p.\mathsf{confirmation\_tail}, \sigma) \\ \wedge\; & \mathsf{linear}(p.\mathsf{block}, p.\mathsf{bftContext}, \mathit{pow}),\end{aligned}$$

over a single node's `pow_store` and `bft_store` (`validity.md`, lines 131–176). Against it the spec generates a finalization witness (reachability of three BFT blocks) and checks the invariant `prefix_inv` — "the PoW history defined by BFT finalization is a path in the PoW block store" — by random simulation (10,000 traces of 20 steps, roughly 132 seconds, no violation found) and by TLC model checking after transpilation to TLA$^+$, on stores of at most 8 blocks at confirmation depth 1, in about three minutes (`README.md`, lines 37–89; `validity.md`, lines 405–410 and 463–482; `src/validity.cfg`). The model contains no network, no adversary, no voting, no epochs, no signatures, none of the bc-side rules (Extension, Last Final Snapshot, Finality depth), no Stalled Mode, and no liveness property.

Remark 6.7 (A divergence the spec itself flags)**.** The Quint operator $\mathsf{linear}(b, \mathit{ctx}, \mathit{store})$ conjoins descent with $\neg(\mathit{ctx}.\mathsf{pow\_hash} = b.\mathsf{hash})$: it *rejects* a proposal whose snapshot equals the parent's snapshot. The book's Linearity rule uses $\preceq_{\mathrm{bc}}$, which includes equality — and equality is required for Π_{bft} liveness, since an honest proposer repeats its parent's `headers_bc` when it cannot extend. The spec's own comment reads "this actually seems allowed. TODO: perhaps remove" (crosslink-spec @ `8edc3e9`, `validity.md`, lines 171–176; tfl-book @ `fe6e1d6`, `construction.md`, lines 71, 573, 611–616, and 627).

Classification. The Quint model and its check results are *specified* in the only sense available — runnable, pinned source — but of deliberately narrow scope. Machine-checking has not progressed since: the spec's last commit is 2025-07-23, the companion `quint-crosslink` repository contains no commits at all, and a web search on 2026-09-05 found no public extension. Coverage of the bc-side rules, of any multi-node or adversarial property, and of liveness is an *open problem*.

6.5 The implementation diverges from the analyzed design

The zebra-crosslink book embeds verbatim copies of the tfl-book's construction and security-analysis chapters, made "for the purposes of precisely committing to a (potentially incomplete, confusing, or inconsistent) precise set of book contents for a security design review"; diffing them against tfl-book @ `fe6e1d6` shows the only changes are the warning admonitions (three prepended to the construction copy, one — the paste notice — to the security-analysis copy), and the copies state that Daira-Emma Hopwood and Jack Grigg "have not reviewed and do not necessarily endorse" them (zebra-crosslink @ `6d02a1b`, `book/src/design/cl2-construction.md` and `design/analysis/cl2-security-analysis.md`). The project's own design chapter is franker still: "These extensions and changes from the Crosslink 2 construction may violate some of the assumptions in the ssecurity [sic] proofs. This still needs to be reviewed after this design is more complete", and "We are using a custom-fit in-house BFT protocol for prototyping. It's [sic] own security properties need to be more thoroughly verified" (`book/src/design.md`, lines 11–12). Two of the admonitions record load-bearing divergences.

Divergence 1: Stalled Mode is dropped. "The zebra-crosslink design in this book diverges from this text by *not* including 'Stalled Mode' rules. This simplification to the protocol rules follows the rationale that we are already relying heavily on the stakers to guard safe operation of the protocol by redelegating appropriately to prevent hazards like BFT stalls" (`book/src/design/cl2-construction.md`, lines 10–12). The implementation as designed therefore has an unbounded finalization gap with spends continuing — precisely the condition the tfl-book's bounded-availability chapter argues is unacceptable, and the regime whose tail-thrashing analysis motivated Stalled Mode in the first place (tfl-book @ `fe6e1d6`, `the-arguments-….md`, lines 17–27; §6.3). No pinned source reconciles the two positions.

Divergence 2: the outer signature is dropped. "The zebra-crosslink design does not use the 'outer signature' referred to in this chapter... This means a direct hash or copy of the most recent BFT finality certificate is insufficient evidence to ensure that specific bitwise copy is canonical or will become canonical in the chain" (`book/src/design/cl2-construction.md`, lines 14–16). The book's model has the proposer re-sign (P, proof_P) with a strongly unforgeable signature (tfl-book @ `fe6e1d6`, `construction.md`, lines 156–159). The April prototype's direct certificate representation is visible in code, but no source supplies a security argument showing what replaces the outer signature's property.

Which BFT protocol, exactly. The book's analysis instantiates Π_{origbft} as adapted Streamlet: notarization at two-thirds of voting units, finality at the second of three adjacent notarized blocks with consecutive epochs, liveness after five consecutive honest-leader epochs (tfl-book @ `fe6e1d6`, `construction.md`, lines 218–241 and 614–618). The April snapshot instead uses "tenderlink" 0.1.0, an in-house Tendermint-style engine pinned from `git.sr.ht/~azmr/stuff` at revision `0e885095…`; an earlier Malachite (Informal Systems) integration survives only as dead code — `src/malctx.rs` still imports `malachitebft_*` crates but is not declared as a module, and `Cargo.lock` contains no malachitebft packages (zebra-crosslink @ `6d02a1b`, `zebra-crosslink/Cargo.toml`; `Cargo.lock`, lines 5445–5459; `src/lib.rs`). The current monolith retains Tenderlink and supplies extensive executable tests, but no published analysis shows that it satisfies the properties the Crosslink safety

arguments assume of Π_{origbft} — Final Agreement, objective block validity, and sound two-thirds notarization. Bin: *open problem*, in the project's own words quoted above.

Parameters, and the black box today. Both prototype snapshots ship $\sigma = 3$ and $L = 7$ under `PROTOTYPE_PARAMETERS`; the April copy is annotated "No verification has been done on the security or performance of these parameters" (zebra-crosslink @ `6d02a1b`, `zebra-crosslink/src/chain.rs`:219–222; crosslink_monolith @ `807b995`, `librustzcash/zcash_primitives/src/bft.rs`:388–404). Production values are unspecified everywhere. Both snapshots use `MAX_BLOCK_REORG_HEIGHT = 99`; the non-finalized state never reorganizes deeper, and Crosslink uses the constant as its pre-BFT fallback finality depth (zebra-crosslink @ `6d02a1b`, `zebra-state/src/constants.rs`:31, `zebra-crosslink/src/lib.rs`:260–292; crosslink_monolith @ `807b995`, `librustzcash/components/zcash_protocol/src/consensus.rs`:639, `zebra-crosslink/zebra-crosslink/src/lib.rs`:464–518). Upstream Zebra has since raised the same constant to 1000 (zebra @ `d1fd7ad`, `zebra-chain/src/parameters/constants.rs`, line 30). The *Ironwood Guide* distinguishes this local Zebra limit from the protocol specification's 100-block rollback depth that validators are expected to handle; neither is an anchor-validity window. Wallet anchor recency is instead governed by confirmation policy (§"Anchor choice, reorgs, and the anonymity set"). We cite the bound as interface, per this series' scope rule; its derivation belongs to the PoW layer.

Implementation evidence of reorg fragility. Deriving the pre-Crosslink final block from Zebra's block locator proved subtle: the node wrote three alternative implementations of `tfl_reorg_final_block_height_hash()` and cross-asserted them, and the retained comment reads "NOTE: possible race condition: only for testing / Sam: YES! Indeed there were race conditions." The node also keeps a stateful `latest_final_block` that shadows the state-service view, mirroring the book's node-local fin_i state machine, whose "record finalization safety hazard" branch arises exactly when the global assumptions fail (zebra-crosslink @ `6d02a1b`, `zebra-crosslink/src/lib.rs`, lines 235–332; tfl-book @ `fe6e1d6`, `construction.md`, lines 471–488). The interleaving of BFT finality with PoW reorg handling is where the implementation has already met race conditions; it deserves adversarial review once the design settles.

6.6 Economic security: the mechanism-design review

The April zebra-crosslink design decomposes security across five components: wallets (privacy and censorship defense), miners (liveness and censorship resistance; unable to violate finality even with more than half the hash power), finalizers (more than one third of stake-weighted votes can stall but cannot unilaterally roll back), stakers (holding finalizers accountable solely by *redelegating* — there is no automatic slashing), and users (last-resort accountability via a user-led emergency hard fork, with STEEM/HIVE cited as precedent). Its own attack scenarios: a collusion of more than half the hash power *and* more than two thirds of stake can violate finality and execute rollback or unavailability attacks; and because staking and redelegation transactions themselves need censorship resistance, an attacker with more than half the hash power can censor redelegations and thereby disable the sole staker-accountability mechanism (zebra-crosslink @ `6d02a1b`, `book/src/design/five-component-model.md`, lines 5–52). The first deployment concentrates the finalizer set deliberately: the active set is the top $K = 100$ finalizers by stake weight, and support for finalizers run by arbitrary or casual users is explicitly out of scope (`book/src/design/scoping.md`, lines 53–57; `nutshell.md`, line 104).

The independent review of this mechanism is the nikete (Nicolas Della Penna) audit: pre-registered 2025-12-11, completion announced by Shielded Labs on 2026-02-05, report at `https://www.nikete.com/crosslink_zebra_audit.pdf`. Its verdict: "safety (irreversibility) is structurally strong once finality is reached", but "finality progress (liveness) is economically fragile in the baseline design and requires explicit incentive constraints". Three failure modes: *hostage holdout* — pending rewards accumulate until a finality-updating block, so a blocking

coalition rationally delays finality to grow its future payout; *zombie set* — the finality gadget becomes permanently inoperable through validator attrition during a freeze; and *phantom security* — if stake can exit economically while a frozen validator set retains voting weight, the effective cost to corrupt the frozen set decays over time. Its recommendations: signer-conditioned commission, first-inclusion miner bounties tied to finalized height, finality-gap telemetry, anti-jackpot constraints on pending rewards during stalls, and governance-level liveness resets (`https://www.nikete.com/crosslink_zebra_audit.pdf`, checked 2026-09-05; pre-registration at `book/src/design/analysis/history/2025-12-11-mech-design.md` and `…/2025-12-11-mech-design/proposal.md` @ `6d02a1b8`). The audit's ground rules bind its scope: no protocol-level slashing in V1 is a design constraint, the assumed economics were a 40/40 issuance split and 5-day epochs with 5–10-day unbonding, and privacy impacts on stakers or finalizers were out of scope (`…/2025-12-11-mech-design/proposal.md`, lines 33, 58, and 64–66). Current `v13` replaces the audited pending-reward mechanism with per-block bond compounding and implements a socially selected termination-and-burn path. The latter resembles the audit's governance-reset recommendation, but the repository does not attribute either change to the report and no follow-up audit establishes that its incentive findings are resolved.

The reward mechanics the audit targets are design-stage: pending rewards R distribute at a finality-updating block as $S_{\text{finalizer}} = c \cdot W_{\text{finalizer}}/W_{\text{total}} \cdot R$ and $S_{\text{staker}} = (1-c) \cdot W_{\text{staker}}/W_{\text{total}} \cdot R$ with commission rate $c = 0.1$. Every staker earns the staker slice regardless of its target finalizer. Totals may be below R only because commission attributable to stake on a non-active finalizer is unpaid (`book/src/design/nutshell.md`, lines 19–37; audit, §3). The hostage-holdout finding is a direct consequence of that "until a finality-updating block" accumulation. One privacy cost is already fixed at design stage: staking positions are ledger state with on-chain amount, target-finalizer key, and per-position redelegation key — "staked amounts are revealed so the community can verify the total ZEC staked and its distribution across finalizers" — mitigated only by quantizing stakes to powers of ten ZEC; the Crosslink Staking Orchard Restriction confines staking transactions to Orchard, fees, and burns, and Penumbra-style shielded delegation is deferred to "continued exploration in future versions" (`nutshell.md`, lines 39–104; `https://shieldedlabs.net/crosslink-updated-staking-design/`, 2025-10-27, checked 2026-09-05).

Classification. The five-component model, the reward split, and the staking design are *designed but unspecified*; the audit findings are *specified* as a pinned report but normative for nothing. Whether the baseline no-automatic-slashing accountability model, or `v13`'s socially selected burn, survives the phantom-security finding is an *open problem* — the audit proposal itself lists "unbounded divergence between PoW progress and a stalled BFT subprotocol" among the failure modes to examine, which is exactly the regime the implementation entered by dropping Stalled Mode (§6.5).

6.7 Standing, siblings, and the open-problem ledger

Normative standing. No Crosslink ZIP exists in the official repository: `e753a6a` (2026-09-03) contains Crosslink only as a citation footnote in draft ZIP 218 (25-second block target spacing) and as the source of the Assured Finality definition cited by `draft-ecc-onchain-accountable-voting`; the zebra-crosslink book says the ZIP draft existed as a Google Doc, to be refined "as the prototype approaches maturity" (`book/src/design.md`, line 7). Public overview and construction pre-drafts appeared in Shielded Labs' ZIP fork at `7cdceca` (2026-05-27), but neither is present on that fork's current main branch or in the official repository. They have no normative standing. NU8 scoping is an open governance discussion ("NU8 Decision Framework", forum thread opened 2026-02-11) in which Crosslink competes with the 25-second-blocks proposal and with Tachyon; their asserted independence is recorded in §5.7. Shielded Labs' roadmap shows Milestones 1–5 complete (2025-03-21 through 2026-04-15) and a productionization phase running Q2 2026 – Q2 2027 — testnet deployment, design iteration, potential integration (`https://shieldedlabs.net/roadmap/`, checked

2026-09-05). No confirmed mainnet activation date exists.

What Crosslink would change for the siblings. Today the stack expresses transaction confirmation entirely as depth policy on the black-box PoW chain: the *Wallet Guide* documents the ZIP-315 `ConfirmationsPolicy` with trusted depth 3 and untrusted depth 10 (§"Transaction construction", Definition "Confirmations policy"), and the *Ironwood Guide* distinguishes the specification's 100-block validator rollback expectation from Zebra's 1000-block local rollback policy; neither is an anchor-validity window (§"Anchor choice, reorgs, and the anonymity set"). Wallet anchor recency is separately governed by confirmation policy. Crosslink would add a protocol-level notion beneath both: finality status is the sole "height-eventual" ledger state it introduces — computable for height H only from a later attesting block, with the guarantee that if the attesting block rolls back, "an alternative block will eventually attest to the finality status of the same candidate block" (zebra-crosslink @ `6d02a1b`, `book/src/design/nutshell.md`, lines 116–124). How wallet policy would consume assured finality is unspecified in every source read. Separately, to prevent a terminology collision: the Zally application uses "finalization" for a tally event: after the default tally timeout, end-of-block processing can finalize a round with empty results (`vote-sdk/x/vote/types/keys.go`, `DefaultTallyTimeout`; `vote-sdk/x/vote/module.go`; `valargroup/vote-sdk` @ `cb915f51`). This has no relation to consensus finality.

The open-problem ledger. We close with the ledger, each entry carrying its bin and, where one exists, its revisit trigger.

1. *The authoritative security argument.* The analyzed design (tfl-book, frozen 2024-12-13, analysis incomplete below `security-analysis.md`, line 54), the April built design (Stalled Mode and the outer signature dropped), and current `v13` have diverged with no document proving either prototype. Trigger: completion or supersession of the tfl-book analysis.

2. *The best-chain assumption transfer.* The published argument that Π_{origbc} properties survive modification into Π_{bc} is explicitly a handwave (§6.1). The separate Final Agreement proof paste error has a complete reconstruction in Remark 6.3 and is not left open here.

3. *The parameter gap.* No quantitative relationship among L, σ, network delay, and honest hash and stake exists; the prototypes' $\sigma = 3, L = 7$ lack a security justification; the April source carries an explicit no-verification warning. Their 99-block rollback bound is node policy, against 1000 blocks in mainline Zebra at `d1fd7ad` (§6.5).

4. *The Stalled Mode question.* Whether the eventual ZIP adopts the book's bounded availability or the prototypes' no-Stalled-Mode stance, and whether dropping the rule reopens the unbounded-gap and tail-thrashing hazards the book used to justify it (§6.5). Trigger: a filed Crosslink ZIP.

5. *The BFT engine.* No analysis shows tenderlink — or any concrete Π_{origbft} candidate — satisfies Final Agreement, objective validity, and sound notarization (§6.5).

6. *Machine-checking.* Coverage stops at isValid, with one flagged divergence from the book (Remark 6.7); the bc-side rules, adversarial settings, and liveness are unchecked. Trigger: any extension of the Quint spec or content in `quint-crosslink`.

7. *Accountability machinery.* Current `v13` can burn bonds targeting socially selected finalizers at a configured hard fork, but it does not infer guilt from on-chain evidence. Evidence-based BFT accountability remains a non-adopted [Recommended] change (§6.3); long-range key hygiene improvements remain tfl-book issue #140 (Remark 6.6).

8. *Audit follow-through.* Adoption of the mechanism-design recommendations, and whether the baseline or current prototype avoids phantom security, are unresolved (§6.6). The prototype's

compounding reward and user-activated burn post-date the audit; no follow-up report covers them.

9. *Design-source discrepancies.* The staking epoch is roughly seven days in the nutshell (a one-day staking day plus a six-day locked phase) but five days in the 2025-10-27 blog post and the audit proposal. Current `v13` instead uses 150-block periods, 70-block action windows with five temporary exception heights, and two 75-block lifecycle delays; no source adopts those as production values (`nutshell.md`:64–80; `proposal.md`:58; Table 2).